Lecture1

Definition and historical progress of OR

What is OR?

- OR is a scientific approach to decision problems, which seeks to determine how best to design and operate a system under conditions requiring the allocation of scarce resources.
- OR can be simply defined as applying advanced quantitative techniques to real life problems with the aim of finding the best solution.

- For example:
  - Allocating available resources to various activities in a way which is most effective
  - Increasing the efficiency of any production system
- Extensive applications in engineering, business and public systems. (e.g. Portfolio management, Manpower planning, Transportation Planning.)
- Used extensively by manufacturing and service industries in decision making

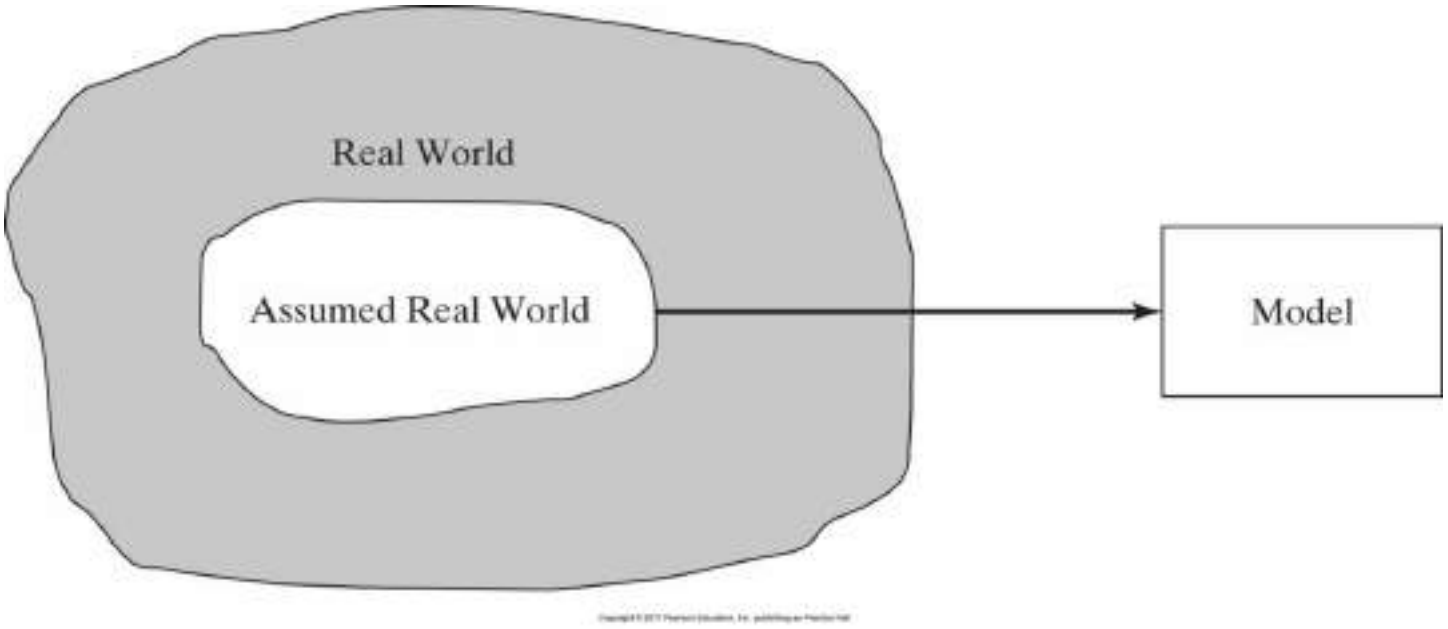
Historical progress of OR

- The first formal activities of OR were initiated in England during World War II, in optimal allocation of scarce war equipment and material.
- There has been significant theoretical developments after 1950.
- And since 1975 with the improvement of computer technology, the progress of OR has become speedy.
- Today OR is also called Management Science.

Phases of an OR Study

- Definition of the problem
- Construction of the model
  - Model is a mathematical representation of a real system.
  - Model is an abstraction of the assumed real world from the real situation by concentrating on the dominant variables that control the behaviour of the real system.
- Solution of the model
- Validation of the model / Sensitivity Analysis
- Implementation of the solution

Figure 1.1 Levels of abstraction in model development [2]



References

[1] *Ahlatçıoğlu, M. , Tiryaki, F., Kantitatif Karar Verme Teknikleri, YTÜ Yayın No: YTÜ.FE.DK-98.0349, İstanbul-1998.*

[2] *H.A.Taha, Operations Research: An Introduction,* Prentice Hall; 9th edition, *Singapore, 2010.*

[3] *OR, I., Operations Research 1, Lecture Notes, Department of Industrial Engineering, Bogaziçi University ,2006.*

Lecture 2

Decision Theory

Decision Making and its basic concepts

Decision Making

a) Decision Making under Certainty
b) Decision Making under Risk
c) Decision Making under Uncertainty

Basic concepts

- **Decision Maker** (DM): a person or a group

  Here, it must be emphasized that the quality of the final decision depends on the abilities of the DM, i.e. more qualified DM a better decision.
- **Objective / Decision Criteria**
- **States (S):** Factors not under control of the DM
- **Actions (A):** Alternatives, Strategies (the variables which are under control of the DM)
- **Results:** Every state and every action causes a result.

Payoff Matrix (Decision Matrix)

- The payoff matrix of a decision problem with *m* alternative actions (a_i , i = 1,2,…,m) and *n* states of nature (S_j ,j = 1,2,…,n) can be represented as

	s_1	s_2	⋯	s_n
a_1	$v(a_1, s_1)$	$v(a_1, s_2)$	⋯	$v(a_1, s_n)$
a_2	$v(a_2, s_1)$	$v(a_2, s_2)$	⋯	$v(a_2, s_n)$
⋮	⋮	⋮	⋮	⋮
a_m	$v(a_m, s_1)$	$v(a_m, s_2)$	⋯	$v(a_m, s_n)$

states

actions

The payoff or outcome associated with action a_i and state S_j is $V(a_i, S_j)$.

An example of constructing a decision matrix

A wholesaler greengrocer considers ordering of apples in boxes with 100 kilos (kg) at the beginning of the week. The purchasing price is 6 TL/kg and the selling price to greengrocers is 10 TL/kg. Also at the end of the week, he sells all the remaining apples to stallholders with the price 4 TL/kg.

Based on his previous experiences, the wholesaler greengrocer knows that he can sell at most 4 boxes of apple in a week. How many boxes of apple should he order? Consider no order and no demand cases.

- $0 \leq a_i \leq 4$, action a_i represents ordering *i* boxes of apples.
- $0 \leq S_j \leq 4$, state S_j represents demanding *j* boxes of apples.
- Purchasing price of 1 box = $6 \times 100 = 600$
- Selling price of 1 box = $10 \times 100 = 1000$
- Selling price of 1 box to stallholders = $4 \times 100 = 400$
- Profit from greengrocers: $1000 - 600 = 400$
- Profit from stallholders: $400 - 600 = -200$

The decision matrix of the greengrocer example

	S_0	S_1	S_2	S_3	S_4
a_0	0	0	0	0	0
a_1	-200	400	400	400	400
a_2	-400	200	800	800	800
a_3	-600	0	600	1200	1200
a_4	-800	-200	400	1000	1600

- $(a_0, S_0) = 0$
- $(a_0, S_1) = 0$
- $(a_0, S_2) = 0$
- $(a_0, S_3) = 0$
- $(a_0, S_4) = 0$

- $(a_1, S_0) = -600 + 400 = -200$
- $(a_1, S_1) = -600 + 1000 = 400$
- $(a_1, S_2) = 400$
- $(a_1, S_3) = 400$
- $(a_1, S_4) = 400$

- $(a_2, S_0) = -400$
- $(a_2, S_1) = 200$
- $(a_2, S_2) = 800$
- $(a_2, S_3) = 800$
- $(a_2, S_4) = 800$

- $(a_3, S_0) = -600$
- $(a_3, S_1) = 0$
- $(a_3, S_2) = 600$
- $(a_3, S_3) = 1200$
- $(a_3, S_4) = 1200$

- $(a_4, S_0) = -800$
- $(a_4, S_1) = -200$
- $(a_4, S_2) = 400$
- $(a_4, S_3) = 1000$
- $(a_4, S_4) = 1600$

$(a_4, S_0) = \underbrace{-600 \times 4}_{\text{cost}} + \underbrace{1000 \times 0}_{\text{sell}} + \underbrace{400 \times 4}_{\text{remain sell stallholders}}$

$= -800$

$(a_4, S_2) = -600 \times 4 + 1000 \times 3 + 400 \times 1$

$= 1000$

$(a_4, S_4) = -600 \times 4 + 1000 \times 3 + 400 \times 1$

$= 1000$

Decision Making under Certainty

- Every parameter of the decision problem is certain.
- The Linear Programming (LP) models (include in 0523051) are conventional examples of decision making under certainty.
- Also, Analytic Hierarchy Process which is designed for prioritizing the alternatives is an example of decision making under certainty.

a payoff matrix under certainty

A \ **S**	S_1
a_1	$V(a_1, S_1)$
a_2	$V(a_2, S_1)$
⋮	⋮
a_n	$V(a_n, S_1)$

- It is obvious which state will occur. So we have only one state. If a_1 is selected, then the result will be $V(a_1, S_1)$. And so on..
- Here the question is which action causes the best result?
- If the elements of payoff matrix represent
  - gain (benefit), the action with will be the optimal.
  - loss (cost), the state with will be the optimal.

For the greengrocer example, suppose that the market demand is 2 boxes of apples. Then the decision matrix becomes:

	S_2	
a_0	0	
a_1	400	
a_2	800	Optimal
a_3	600	
a_4	400	

Analytic Hierarchy Process

- You receive full academic scholarship from three universities.
- To select a university, you specify two main criteria: location and academic reputation
- Give a weight to criteria, determine the scores of universities according to these criteria, and calculate the ranking weights for each university.

- A systematic procedure - Analytic Hierarchy Process
  not included in our course

Objective vs. Criterion

Decision Making under Risk

- Under conditions of risk, the payoffs associated with each decision alternative (states) are described by probability distributions.
- Firstly, the expected value of each action is determined with

$$E(a_i) = \sum_j V(a_i, s_j) * P\{s_j\}$$

- The action which gives the best expected value is the optimal. (Two case: gain or loss)

◊ For the greengrocer example, suppose that the sales record of the last 50 weeks are given as:

Demand of boxes	# of week
0	5
1	10
2	20
3	10
4	5

Demand of boxes	# of week	Probability
0	5	0.1 (5/50)
1	10	0.2 (10/50)
2	20	0.4 (20/50)
3	10	0.2 (10/50)
4	+ 5	+ 0.1 (5/50)
	50	1.0

	S_0	S_1	S_2	S_3	S_4
	0.1	0.2	0.4	0.2	0.1
a_0	0	0	0	0	0
a_1	-200	400	400	400	400
a_2	-400	200	800	800	800
a_3	-600	0	600	1200	1200
a_4	-800	-200	400	1000	1600

- $E(a_0) = 0$
- $E(a_1) = 0.1(-200) + 0.2(400) + 0.4(400) + 0.2(400) + 0.1(400)$
  $= 340$
- $E(a_2) = 560$ *** (Largest Expected Value)
- $E(a_3) = 540$
- $E(a_4) = 400$

Based on these sales records the optimal action is a_2.

Decision Making under Uncertainty

The difference between making a decision under risk and under uncertainty is that in the case of uncertainty, the probability distribution associated with the states $s_j, j = 1, 2, \ldots, n$, is either unknown or cannot be determined. This lack of information has led to the development of the following criteria for analyzing the decision problem:

1. Laplace
2. Minimax (Pessimistic)
3. Savage (Regret)
4. Hurwicz (Optimistic)

These criteria differ in how conservative the decision maker is in the face of uncertainty.

In addition to these four criteria, Minimin criteria will be explained also.

1. Laplace (Equal Probability Criterion)

The **Laplace** criterion is based on the **principle of insufficient reason**. Because the probability distributions are not known, there is no reason to believe that the probabilities associated with the states of nature are different. The alternatives are thus evaluated using the *optimistic* assumption that all states are equally likely to occur—that is, $P\{s_1\} = P\{s_2\} = \cdots = P\{s_n\} = \frac{1}{n}$. Given that payoff $v(a_i, s_j)$ represents gain, the best alternative is the one that yields

$$\max_{a_i}\left\{\frac{1}{n}\sum_{j=1}^{n} v(a_i, s_j)\right\}$$

If $v(a_i, s_j)$ represents loss, then minimization replaces maximization.

	S_0	S_1	S_2	S_3	S_4	Laplace
a_0	0	0	0	0	0	0
a_1	-200	400	400	400	400	280
a_2	-400	200	800	800	800	440
a_3	-600	0	600	1200	1200	480
a_4	-800	-200	400	1000	1600	400

→ $\frac{1}{5}(0+0+\ldots) = 0$

→ $1400 \cdot \frac{1}{5} = 280$

→ expected values optimal gain.

expected value

- a_3 is selected according to the Laplace criterion.

2. Maximin (Minimax) (Pessimistic)

The **maximin (minimax)** criterion is based on the conservative attitude of making the best of the worst possible conditions. If $v(a_i, s_j)$ is loss, then we select the action that corresponds to the *minimax* criterion

$$\min_{a_i}\left\{\max_{s_j} v(a_i, s_j)\right\}$$

If $v(a_i, s_j)$ is gain, we use the *maximin* criterion given by

$$\max_{a_i}\left\{\min_{s_j} v(a_i, s_j)\right\}$$

(Row Max)

	S_0	S_1	S_2	S_3	S_4	*Pessimistic*	
a_0	*0*	*0*	*0*	*0*	*0*	0	→ selected
a_1	*-200*	*400*	*400*	*400*	*400*	-200	
a_2	*-400*	*200*	*800*	*800*	*800*	-400	
a_3	*-600*	*0*	*600*	*1200*	*1200*	-600	
a_4	*-800*	*-200*	*400*	*1000*	*1600*	-800	

2 *. Minimin(Maximax) (Optimistic)

- The minimin (maximax) criterion is based on the optimistic attitude of making the best of the best possible conditions.

- If $V(a_i, S_j)$ is loss, then we select the action that corresponds to the minimin criterion

$$\min_{a_i}\left\{\min_{S_j} V(a_i, S_j)\right\}$$

min value of the decision matrix

- If $V(a_i, S_j)$ is gain, we use the maximax criterion given by

$$\max_{a_i}\left\{\max_{S_j} V(a_i, S_j)\right\}$$

max value of the decision matrix (Row Max)

	S_0	S_1	S_2	S_3	S_4	*Optimistic*
a_0	0	0	0	0	0	0
a_1	-200	400	400	400	400	400
a_2	-400	200	800	800	800	800
a_3	-600	0	600	1200	1200	1200
a_4	-800	-200	400	1000	1600	1600

→ Selected

3. Savage

The **Savage regret** criterion aims at moderating conservatism in the *minimax (maximin)* criterion by replacing the (gain or loss) payoff matrix $v(a_i, s_j)$ with a *loss* (or regret) $r(a_i, s_j)$ matrix, using the following transformation:

$$r(a_i, s_j) = \begin{cases} v(a_i, s_j) - \min\limits_{a_k}\{v(a_k, s_j)\}, & \text{if } v \text{ is loss} \\ \max\limits_{a_k}\{v(a_k, s_j)\} - v(a_i, s_j), & \text{if } v \text{ is gain} \end{cases}$$

Decision matrix

	S_0	S_1	S_2	S_3	S_4
a_0	0	0	0	0	0
a_1	-200	400	400	400	400
a_2	-400	200	800	800	800
a_3	-600	0	600	1200	1200
a_4	-800	-200	400	1000	1600

max (column) − current

Regret matrix

	S_0	S_1	S_2	S_3	S_4	Pessimistic
a_0	0	400	800	1200	1600	1600
a_1	200	0	400	800	1200	1200
a_2	400	200	0	400	800	800
a_3	600	400	200	0	400	600
a_4	800	600	400	200	0	800

→ selected

A regret matrix is always a loss type matrix !!!

Pessimistic criterion is applied to regret matrix.

To show why the Savage criterion "moderates" the minimax (maximin) criterion, consider the following *loss*, $v(a_i, s_j)$, matrix

	s_1	s_2	Row max
a_1	\$11,000	\$90	\$11,000
a_2	\$10,000	\$10,000	**\$10,000** ← Minimax

The application of the minimax criterion shows that a_2, with a definite loss of \$10,000, is preferable. However, we may choose a_1, because there is a chance of limiting the loss to \$90 only if s_2 is realized.

Let us see what happens if we use the following regret, $r(a_i, v_j)$, matrix instead:

	s_1	s_2	Row max
a_1	\$1000	\$0	**\$1000** ← Minimax
a_2	\$0	\$9910	\$9910

The minimax criterion, when applied to the regret matrix, will select a_1, as desired.

4. Hurwicz

The last test to be considered is **Hurwicz** criterion, which is designed to reflect decision-making attitudes, ranging from the most optimistic to the most pessimistic (or conservative). Define $0 \leq \alpha \leq 1$, and assume that $v(a_i, s_j)$ represents gain. Then the selected action must be associated with

$$\max_{a_i}\left\{\alpha \max_{s_j} v(a_i, s_j) + (1 - \alpha)\min_{s_j} v(a_i, s_j)\right\}$$

The parameter α is called the **index of optimism**. If $\alpha = 0$, the criterion is conservative because it applies the regular minimax criterion. If $\alpha = 1$, the criterion produces optimistic results because it seeks *the best of the best* conditions. We can adjust the degree of optimism (or pessimism) through a proper selection of the value of α in the specified $(0, 1)$ range. In the absence of strong feeling regarding optimism and pessimism, $\alpha = .5$ may be an appropriate choice.

If $v(a_i, s_j)$ represents loss, then the criterion is changed to

$$\min_{a_i}\left\{\alpha \min_{s_j} v(a_i, s_j) + (1 - \alpha)\max_{s_j} v(a_i, s_j)\right\}$$

Hurwicz : α (Optimistic) + (1-α) (Pessimistic)

$$\alpha \begin{bmatrix} 0 \\ 400 \\ 800 \\ 1200 \\ 1600 \end{bmatrix} + (1-\alpha) \begin{bmatrix} 0 \\ -200 \\ -400 \\ -600 \\ -800 \end{bmatrix} = \begin{bmatrix} 0 \\ 600\alpha - 200 \\ 1200\alpha - 400 \\ 1800\alpha - 600 \\ 2400\alpha - 800 \end{bmatrix} = \begin{bmatrix} 0 \\ A \\ 2A \\ 3A \\ 4A \end{bmatrix} \quad \text{where } A = 600\alpha - 200$$

$(\max\{0, A, 2A, 3A, 4A\})$

Since the payoff matrix represents gain and the critical point is $1/3$

for $\alpha \in [0, 1/3]$, a_0 is optimal.

for $\alpha \in [1/3, 1]$, a_4 is optimal.

Recall that, a_0 is the optimal desicion for the pessimistic criterion and a_4 is the optimal desicion for the optimistic criterion.

EX) $\alpha = 0.7 \Rightarrow$

$$H = \begin{bmatrix} 0 \\ 420 - 200 \\ 840 - 400 \\ \vdots \end{bmatrix} = \begin{bmatrix} 0 \\ 220 \\ 440 \\ \vdots \end{bmatrix}$$

	S_0	S_1	S_2	S_3	S_4	Laplace	Pessimistic	Optimistic	Hurwicz ($\alpha = 0.7$)
a_0	0	0	0	0	0	0	0 **	0	0
a_1	-200	400	400	400	400	280	-200	400	220
a_2	-400	200	800	800	800	440	-400	800	440
a_3	-600	0	600	1200	1200	480 **	-600	1200	660
a_4	-800	-200	400	1000	1600	400	-800	1600 **	880 **

	S_0	S_1	S_2	S_3	S_4	Pessimist
a_0	0	400	800	1200	1600	1600
a_1	200	0	400	800	1200	1200
a_2	400	200	0	400	800	800
a_3	600	400	200	0	400	400
a_4	800	600	400	200	0	0 **

Example

National Outdoors School (NOS) is preparing a summer campsite in the heart of Alaska to train individuals in wilderness survival. NOS estimates that attendance can fall into one of four categories: 200, 250, 300, and 350 persons. The cost of the campsite will be the smallest when its size meets the demand exactly. Deviations above or below the ideal demand levels incur additional costs resulting from building surplus (unused) capacity or losing income opportunities when the demand is not met. Letting a_1 to a_4 represent the sizes of the campsites (200, 250, 300, and 350 persons) and s_1 to s_4 the level of attendance, the following table summarizes the cost matrix (in thousands of dollars) for the situation.

	s_1	s_2	s_3	s_4
a_1	5	10	18	25
a_2	8	7	12	23
a_3	21	18	12	21
a_4	30	22	19	15

Laplace. Given $P\{s_j\} = \frac{1}{4}, j = 1$ to 4, the expected values for the different actions are computed as

$$E(a_1) = \frac{1}{4}(5 + 10 + 18 + 25) = \$14,500$$

$$E(a_2) = \frac{1}{4}(8 + 7 + 12 + 23) = \$12,500 \rightarrow \text{Optimum}$$

$$E(a_3) = \frac{1}{4}(21 + 18 + 12 + 21) = \$18,000$$

$$E(a_4) = \frac{1}{4}(30 + 22 + 19 + 15) = \$21,500$$

Minimax. The minimax criterion produces the following matrix:

	s_1	s_2	s_3	s_4	Row Max
a_1	5	10	18	25	25
a_2	8	7	12	23	23
a_3	21	18	12	21	21 → Minimax
a_4	30	22	19	15	30

Savage. The regret matrix is determined by subtracting 5, 7, 12, and 15 from columns 1 to 4, respectively. Thus,

	s_1	s_2	s_3	s_4
a_1	5	10	18	25
a_2	8	7	12	23
a_3	21	18	12	21
a_4	30	22	19	15

⇨

	s_1	s_2	s_3	s_4	Row max
a_1	0	3	6	10	10
a_2	3	0	0	8	8 → minimax
a_3	16	11	0	6	16
a_4	25	15	7	0	25

Hurwicz. The following table summarizes the computations.

Alternative	Row min (Optimistic)	Row max (Pessimistic)	$\alpha(\text{Row min}) + (1 - \alpha)(\text{Row max})$
a_1	5	25	$25 - 20\alpha$ → 25 − 10 = 15
a_2	7	23	$23 - 16\alpha$ 23 − 8 = 15
a_3	12	21	$21 - 9\alpha$
a_4	15	30	$30 - 15\alpha$

Using an appropriate α, we can determine the optimum alternative. For example, at $\alpha = .5$ (→ 1/2), either a_1 or a_2 will yield the optimum, and at $\alpha = .25$, a_3 is the optimum.

Minimin: The minimin criterion produces the following decision:

	s_1	s_2	s_3	s_4
a_1	5	10	18	25
a_2	8	7	12	23
a_3	21	18	12	21
a_4	30	22	19	15

Homework (DT)

1- Hank is an intelligent student and usually makes good grades, provided that he can review the course material the night before the test. For tomorrow's test, Hank is faced with a small problem. His fraternity brothers are having an all-night party in which he would like to participate. Hank has three options:

a_1 = Party all night

a_2 = Divide the night equally between studying and partying

a_3 = Study all night

Tomorrow's exam can be easy (s_1), moderate (s_2), or tough (s_3), depending on the professor's unpredictable mood. Hank anticipates the following scores:

	s_1	s_2	s_3
a_1	85	60	40
a_2	92	85	81
a_3	100	88	82

(a) Recommend a course of action for Hank (based on each of the four criteria of decisions under uncertainty).

(b) Suppose that Hank is more interested in the letter grade he will get. The dividing scores for the passing letter grades A to D are 90, 80, 70, and 60, respectively. Would this attitude toward grades call for a change in Hank's course of action?

2- For the upcoming planting season, Farmer McCoy can plant corn (a_1), plant wheat (a_2), plant soybeans (a_3), or use the land for grazing (a_4). The payoffs associated with the different actions are influenced by the amount of rain: heavy rainfall (s_1), moderate rainfall (s_2), light rainfall (s_3), or drought season (s_4).

The payoff matrix (in thousands of dollars) is estimated as

	s_1	s_2	s_3	s_4
a_1	−20	60	30	−5
a_2	40	50	35	0
a_3	−50	100	45	−10
a_4	12	15	15	10

Develop a course of action for Farmer McCoy.

References:

[1] *Ahlatçıoğlu, M. , Operations Research , Lecture Notes, Department of Mathematical Enginieering, Yildiz Technical University ,2006.*

[2] *H.A.Taha, Operations Research: An Introduction,* Prentice Hall; 9th edition, *Singapore, 2010.*

[3] *Ahlatçıoğlu, M. , Tiryaki, F., Kantitatif Karar Verme Teknikleri, YTÜ Yayın No: YTÜ.FE.DK-98.0349, İstanbul-1998.*

Lecture 3

An introduction to Network Theory

Network models

Network definitions

Network Models

A multitude of operations research situations can be modeled and solved as networks (nodes connected by branches):

1. Design of an offshore natural-gas pipeline network connecting well heads in the Gulf of Mexico to an inshore delivery point. The objective of the model is to minimize the cost of constructing the pipeline.

minimal spanning tree

2. Determination of the shortest route between two cities in an existing network of roads.

shortest - route algorithm

3. Determination of the maximum capacity (in tons per year) of a coal slurry pipeline network joining coal mines in Wyoming with power plants in Houston. (Slurry pipelines transport coal by pumping water through specially designed pipes.)

maximal - flow algorithm

4. Determination of the time schedule (start and completion dates) for the activities of a construction project.

critical path (CPM) algorithm

management

5. Determination of the minimum-cost flow schedule from oil fields to refineries through a pipeline network.

min - cost capacitated network algorithm

2

Network Definitions

A network consists of a set of **nodes** linked by **arcs** (or branches). The notation for describing a network is (N, A), where N is the set of nodes and A is the set of arcs. As an illustration, the network in Figure 6.1 is described as

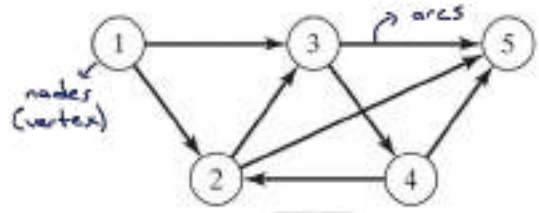


FIGURE 6.1 Example of (N, A) Network

$N = \{1, 2, 3, 4, 5\}$

$A = \{(1,2), (1,3), (2,3), (2,5), (3,4), (3,5), (4,2), (4,5)\}$

Associated with each network is a **flow** (e.g., oil products flow in a pipeline and automobile traffic flows in highways). In general, the flow in a network is limited by the capacity of its arcs, which may be finite or infinite.

3

An arc is said to be **directed** or **oriented** if it allows positive flow in one direction and zero flow in the opposite direction. A **directed network** has all directed arcs.

A **path** is a sequence of distinct arcs that join two nodes through other nodes regardless of the direction of flow in each arc. A path forms a **cycle** or a **loop** if it connects a node to itself through other nodes. For example, in Figure 6.1, the arcs (2, 3), (3, 4), and (4, 2) form a cycle.

A **connected network** is such that every two distinct nodes are linked by at least one path. The network in Figure 6.1 demonstrates this type of network. A **tree** is a *cycle-free* connected network comprised of a *subset* of all the nodes, and a **spanning tree** is a tree that links *all* the nodes of the network. Figure 6.2 provides examples of a tree and a spanning tree from the network in Figure 6.1.

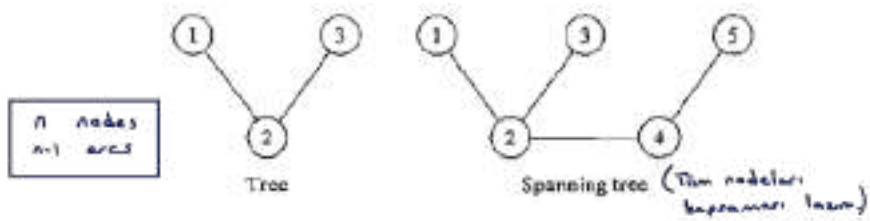


Figure 6.2. Examples of a tree and a spanning tree

4

Example 6.1-1 (Bridges of Königsberg)

The Prussian city of Königsberg (now Kalingrad in Russia) was founded in 1254 on the banks of river Pergel with seven bridges connecting its four sections (labeled *A*, *B*, *C*, and *D*) as shown in Figure 6.3. A problem circulating among the inhabitants of the city was to find out if a *round trip* of the four sections could be made with each bridge being crossed exactly once. No limits were set on the number of times any of the four sections could be visited.

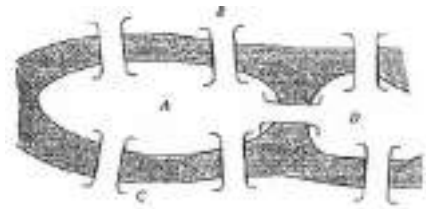


Figure 6.3. Bridges of Königsberg

In the mid-eighteenth century, the famed mathematician Leonhard Euler developed a special "path construction" argument to prove that it was impossible to make such a trip. Later, in the early nineteenth century the same problem was solved by representing the situation as a network in which each of the four sections (*A*, *B*, *C*, and *D*) is a node and each bridge is an arc joining applicable nodes, as shown in Figure 6.4.

5

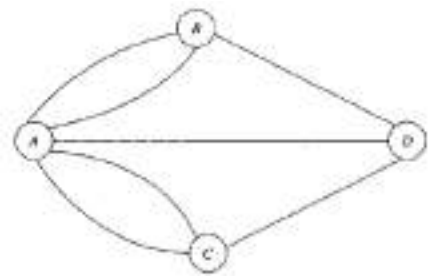


Figure 6.4. Bridges of Königsberg

The network-based solution is that the desired round trip (starting and ending in one section of the city) is impossible, because there are four nodes and each is associated with an *odd* number of arcs, which does not allow distinct entrance and exit (and hence distinct use of the bridges) to each section of the city.[1] The example demonstrates how the solution of the problem is facilitated by using network representation.

[1] General solution: A tour exists if all nodes have an even number of branches or if exactly two nodes have an odd number of branches. Else no tour exists. See B. Hopkins and R. Wilson, "The Truth about Königsberg," *College Math Journal*, Vol. 35, No. 3, pp. 198–207, 2004.

6

PROBLEM SET 6.1A

1. For each network in Figure 6.5 determine (a) a path, (b) a cycle, (c) a tree, and (d) a spanning tree.
2. Determine the sets N and A for the networks in Figure 6.5.

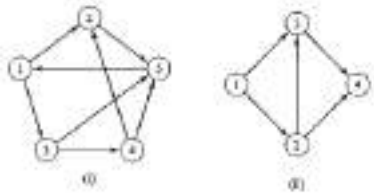


FIGURE 6.5

3. Draw the network defined by

$$N = \{1, 2, 3, 4, 5, 6\}$$

$$A = \{(1, 2), (1, 5), (2, 3), (2, 4), (3, 4), (3, 5), (4, 3), (4, 6), (5, 2), (5, 6)\}$$

Reference: *H.A.Taha, Operations Research: An Introduction,* Prentice Hall; 9th edition, *Singapore, 2010.* 7

Lecture 4

Minimal Spanning Tree

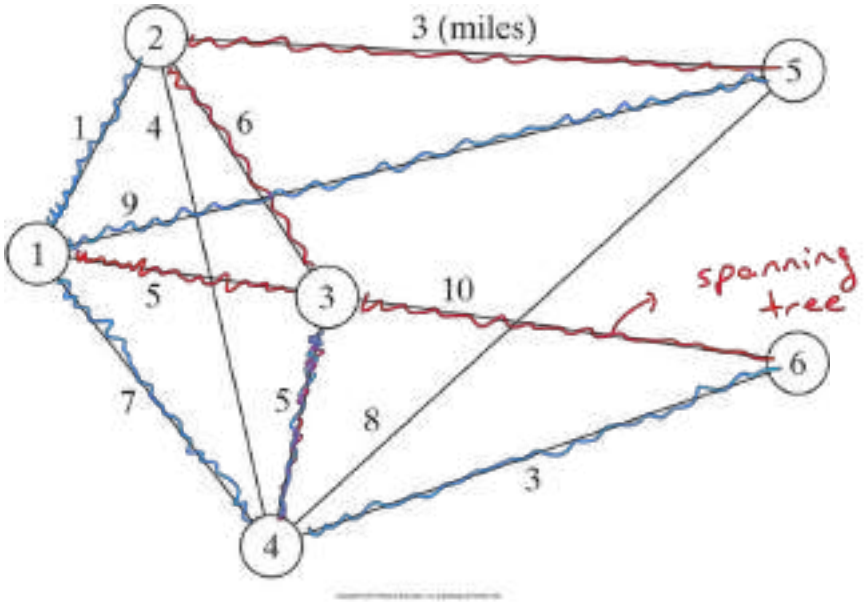
MINIMAL SPANNING TREE ALGORITHM

$N = \{1, 2, 3, 4, 5, 6\}$

The minimal spanning tree algorithm deals with linking the nodes of a network, directly or indirectly, using the shortest total length of connecting branches. A typical application occurs in the construction of paved roads that link several rural towns. The road between two towns may pass through one or more other towns. The most economical design of the road system calls for minimizing the total miles of paved roads, a result that is achieved by implementing the minimal spanning tree algorithm.

Example 6.2-1

Midwest TV Cable Company is in the process of providing cable service to five new housing development areas. Figure 6.6 depicts possible TV linkages among the five areas. The cable miles are shown on each arc. Determine the most economical cable network.



$ST_1 = 5 + 10 + 5 + 6 + 3$

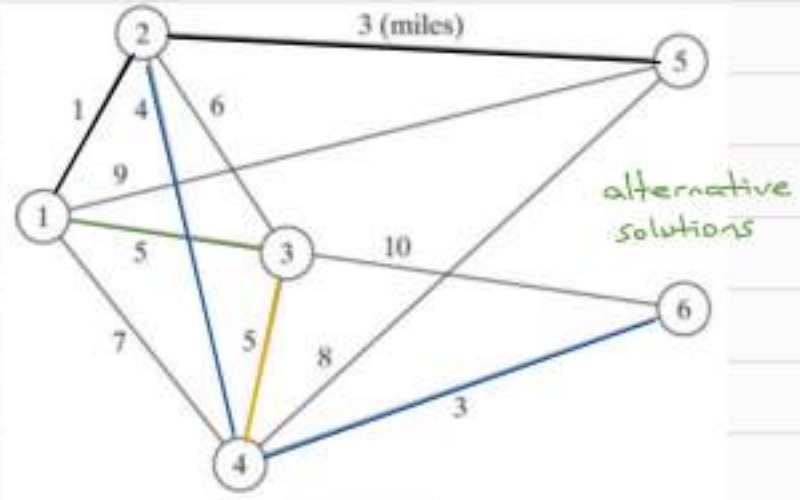
$= 29$

$ST_2 = 1 + 9 + 7 + 5 + 3$

$= 25$

$ST_i = ?$

Figure 6.6 Cable connections for Midwest TV Cable Company



$C_0 = \emptyset$ $\quad \bar{C}_0 = N = \{1,2,3,4,5,6\}$

$C_1 = 1$ $\quad \bar{C}_1 = N \setminus C_1 = \{2,3,4,5,6\}$ $\quad i^* = ?$

$\boxed{(1,2)}_{1}, (1,3)_{5}, (1,4)_{7}, (1,5)_{9}, \cancel{(1,6)}$

$j^* = 2$

$C_2 = \{1,2\}$ $\quad \bar{C}_2 = \{3,4,5,6\}$

$(1,3)_{5}, (1,4)_{7}, (1,5)_{9}, \cancel{(1,6)}, (2,3)_{6}, (2,4)_{4}, \boxed{(2,5)_{3}}, \cancel{(2,6)}$

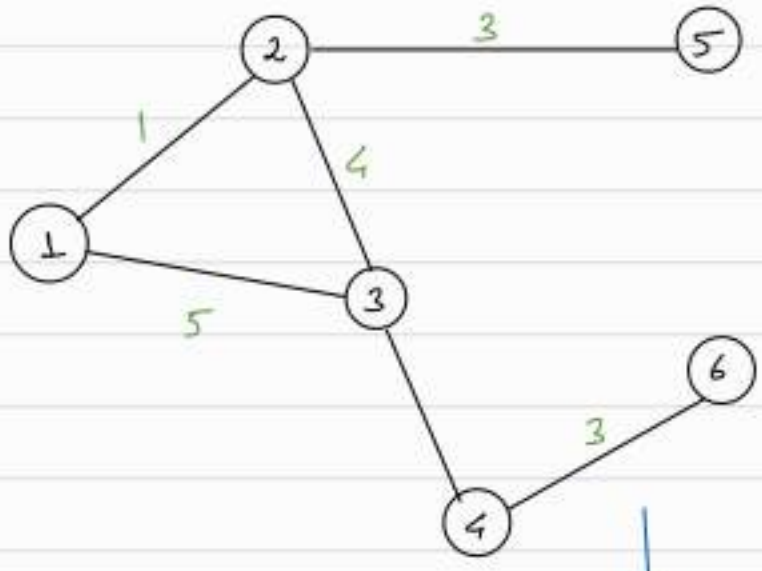
$j^* = 5$

$C_3 = \{1,2,5\}$ $\quad \bar{C}_3 = \{3,4,6\}$ $\quad j^* = 4$

$C_4 = \{1,2,5,4\}$ $\quad \bar{C}_4 = \{3,6\}$ $\quad j^* = 6$

$C_5 = \{1,2,5,4,6\}$ $\quad \bar{C}_5 = \{3\}$

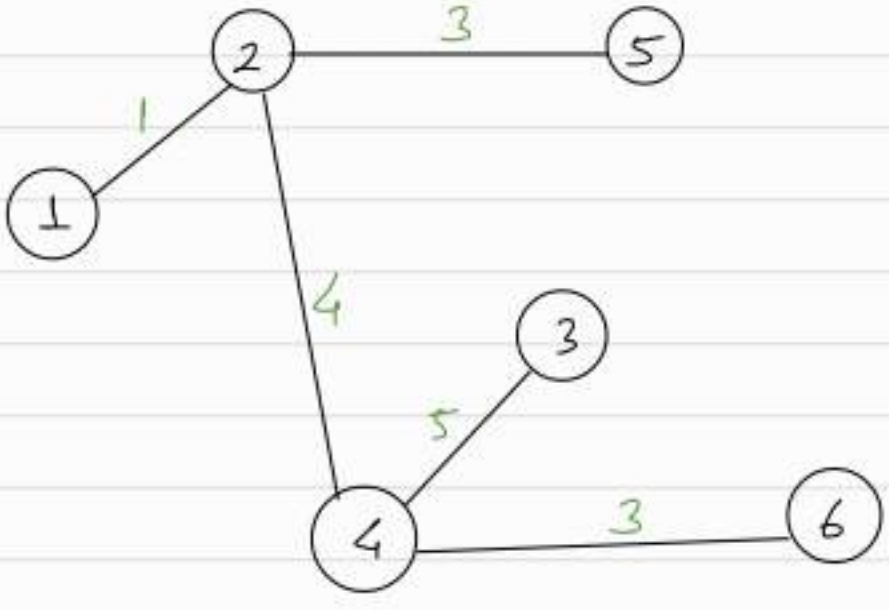
$C_6 = N$ $\quad \bar{C}_6 = \emptyset$ $\quad$ STOP



minimum total distance = 1 + 4 + 3 + 5 + 3

= 16

ST_2 :



The steps of the procedure are given as follows. Let $N = \{1, 2, \ldots, n\}$ be the set of nodes of the network and define

C_k = Set of nodes that have been permanently connected at iteration k
$\overline{C}_k$ = Set of nodes as yet to be connected permanently after iteration k

Step 0. Set $C_0 = \emptyset$ and $\overline{C}_0 = N$.

Step 1. Start with *any* node i in the unconnected set $\overline{C}_0$ and set $C_1 = \{i\}$, which renders $\overline{C}_1 = N - \{i\}$. Set $k = 2$.

General step *k*. Select a node, j^*, in the unconnected set $\overline{C}_{k-1}$ that yields the shortest arc to a node in the connected set C_{k-1}. Link j^* permanently to C_{k-1} and remove it from $\overline{C}_{k-1}$; that is,

$$C_k = C_{k-1} + \{j^*\}, \overline{C}_k = \overline{C}_{k-1} - \{j^*\}$$

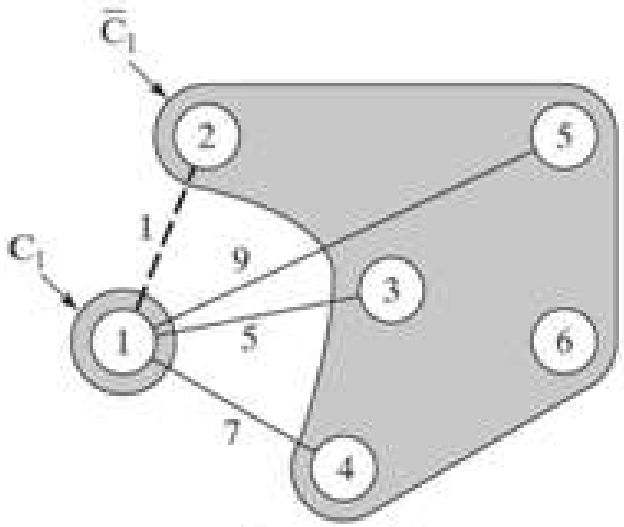
If the set of unconnected nodes, $\overline{C}_k$, is empty, stop. Otherwise, set $k = k + 1$ and repeat the step.

The algorithm starts at node 1 (any other node will do as well), which gives

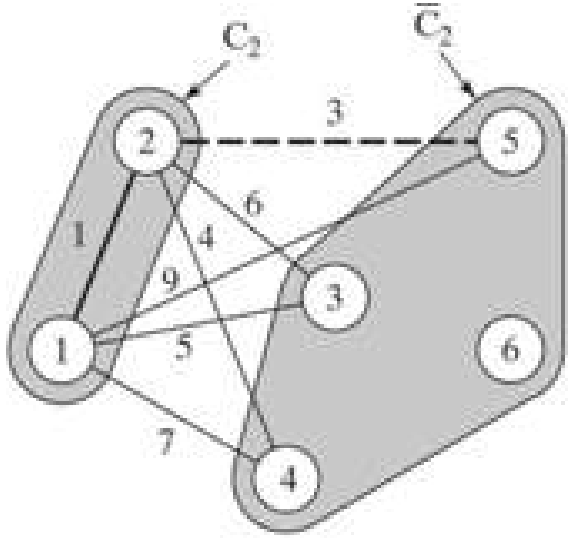
$$C_1 = \{1\}, \overline{C}_1 = \{2, 3, 4, 5, 6\}$$

The iterations of the algorithm are summarized in Figure 6.7. The thin arcs provide all the candidate links between C and $\overline{C}$. The thick branches represent the permanent links between the nodes of the connected set C, and the dashed branch represents the new (permanent) link added at each iteration. For example, in iteration 1, branch (1, 2) is the shortest link (= 1 mile) among all the candidate branches from node 1 to nodes 2, 3, 4, 5, and 6 of the unconnected set $\overline{C}_1$. Hence, link (1, 2) is made permanent and $j^* = 2$, which yields

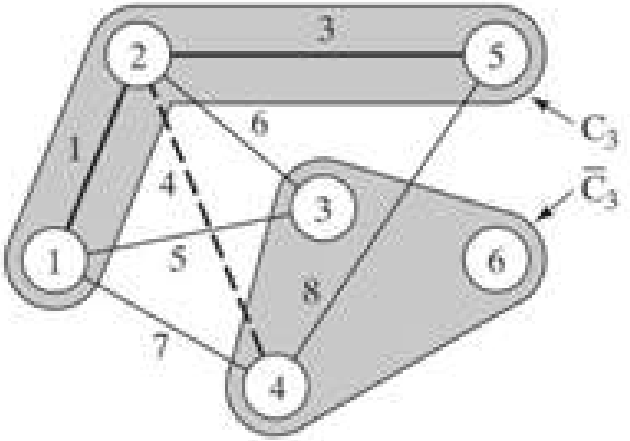
$$C_2 = \{1, 2\}, \overline{C}_2 = \{3, 4, 5, 6\}$$



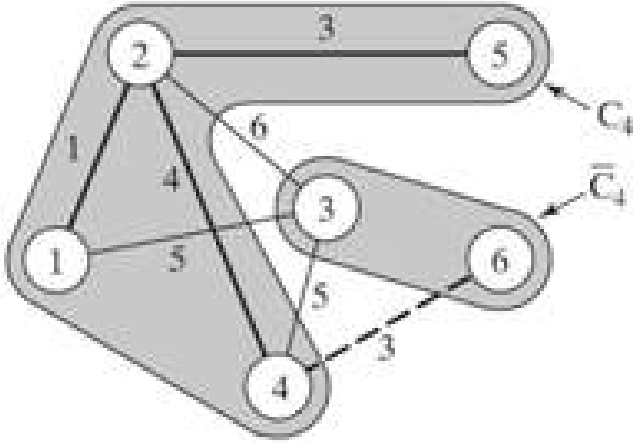
Iteration 1



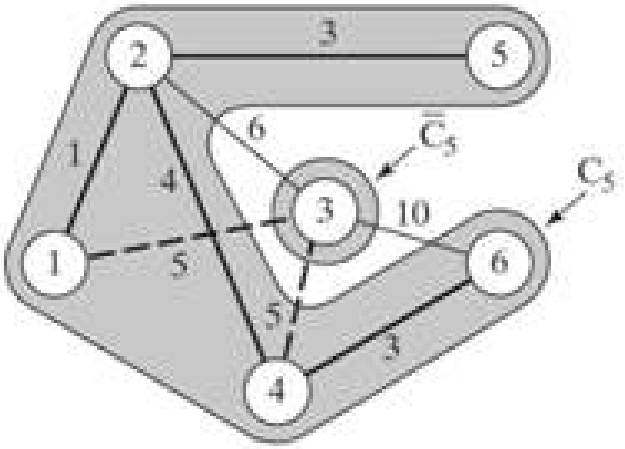
Iteration 2



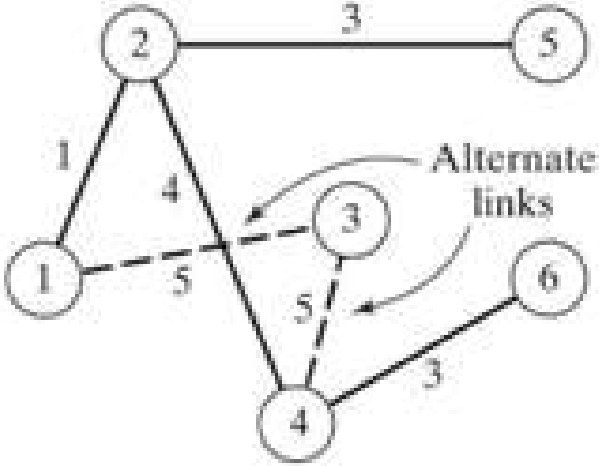
Iteration 3



Iteration 4



Iteration 5



Iteration 6
(Minimal spanning tree)

The solution is given by the minimal spanning tree shown in iteration 6 of Figure 6.7. The resulting minimum cable miles needed to provide the desired cable service are $1 + 3 + 4 + 3 + 5 = 16$ miles.

PROBLEM SET 6.2A

1. Solve Example 6.2-1 starting at node 5 (instead of node 1), and show that the algorithm produces the same solution.
2. Determine the minimal spanning tree of the network of Example 6.2-1 under each of the following separate conditions:
   - *(a) Nodes 5 and 6 are linked by a 2-mile cable.
   - (b) Nodes 2 and 5 cannot be linked.
   - (c) Nodes 2 and 6 are linked by a 4-mile cable.
   - (d) The cable between nodes 1 and 2 is 8 miles long.
   - (e) Nodes 3 and 5 are linked by a 2-mile cable.
   - (f) Node 2 cannot be linked directly to nodes 3 and 5.

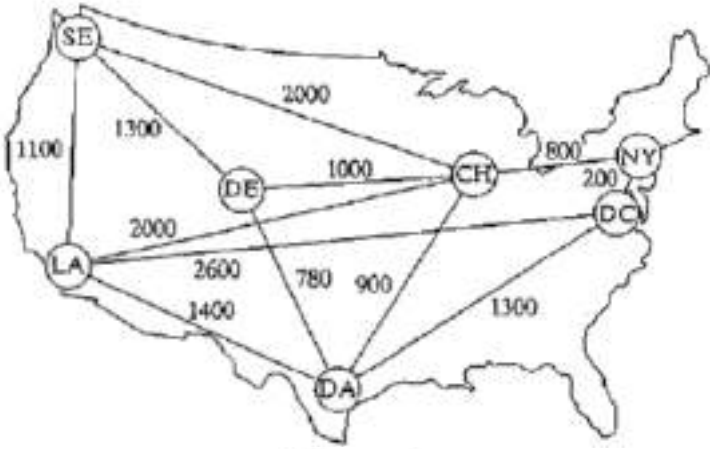


FIGURE 6.8

3. In intermodal transportation, loaded truck trailers are shipped between railroad terminals on special flatbed carts. Figure 6.8 shows the location of the main railroad terminals in the United States and the existing railroad tracks. The objective is to decide which tracks should be "revitalized" to handle the intermodal traffic. In particular, the Los Angeles (LA) terminal must be linked directly to Chicago (CH) to accommodate expected heavy traffic. Other than that, all the remaining terminals can be linked, directly or indirectly, such that the total length (in miles) of the selected tracks is minimized. Determine the segments of the railroad tracks that must be included in the revitalization program.

Reference: *H.A.Taha, Operations Research: An Introduction*, Prentice Hall; 9th edition, *Singapore, 2010.*

Homework (MST)

- Solve the same example starting at node 5 (instead of node 1), and show that the algorithm produces the same solution.

Reference: [1] *H.A.Taha, Operations Research: An Introduction,* Prentice Hall; 9th edition, *Singapore, 2010.*

Lecture 5

Shortest-Route

(Shortest Path)

Shortest-route(SR) problem

- The shortest route problem determines the shortest route between a source and a destination in a transportation network.
- Driving directions on web mapping websites like Mapquest or Google Maps.
- Source and destination nodes
  - The **single-source shortest path problem**, in which we have to find shortest routes from a source node to all other nodes.
  - The **all-pairs shortest path problem**, in which we have to find shortest routes between every pair of nodes.
- The sign of the arc capacities

dijkstra

floyd solved by them

Shortest-Route Algorithms

This section presents two algorithms for solving both cyclic (i.e., containing loops) and acyclic networks:

1. Dijkstra's algorithm
2. Floyd's algorithm

Dijkstra's algorithm is designed to determine the shortest routes between the source node and every other node in the network. Floyd's algorithm is more general because it allows the determination of the shortest route between *any* two nodes in the network.

Dijkstra's Algorithm. Let u_i be the shortest distance from source node 1 to node i, and define d_{ij} (≥ 0) as the length of arc (i, j). Then the algorithm defines the label for an immediately succeeding node j as

$$[u_j, i] = [u_i + d_{ij}, i], d_{ij} \geq 0$$

The label for the starting node is [0, —], indicating that the node has no predecessor.

Node labels in Dijkstra's algorithm are of two types: *temporary* and *permanent*. A temporary label is modified if a shorter route to a node can be found. If no better route can be found, the status of the temporary label is changed to permanent.

Dijkstra's Algorithm

Step 0. Label the source node (node 1) with the *permanent* label $[0, —]$. Set $i = 1$.

Step i. **(a)** Compute the *temporary* labels $[u_i + d_{ij}, i]$ for each node j that can be reached from node i, *provided j is not permanently labeled.* If node j is already labeled with $[u_j, k]$ through another node k and if $u_i + d_{ij} < u_j$, replace $[u_j, k]$ with $[u_i + d_{ij}, i]$.

(b) If *all* the nodes have *permanent* labels, stop. Otherwise, select the label $[u_r, s]$ having the shortest distance $(= u_r)$ among all the *temporary* labels (break ties arbitrarily). Set $i = r$ and repeat step i.

Example 6.3-4

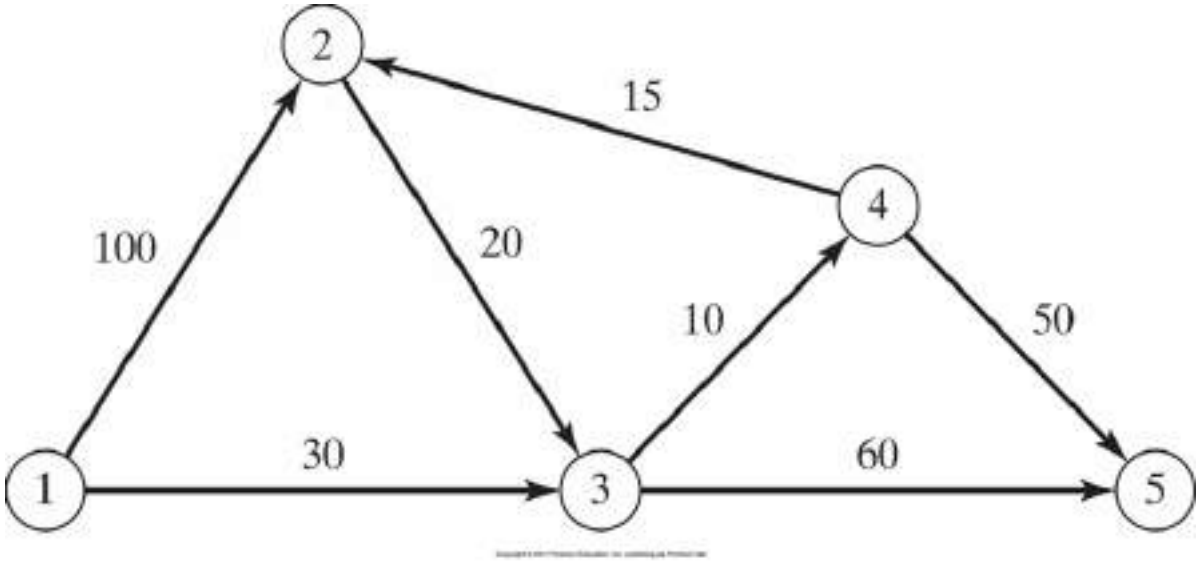
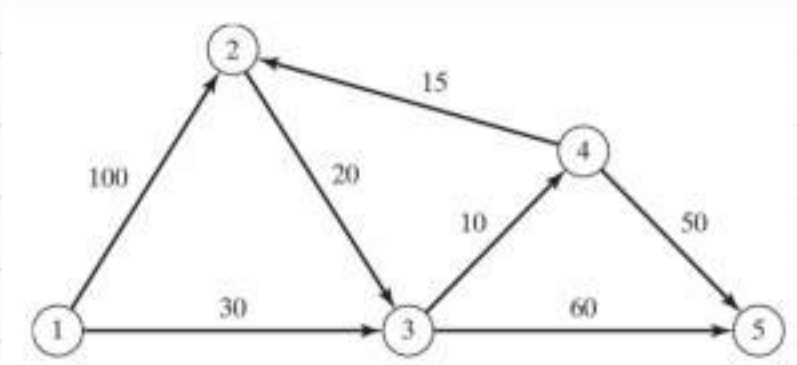


Figure 6.15 Network Example for Dijkstra's shortest-route algorithm

The network in Figure 6.15 gives the permissible routes and their lengths in miles between city 1 (node 1) and four other cities (nodes 2 to 5). Determine the shortest routes between city 1 and each of the remaining four cities.



1 → othercities

Iteration

Nodes	Label	Status
1	[0, -]	P
2	[0+100, 1]	T
3	[0+30, 1]	T

1	[0, -]	P
2	[100, 1]	T
3	[30, 1]	P
4	$[v_3 + d_{34}, 3] = [30+10, 3]$	T ★
5	[30+60, 3]	T

1	[0, -]	P
2	[100, 1] or [40+15, 4]	T ★
3	[30, 1]	P
4	[40, 3]	P
5	[90, 3] or [40+50, 4]	T

1	[0, -]	P
2	[55, 4]	P
3	[30, 1]	P
4	[40, 3]	P
5	[90, 3] or [90, 4]	T ★ → P

stop because all node permanent

Backtracking:

Shortest path

5 → [90, 3] → 3 → [30, 1] → 1

1 : 3 : 5 → 90

[90, 4] → 4 → [40, 3] → 3 → [30, 1] → 1

1 : 3 : 4 : 5 → 90

Source Node : 1 S.P. from 1 to 5

Shortest distance 1 → 5 : 90

Iteration 0. Assign the *permanent* label [0, —] to node 1.

Iteration 1. Nodes 2 and 3 can be reached from (the last permanently labeled) node 1. Thus, the list of labeled nodes (temporary and permanent) becomes

Node	Label	Status
1	[0, -]	Permanent
2	[0+100, 1] = [100, 1]	Temporary
3	[0+30, 1] = [30, 1]	Temporary

For the two temporary labels [100, 1] and [30, 1], node 3 yields the smaller distance ($u_3 = 30$). Thus, the status of node 3 is changed to permanent.

Iteration 2. Nodes 4 and 5 can be reached from node 3, and the list of labeled nodes becomes

Node	Label	Status
1	[0, -]	Permanent
2	[100, 1]	Temporary
3	[30, 1]	Permanent
4	[30+10, 3] = [40, 3]	Temporary
5	[30+60, 3] = [90, 3]	Temporary

The status of the temporary label [40, 3] at node 4 is changed to permanent ($u_4 = 40$).

Iteration 3. Nodes 2 and 5 can be reached from node 4. Thus, the list of labeled nodes is updated as

Node	Label	Status
1	[0, -]	Permanent
2	[40+15, 4] = [55, 4]	Temporary
3	[30, 1]	Permanent
4	[40, 3]	Permanent
5	[90, 3] or [40+50, 4] = [90, 4]	Temporary

Node 2's temporary label [100, 1] obtained in iteration 1 is changed to [55, 4] in iteration 3 to indicate that a shorter route has been found through node 4. Also, in iteration 3, node 5 has two alternative labels with the same distance $u_5 = 90$. The list for iteration 3 shows that the label for node 2 is now permanent.

Iteration 4. Only node 3 can be reached from node 2. However, node 3 has a permanent label and cannot be relabeled. The new list of labels remains the same as in iteration 3 except that the label at node 2 is now permanent. This leaves node 5 as the only temporary label. Because node 5 does not lead to other nodes, its status is converted to permanent, and the process ends.

The computations of the algorithm can be carried out more easily on the network, as Figure 6.16 demonstrates.

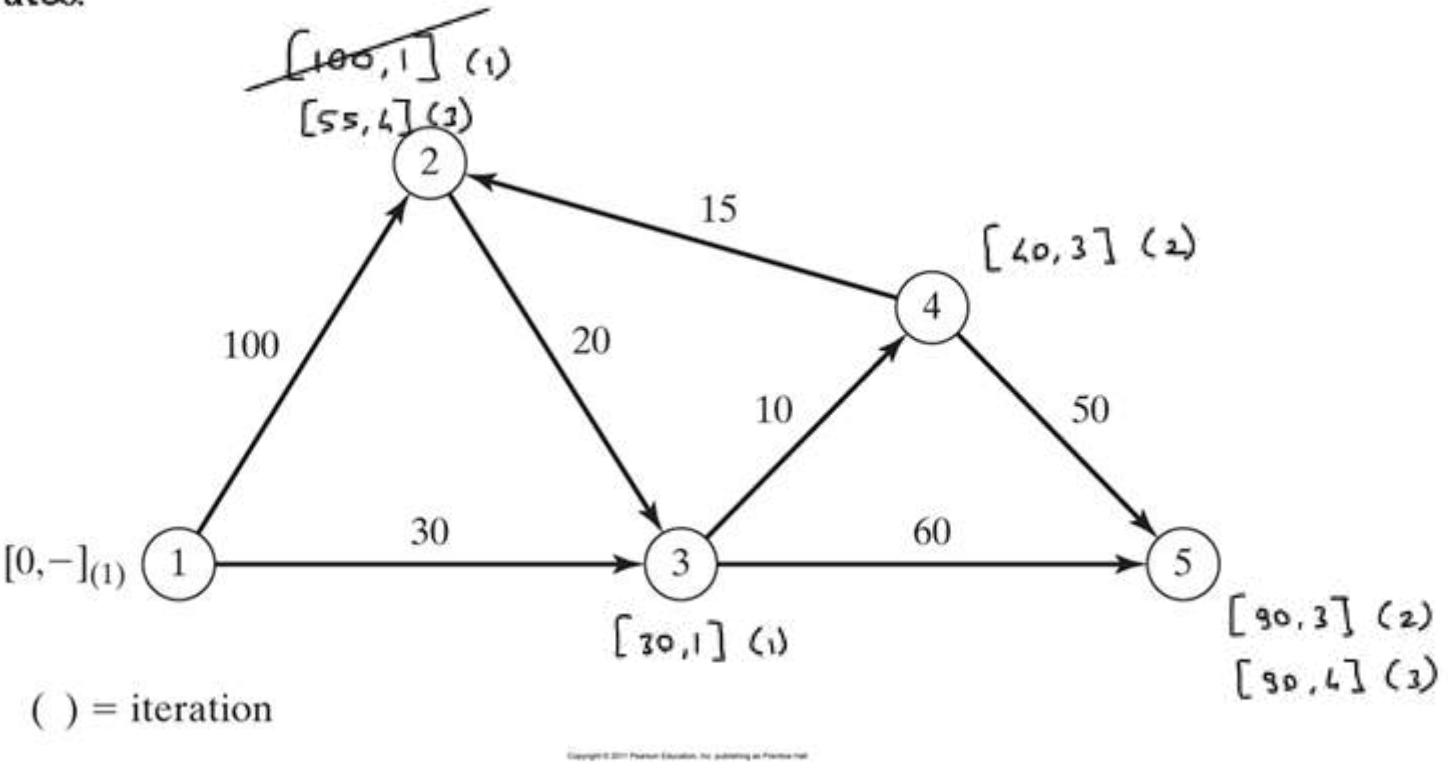


Figure 6.16 Dijkstra's labeling procedure

The shortest route between nodes 1 and any other node in the network is determined by starting at the desired destination node and backtracking through the nodes using the information given by the permanent labels. For example, the following sequence determines the shortest route from node 1 to node 2:

(2) → [55, 4] → (4) → [40, 3] → 3 → [30, 1] → (1)

Thus, the desired route is 1 → 3 → 4 → 2 with a total length of 55 miles.

Homework (SR-2)

1) The network in Figure 6.17 gives the distances in miles between pairs of cities 1, 2, . . . , and 8. Use Dijkstra's algorithm to find the shortest route between the following cities:

(a) Cities 1 and 8

(b) Cities 1 and 6

(c) Cities 4 and 8

(d) Cities 2 and 6

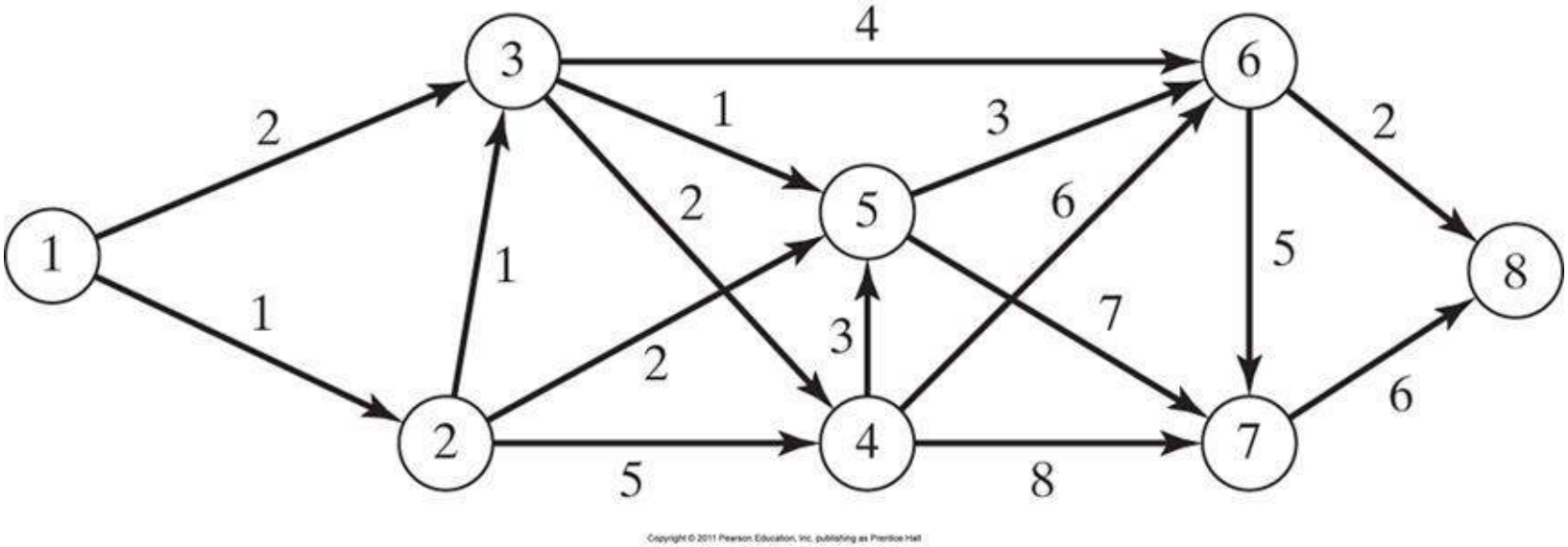


Figure 6.17 Network for Problem 1

Floyd's Algorithm. Floyd's algorithm is more general than Dijkstra's because it determines the shortest route between *any* two nodes in the network. The algorithm represents an n-node network as a square matrix with n rows and n columns. Entry (i, j) of the matrix gives the distance d_{ij} from node i to node j, which is finite if i is linked directly to j, and infinite otherwise.

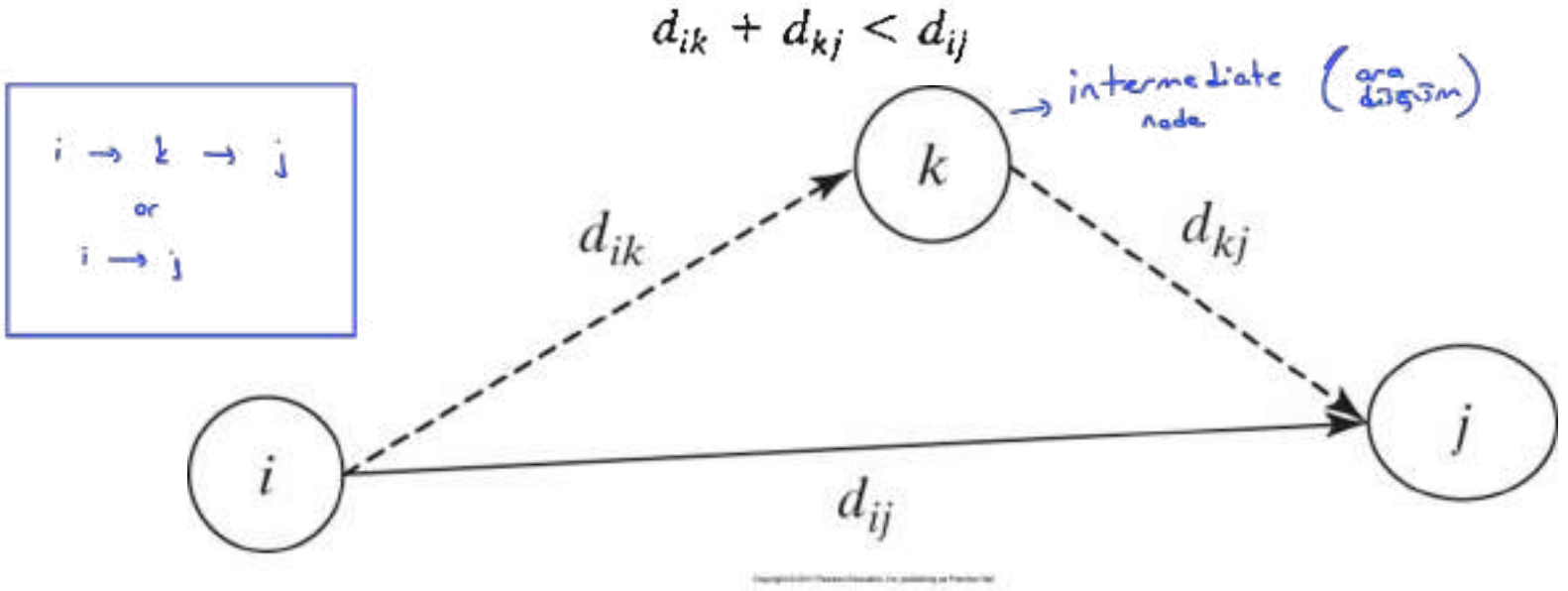


Figure 6.19 Floyd's triple operation

In this case, it is optimal to replace the direct route from $i \rightarrow j$ with the indirect route $i \rightarrow k \rightarrow j$. This **triple operation** exchange is applied systematically to the network using the following steps:

Floyd's Algorithm

$S_{12} = 2$ direct

$S_{12} = k$, $k \neq 1$, $k \neq 2$ indirect

düğüm dizisi matrisi

Step 0. Define the starting distance matrix D_0 and node sequence matrix S_0 as given below. The diagonal elements are marked with (—) to indicate that they are blocked. Set $k = 1$.

$$D_0 = \begin{array}{c|cccccc} & 1 & 2 & \cdots & j & \cdots & n \\ \hline 1 & — & d_{12} & \cdots & d_{1j} & \cdots & d_{1n} \\ 2 & d_{21} & — & \cdots & d_{2j} & \cdots & d_{2n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ i & d_{i1} & d_{i2} & \cdots & d_{ij} & \cdots & d_{in} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ n & D_{n1} & d_{n2} & \cdots & d_{nj} & \cdots & — \end{array}$$

$$S_0 = \begin{array}{c|cccccc} & 1 & 2 & \cdots & j & \cdots & n \\ \hline 1 & — & 2 & \cdots & j & \cdots & n \\ 2 & 1 & — & \cdots & j & \cdots & n \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ i & 1 & 2 & \cdots & j & \cdots & n \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ n & 1 & 2 & \cdots & j & \cdots & — \end{array}$$

* düğümler arası mesafeleri belirtir

General step k. Define row k and column k as *pivot row* and *pivot column*. Apply the *triple operation* to each element d_{ij} in D_{k-1}, for all i and j. If the condition

$$d_{ik} + d_{kj} < d_{ij}, (i \neq k, j \neq k, \text{and } i \neq j)$$

is satisfied, make the following changes:

(a) Create D_k by replacing d_{ij} in D_{k-1} with $d_{ik} + d_{kj}$

(b) Create S_k by replacing s_{ij} in S_{k-1} with k. Set $k = k + 1$. If $k = n + 1$, stop; else repeat step k.

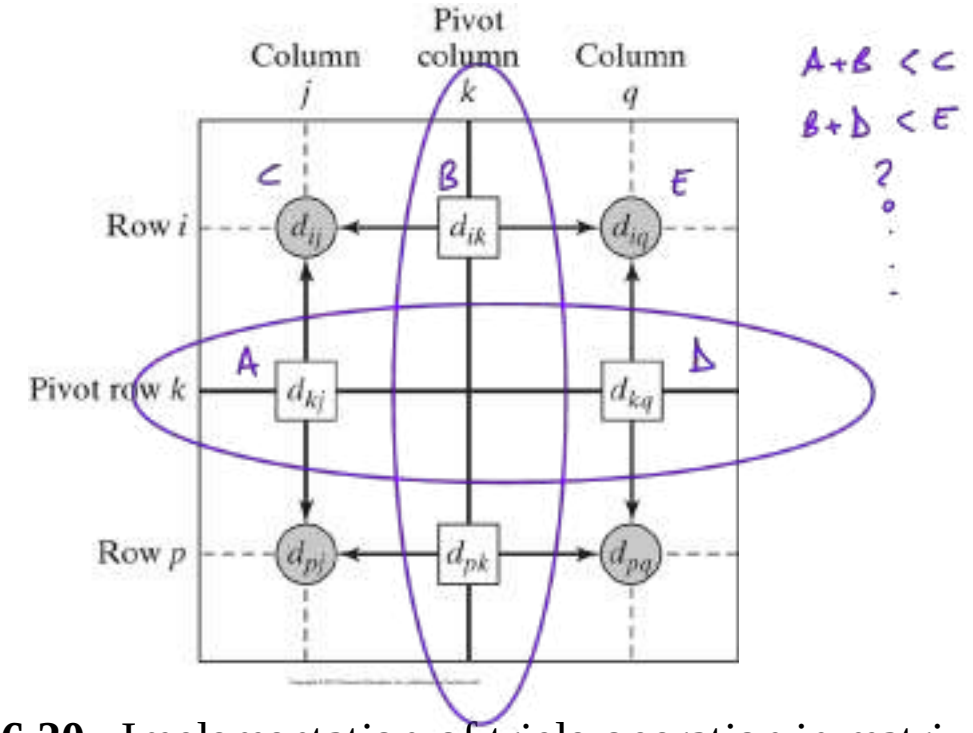


Figure 6.20 Implementation of triple operation in matrix form

Step k of the algorithm can be explained by representing D_{k-1} as shown in Figure 6.20. Here, row k and column k define the current pivot row and column. Row i represents any of the rows $1, 2, \ldots,$ and $k-1$, and row p represents any of the rows $k+1, k+2, \ldots,$ and n. Similarly, column j represents any of the columns $1, 2, \ldots,$ and $k-1$, and column q represents any of the columns $k+1, k+2, \ldots,$ and n. The *triple operation* can be applied as follows. If the sum of the elements on the pivot row and the pivot column (shown by squares) is smaller than the associated intersection element (shown by a circle), then it is optimal to replace the intersection distance by the sum of the pivot distances.

After n steps, we can determine the shortest route between nodes i and j from the matrices D_n and S_n using the following rules:

1. From D_n, d_{ij} gives the shortest distance between nodes i and j.
2. From S_n, determine the intermediate node $k = s_{ij}$ that yields the route $i \rightarrow k \rightarrow j$. If $s_{ik} = k$ and $s_{kj} = j$, stop; all the intermediate nodes of the route have been found. Otherwise, repeat the procedure between nodes i and k, and between nodes k and j.

Example 6.3-5

For the network in Figure 6.21, find the shortest routes between every two nodes. The distances (in miles) are given on the arcs. Arc (3,5) is directional, so that no traffic is allowed from node 5 to node 3. All the other arcs allow two-way traffic.

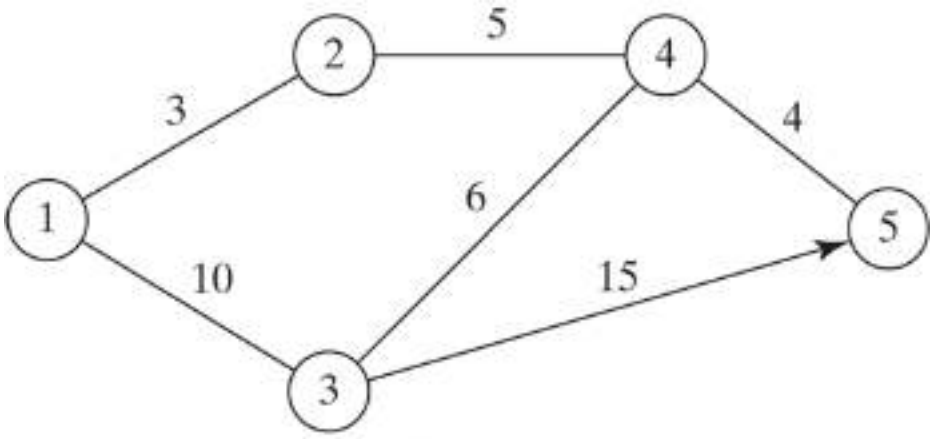


Figure 6.21 Network for Example 6.3-5

$n = 5 \rightarrow$ step 0

Pivot

$k = 1$

$D_0 =$

	1	2	3	4	5
1	–	3	10	∞	∞
2	3	–	∞	5	∞
3	10	∞	–	6	15
4	∞	5	6	–	4
5	∞	∞	∞	4	–

$d_{21} + d_{13} \overset{?}{=} d_{23}$

$S_0 =$

	1	2	3	4	5
1	—	2	3	4	5
2	1	—	3	4	5
3	1	2	—	4	5
4	1	2	3	—	5
5	1	2	3	4	—

$k = 2$

$D_1 =$

	1	2	3	4	5
1	–	3	10	∞	∞
2	3	–	13	5	∞
3	10	13	–	6	15
4	∞	5	6	–	4
5	∞	∞	∞	4	–

$S_1 =$

	1	2	3	4	5
1	–	2	3	4	5
2	1	–	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	–

$k = 3$

$D_2 =$

	1	2	3	4	5
1	–	3	10	8	∞
2	3	–	13	5	∞
3	10	13	–	6	15
4	8	5	6	–	4
5	∞	∞	∞	4	–

$S_2 =$

	1	2	3	4	5
1	–	2	3	2	5
2	1	–	1	4	5
3	1	1	-	4	5
4	2	2	3	-	5
5	1	2	3	4	–

$k = 4$

$D_3 =$

	1	2	3	4	5
1	–	3	10	8	25*
2	3	–	13*	5	28*
3	10	13*	–	6	15*
4	8	5	6	–	4
5	∞*	∞*	∞*	4	–

$S_3 =$

	1	2	3	4	5
1	–	2	3	2	3*
2	1	–	1	4	3*
3	1	1*	–	4	5*
4	2	2	3	–	5
5	1*	2*	3*	4	5

$k = 5$

$D_4 = D_5$

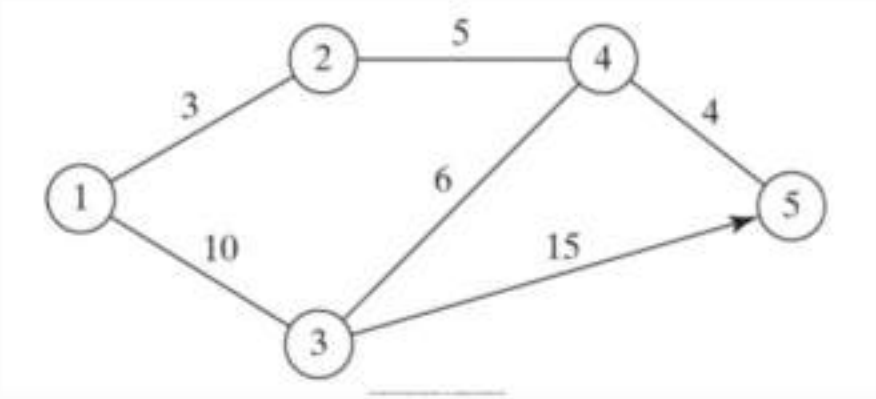
	1	2	3	4	5
1	–	3	10	8	12
2	3	–	11	5	9
3	10	11	–	6	10
4	8	5	6	–	4
5	12	9	10	4	–

$S_4 = S_5$

	1	2	3	4	5
1	–	2	3	2	4
2	1	–	4	4	4
3	1	4	–	4	4
4	2	2	3	–	5
5	4	4	4	4	–

No improvement

$k = 6 = n+1$ STOP



$D_4 =$

	1	2	3	4	5
1	–	3	10	8	12
2	3	–	11	5	9
3	10	11	–	6	10
4	8	5	6	–	4
5	12	9	10	4	–

$S_4 =$

	1	2	3	4	5
1	–	2	3	2	4
2	1	–	4	4	4
3	1	4	–	4	4
4	2	2	3	–	5
5	4	4	4	4	–

Shortest Distance Route

from $1 \rightarrow 5$

$d_{15} = ?_{0}$ 12

$1 \rightarrow 5$ $S_{15} = 4$

$1 \rightarrow 4 \rightarrow 5$

$S_{14} = 2$, $S_{45} = 5$

direct

$1 \rightarrow 2 \rightarrow 4 \rightarrow 5$

shortest distance

$3 \rightarrow 5$

$d_{35} = 10$, $S_{35} = 4$

$3 \rightarrow 4 \rightarrow 5$

$S_{34} \rightarrow 4$, $S_{45} = 5$

$3 \rightarrow 4 \rightarrow 5$

shortest path

Iteration 0. The matrices D_0 and S_0 give the initial representation of the network. D_0 is symmetrical, except that $d_{53} = \infty$ because no traffic is allowed from node 5 to node 3.

Iteration 1. Set $k = 1$. The pivot row and column are shown by the lightly shaded first row and first column in the D_0-matrix. The darker cells, d_{23} and d_{32}, are the only ones that can be improved by the *triple operation*. Thus, D_1 and S_1 are obtained from D_0 and S_0 in the following manner:

1. Replace d_{23} with $d_{21} + d_{13} = 3 + 10 = 13$ and set $s_{23} = 1$.
2. Replace d_{32} with $d_{31} + d_{12} = 10 + 3 = 13$ and set $s_{32} = 1$.

Iteration 2. Set $k = 2$, as shown by the lightly shaded row and column in D_1. The *triple operation* is applied to the darker cells in D_1 and S_1. The resulting changes are shown in bold in D_2 and S_2.

Iteration 3. Set $k = 3$, as shown by the shaded row and column in D_2. The new matrices are given by D_3 and S_3.

Iteration 4. Set $k = 4$, as shown by the shaded row and column in D_3. The new matrices are given by D_4 and S_4.

Iteration 5. Set $k = 5$, as shown by the shaded row and column in D_4. No further improvements are possible in this iteration.

The final matrices D_4 and S_4 contain all the information needed to determine the shortest route between any two nodes in the network. For example, from D_4, the shortest distance from node 1 to node 5 is $d_{15} = 12$ miles. To determine the associated route, recall that a segment (i, j) represents a direct link only if $s_{ij} = j$. Otherwise, i and j are linked through at least one other intermediate node. Because $s_{15} = 4 \neq 5$, the route is initially given as $1 \rightarrow 4 \rightarrow 5$. Now, because $s_{14} = 2 \neq 4$, the segment $(1, 4)$ is not a *direct* link, and $1 \rightarrow 4$ is replaced with $1 \rightarrow 2 \rightarrow 4$, and the route $1 \rightarrow 4 \rightarrow 5$ now becomes $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$. Next, because $s_{12} = 2$, $s_{24} = 4$, and $s_{45} = 5$, no further "dissecting" is needed, and $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$ defines the shortest route.

Homework (SR-3)

1) In Example 6.3-5, use Floyd's algorithm to determine the shortest routes between each of the following pairs of nodes:

(a) From node 5 to node 1.
(b) From node 3 to node 5.
(c) From node 5 to node 3.
(d) From node 5 to node 2.

2) Six kids, Joe, Kay, Jim, Bob, Rae, and Kim, play a variation of *hide and seek*. The hiding place of a child is known only to a select few of the other children. A child is then paired with another with the objective of finding the partner's hiding place. This may be achieved through a chain of other kids who eventually will lead to discovering where the designated child is hiding. For example, suppose that Joe needs to find Kim and that Joe knows where Jim is hiding, who in turn knows where Kim is. Thus, Joe can find Kim by first finding Jim, who in turn will lead Joe to Kim. The following list provides the whereabouts of the children:

Joe knows the hiding places of Bob and Kim.
Kay knows the hiding places of Bob, Jim, and Rae.
Jim and Bob each know the hiding place of Kay only.
Rae knows where Kim is hiding.
Kim knows where Joe and Bob are hiding.

Linear Programming Formulation of the SR Problem

Define x_{ij} = amount of flow in arc (i, j)

1

$$= \begin{cases} 1, \text{ if arc } (i, j) \text{ is on the shortest route} \\ 0, \text{ otherwise} \end{cases}$$

c_{ij} = length of arc (i, j)

Thus, the objective function of the linear program becomes Minimize $z = \sum_{\substack{\text{all defined} \\ \text{arcs } (i,j)}} c_{ij}x_{ij}$

2

Mathematically, this translates for node j to

$$\begin{pmatrix} \text{External input} \\ \text{into node } j \end{pmatrix} + \sum_{\substack{i \\ \text{all defined} \\ \text{arcs } (i,j)}} x_{ij} = \begin{pmatrix} \text{External output} \\ \text{from node } j \end{pmatrix} + \sum_{\substack{k \\ \text{all defined} \\ \text{arcs } (j,k)}} x_{jk}$$

3

Consider the shortest-route network of Example 6.3-4. Suppose that we want to determine the shortest route from node 1 to node 2—that is, $s = 1$ and $t = 2$. Figure 6.24 shows how the unit of flow enters at node 1 and leaves at node 2.

We can see from the network that the flow-conservation equation yields

$$\begin{aligned}
&\text{Node 1:} && 1 = x_{12} + x_{13}\\
&\text{Node 2:} && x_{12} + x_{42} = x_{23} + 1\\
&\text{Node 3:} && x_{13} + x_{23} = x_{34} + x_{35}\\
&\text{Node 4:} && x_{34} = x_{42} + x_{45}\\
&\text{Node 5:} && x_{35} + x_{45} = 0
\end{aligned}$$

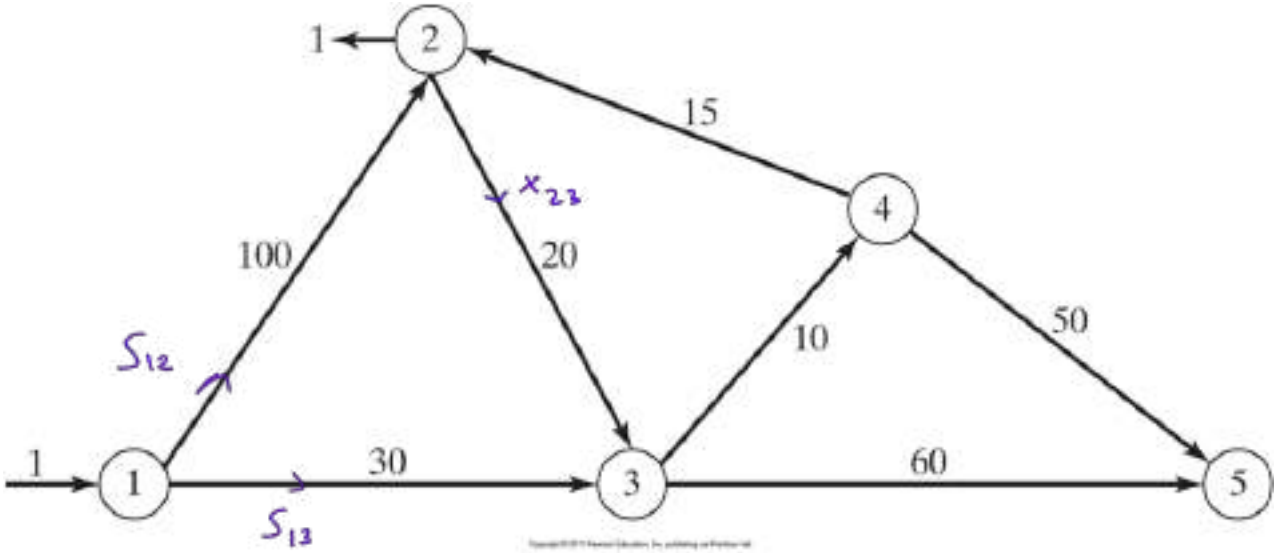


Figure 6.24 Insertion of unit flow to determine shortest route between node $s = 1$ and node $t = 2$

The complete LP can be expressed as

sparse coefficient matrix

	x_{12}	x_{13}	x_{23}	x_{34}	x_{35}	x_{42}	x_{45}	
Minimize z =	100	30	20	10	60	15	50	
Node 1	1	1						= 1
Node 2	−1		1			−1		= −1
Node 3		−1	−1	1	1			= 0
Node 4				−1		1	1	= 0
Node 5					−1		−1	= 0

Notice that column x_{ij} has exactly one "1" entry in row i and one "−1" entry in row j, a typical property of a network LP.

The optimal solution (obtained by TORA, file toraEx6.3-6.txt) is

$$z = 55, x_{13} = 1, x_{34} = 1, x_{42} = 1$$

This solution gives the shortest route from node 1 to node 2 as $1 \rightarrow 3 \rightarrow 4 \rightarrow 2$, and the associated distance is $z = 55$ (miles).

References

[1] *Ahlatçıoğlu, M. , Tiryaki, F., Kantitatif Karar Verme Teknikleri, YTÜ Yayın No: YTÜ.FE.DK-98.0349, İstanbul-1998.*

[2] *H.A.Taha, Operations Research: An Introduction,* Prentice Hall; 9th edition, *Singapore, 2010.*

Lecture 6

Maximal Flow

Maximal Flow (MF) Problem

Consider a network of pipelines that transports crude oil from oil wells to refineries. Intermediate booster and pumping stations are installed at appropriate design distances to move the crude in the network. Each pipe segment has a finite maximum discharge rate of crude flow (or capacity). A pipe segment may be uni- or bidirectional, depending on its design. Figure 6.27 demonstrates a typical pipeline network. How can we determine the maximum capacity of the network between the wells and the refineries?

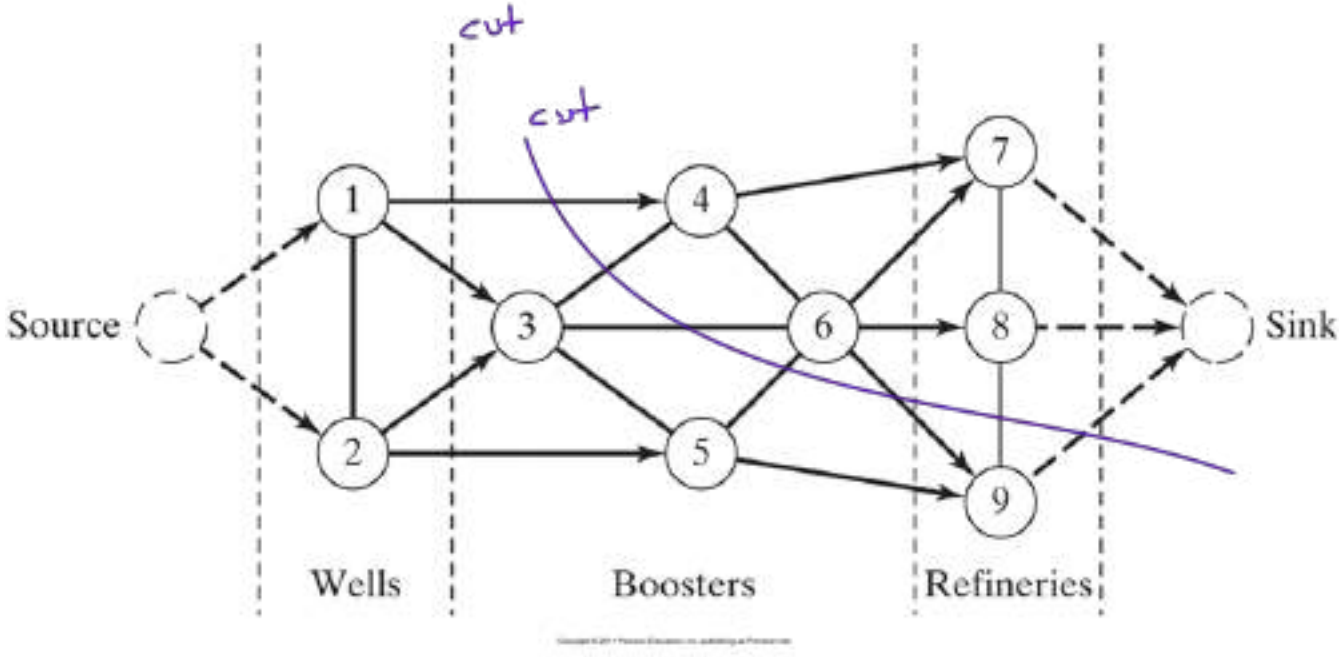


Figure 6.27 Capacitated network connecting wells and refineries through booster stations

The solution of the proposed problem requires equipping the network with a single source and a single sink by using unidirectional infinite capacity arcs as shown by dashed arcs in Figure 6.27.

Given arc (i, j) with $i < j$, we use the notation $(\overline{C}_{ij}, \overline{C}_{ji})$ to represent the flow capacities in the two directions $i \rightarrow j$ and $j \rightarrow i$, respectively. To eliminate ambiguity, we place $\overline{C}_{ij}$ on the arc next to node i with $\overline{C}_{ji}$ placed next to node j, as shown in Figure 6.28.

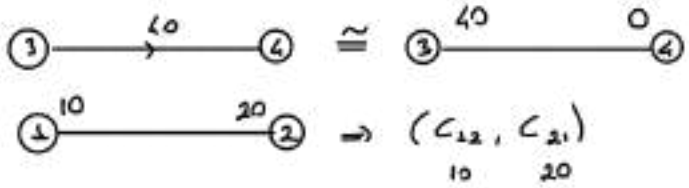


Figure 6.28
Arc Flows C_{ij} from $i \rightarrow j$
and C_{ji} from $j \rightarrow i$

Enumeration of cuts

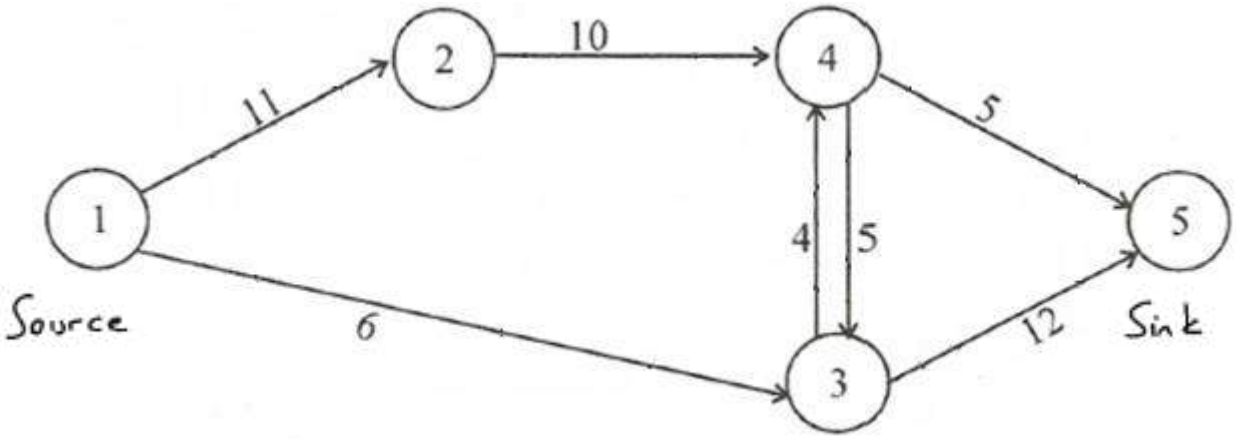
A **cut** defines a set of arcs which when deleted from the network will cause a total disruption of flow between the source and sink nodes. The **cut capacity** equals the sum of the capacities of its arcs. Among *all* possible cuts in the network, the cut with the *smallest capacity* gives the maximum flow in the network.

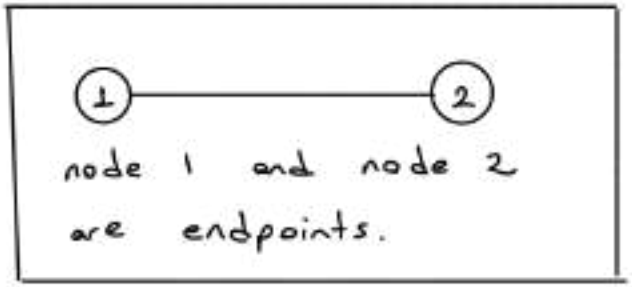
Another Definition:

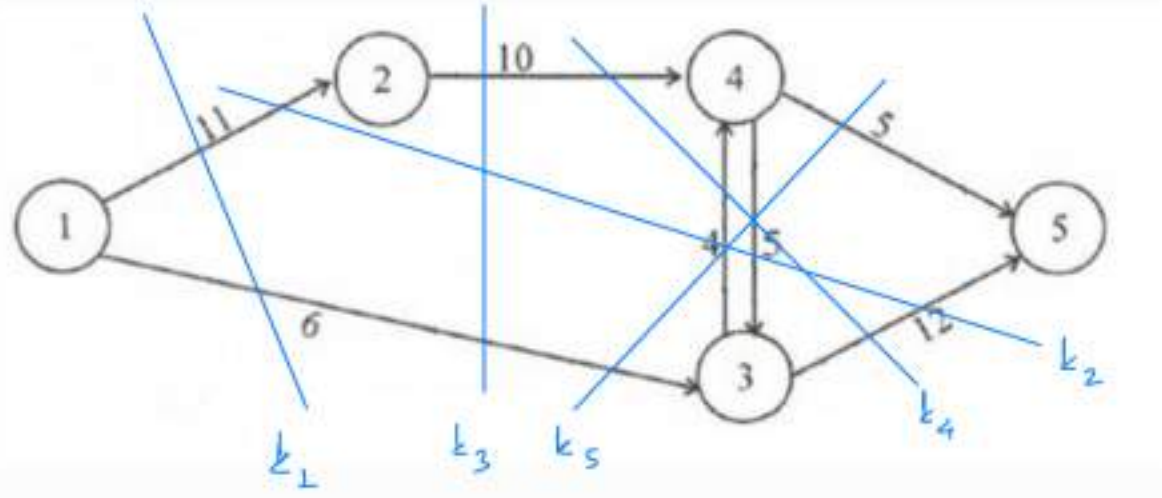
A ***cut*** is a partition of the node set N into two subsets S and $\overline{S} = N - S$.

Alternatively, we can define a ***cut*** as the set of arcs whose endpoints belong to the different subsets S and $\overline{S}$.

Example: Determining the capacity of all possible cuts, find the maximal flow from the source node 1 to the sink node 5.







$k_1 : C(X, \bar{X}) = 17$

$k_2 : C(X, \bar{X}) = 27$

$k_3 : C(X, \bar{X}) = 16$

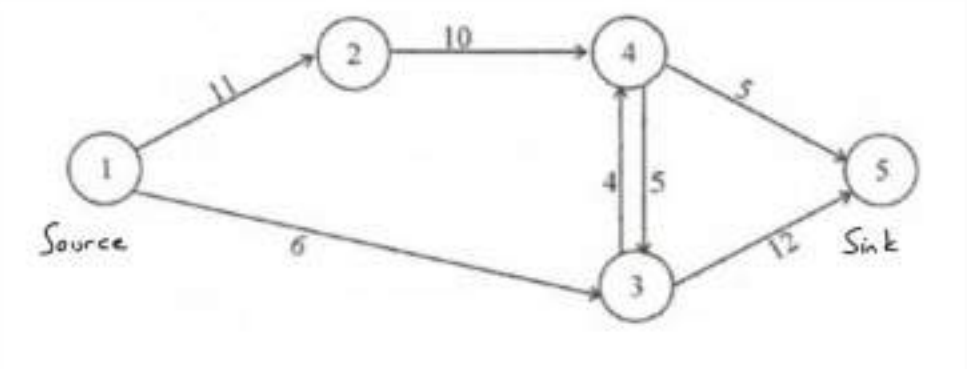
$k_4 : C(X, \bar{X}) = 26$

$k_5 : C(X, \bar{X}) = 16$

$k_6 : C(X, \bar{X}) = 17$

$k_3 \quad X = \{1, 2\}, \quad \bar{X} = \{3, 4, 5\}$

$k_5 \quad X = \{1, 2, 4\}, \quad X = \{3, 5\}$



- $N = \{1, 2, 3, 4, 5\}$
  - $\hookrightarrow S \rightarrow$ Source
  - $\hookrightarrow \bar{S} \rightarrow$ Sink

- The total number of cuts in this network !

$1 \in S$, $5 \in \bar{S}$

{source} {sink}

$\{2, 3, 4\} \longrightarrow 2^3 = 8 \longrightarrow \emptyset$

↘ although some cuts.

- All possible cuts.

C_1 : $S = \{1\}$, $\bar{S} = \{2, 3, 4, 5\} \longrightarrow \emptyset$

C_2 : $S = \{1, 2\}$, $\bar{S} = \{3, 4, 5\} \longrightarrow \{2\}$

C_3 : $S = \{1, 3\}$, $\bar{S} = \{2, 4, 5\} \longrightarrow \{3\}$

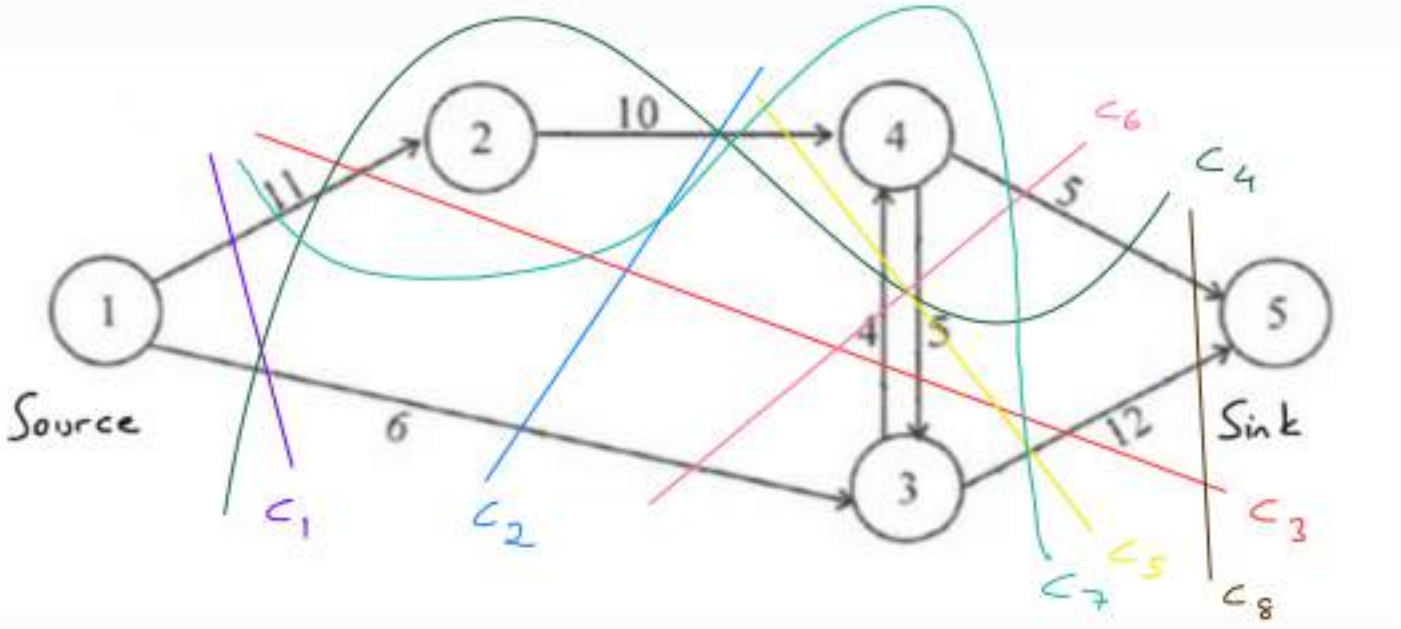
C_4 : $S = \{1, 4\}$, $\bar{S} = \{2, 3, 5\} \longrightarrow \{4\}$

C_5 : $S = \{1, 2, 3\}$, $\bar{S} = \{4, 5\} \longrightarrow \{2, 3\}$

C_6 : $S = \{1, 2, 4\}$, $\bar{S} = \{3, 5\} \longrightarrow \{2, 4\}$

C_7 : $S = \{1, 3, 4\}$, $\bar{S} = \{2, 5\} \longrightarrow \{3, 4\}$

C_8 : $S = \{1, 2, 3, 4\}$, $\bar{S} = \{5\} \longrightarrow \{2, 3, 4\}$



C_1 = (1,2), (1,3), ~~(1,4)~~, ~~(1,5)~~

$11 + 6 = \underline{17}$

C_2 = (1,3), ~~(1,4)~~, ~~(1,5)~~, ~~(2,3)~~, (2,4), ~~(2,5)~~

$6 + 10 = \underline{16}$

C_3 = (1,2), (3,5), (3,4)

$11 + 12 + 4 = \underline{27}$

C_4 = (1,2), (1,3), (4,3), (4,5)

$11 + 6 + 5 + 5 = \underline{27}$

C_5 = (2,4), (3,4), (3,5)

$10 + 4 + 12 = \underline{26}$

C_6 = (4,5), (4,3), (1,3)

$5 + 5 + 6 = \underline{16}$

C_7 = (1,2), (3,5), (4,5)

$11 + 12 + 5 = 28$

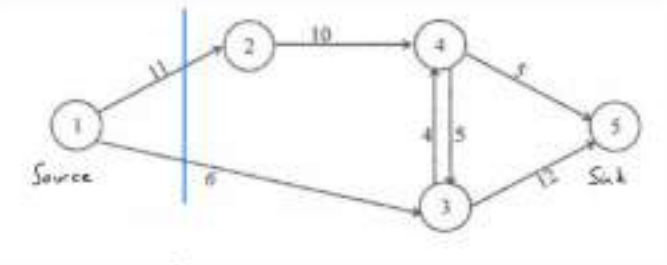
C_8 = (4,5), (3,5)

$5 + 12 = 17$

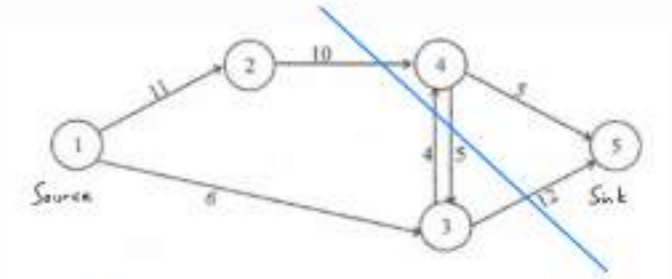
maximal flow amount

= min { cut capacities }

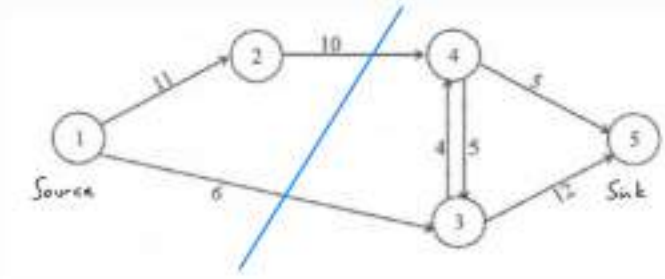
= 16 →



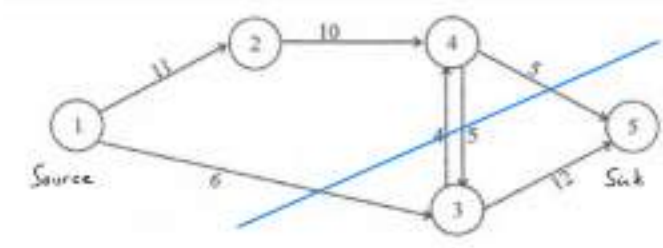
C_1



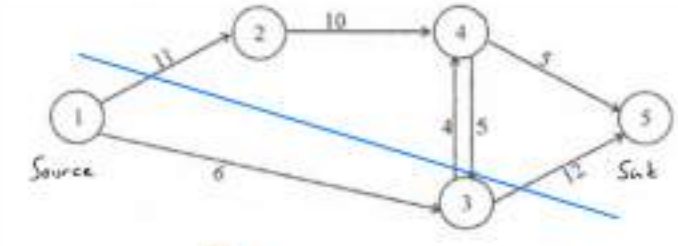
C_5



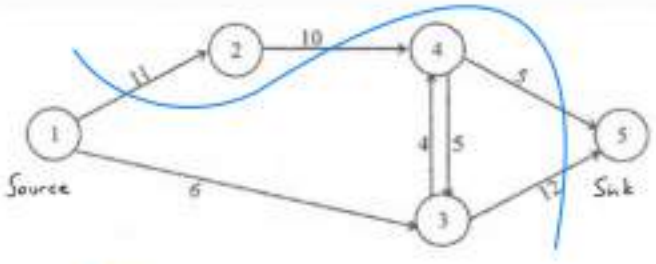
C_2



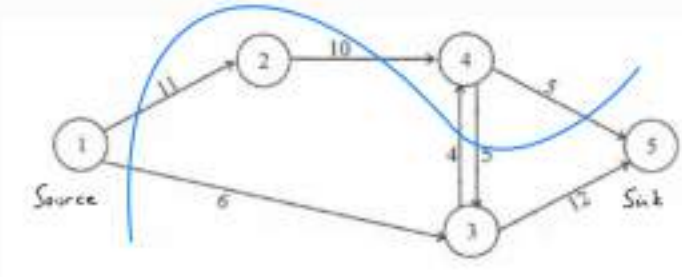
C_6



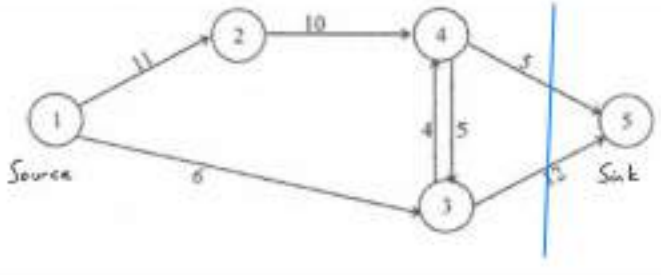
C_3



C_7



C_4

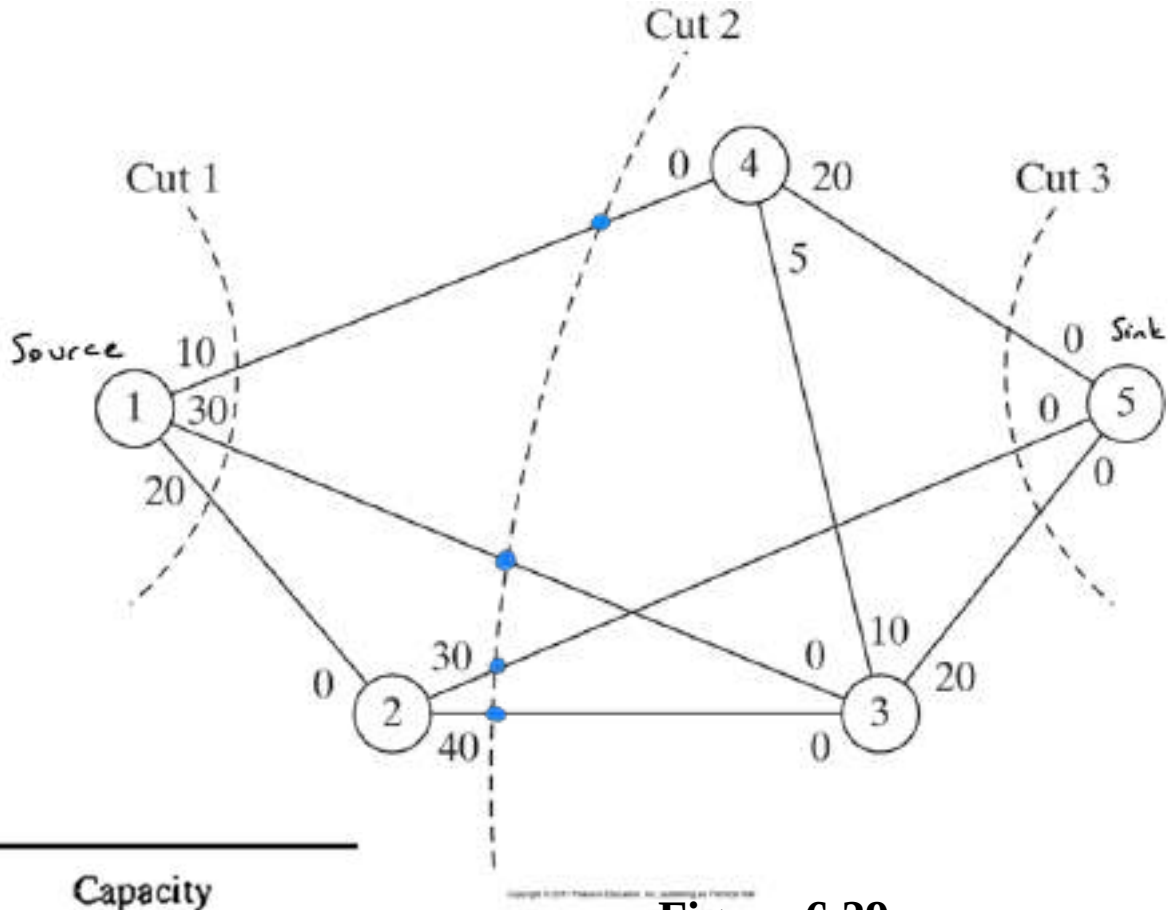


C_8

Example 6.4.1

Consider the network in Figure 6.29. The bidirectional capacities are shown on the respective arcs using the convention in Figure 6.28. For example, for arc (3,4), the flow limit is 10 units from 3 to 4 and 5 units from 4 to 3.

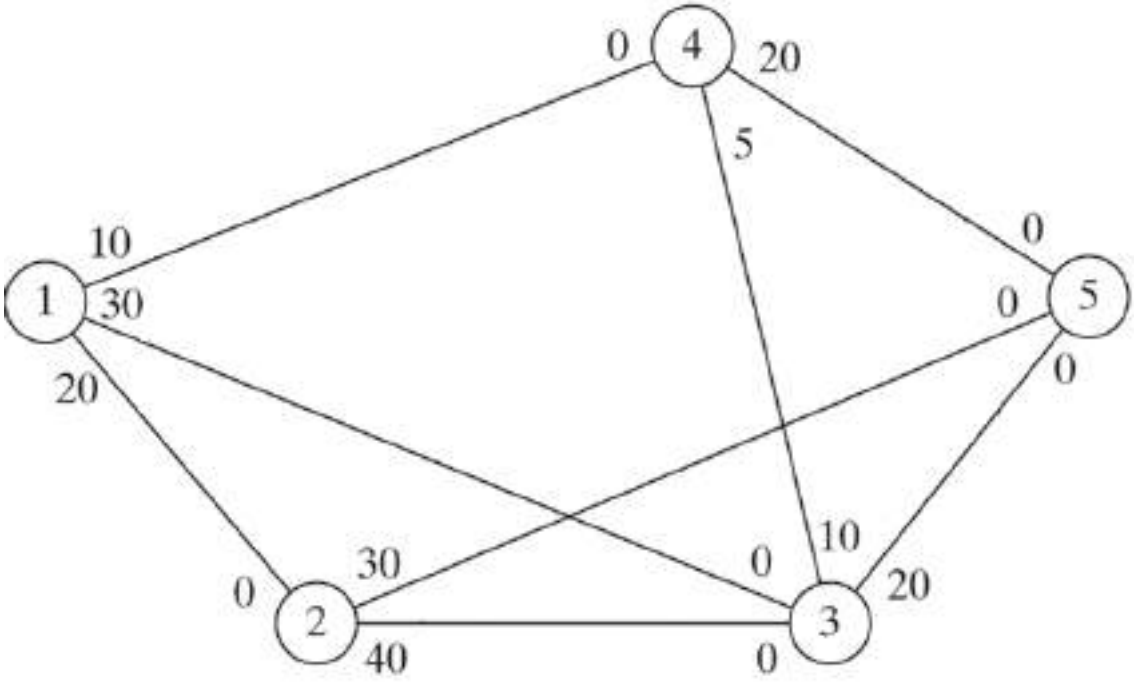
Figure 6.29 illustrates three cuts whose capacities are computed in the following table



Cut	Associated arcs	Capacity
1	(1,2), (1,3), (1,4)	20 + 30 + 10 = 60
2	(1,3), (1,4), (2,3), (2,5)	30 + 10 + 40 + 30 = 110
3	(2,5), (3,5), (4,5)	30 + 20 + 20 = 70

Figure 6.29
Examples of cuts in flow networks

The only information we can glean from the three cuts is that the maximum flow in the network cannot exceed 60 units. To determine the maximum flow, it is necessary to enumerate *all* the cuts, a difficult task for the general network. Thus, the need for an efficient algorithm is imperative.



Homework (MF-1): Consider the network in Figure 6.29.
Enumerate all the cuts and determine the maximum flow.

Maximal Flow Algorithm

The maximal flow algorithm is based on finding **breakthrough paths** with net *positive* flow between the source and sink nodes. Each path commits part or all of the capacities of its arcs to the total flow in the network.

Consider arc (i, j) with (initial) capacities $(\overline{C}_{ij}, \overline{C}_{ji})$. As portions of these capacities are committed to the flow in the arc, the **residuals** (or remaining capacities) of the arc are updated. We use the notation (c_{ij}, c_{ji}) to represent these residuals.

For a node j that receives flow from node i, we attach a label $[a_j, i]$, where a_j is the flow from node i to node j. The steps of the algorithm are thus summarized as follows.

Maximal Flow Algorithm (1/2)

Step 1. For all arcs (i, j), set the residual capacity equal to the initial capacity— that is $(c_{ij}, c_{ji}) = (\overline{C}_{ij}, \overline{C}_{ji})$. Let $a_1 = \infty$ and label source node 1 with $[\infty, -]$. Set $i = 1$, and go to step 2.

Step 2. Determine S_i, the set of unlabeled nodes j that can be reached directly from node i by arcs with *positive* residuals (that is, $c_{ij} > 0$ for all $j \in S_i$). If $S_i \neq \emptyset$, go to step 3. Otherwise, go to step 4.

Step 3. Determine $k \in S_i$ such that

$$c_{ik} = \max_{j \in S_i} \{c_{ij}\}$$

Set $a_k = c_{ik}$ and label node k with $[a_k, i]$. If $k = n$, the sink node has been labeled, and a *breakthrough path* is found, go to step 5. Otherwise, set $i = k$, and go to step 2.

Step 4. **(Backtracking).** If $i = 1$, no breakthrough is possible; go to step 6. Otherwise, let r be the node that has been labeled *immediately* before current node i and remove i from the set of nodes adjacent to r. Set $i = r$, and go to step 2.

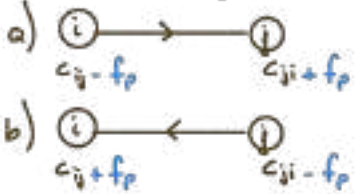
Maximal Flow Algorithm (2/2)

Step 5. **(Determination of residuals).** Let $N_p = (1, k_1, k_2, \ldots, n)$ define the nodes of the pth breakthrough path from source node 1 to sink node n. Then the maximum flow along the path is computed as

f_p → amount for the corresponding (karşılık gelen tutar)

$$f_p = \min\{a_1, a_{k_1}, a_{k_2}, \ldots, a_n\}$$ (athlım yolu)

The residual capacity of each arc along the breakthrough path is *decreased* by f_p in the direction of the flow and *increased* by f_p in the reverse direction—that is, for nodes i and j on the path, the residual flow is changed from the current (c_{ij}, c_{ji}) to



(a) $(c_{ij} - f_p, c_{ji} + f_p)$ if the flow is from i to j

(b) $(c_{ij} + f_p, c_{ji} - f_p)$ if the flow is from j to i

Reinstate any nodes that were removed in step 4. Set $i = 1$, and return to step 2 to attempt a new breakthrough path.

Step 6. **(Solution).** **(a)** Given that m breakthrough paths have been determined, the maximal flow in the network is

$$\boxed{F = f_1 + f_2 + \cdots + f_m}$$

(b) Using the *initial* and *final* residuals of arc (i, j), $(\overline{C}_{ij}, \overline{C}_{ji})$ and (c_{ij}, c_{ji}), respectively, the optimal flow in arc (i, j) is computed as follows: Let $(\alpha, \beta) = (\overline{C}_{ij} - c_{ij}, \overline{C}_{ji} - c_{ji})$. If $\alpha > 0$, the optimal flow from i to j is α. Otherwise, if $\beta > 0$, the optimal flow from j to i is β. (It is impossible to have both α and β positive.)

Example 6.4.2

Determine the maximal flow in the network of Example 6.4-1 (Figure 6.29). Figure 6.31 provides a graphical summary of the iterations of the algorithm. You will find it helpful to compare the description of the iterations with the graphical summary.

Iteration 1. Set the initial residuals (c_{ij}, c_{ji}) equal to the initial capacities $(\overline{C}_{ij}, \overline{C}_{ji})$.

Step 1. Set $a_1 = \infty$ and label node 1 with $[\infty, —]$. Set $i = 1$.

Step 2. $S_1 = \{2, 3, 4\}$ $(\neq \emptyset)$.

Step 3. $k = 3$, because $c_{13} = \max\{c_{12}, c_{13}, c_{14}\} = \max\{20, 30, 10\} = 30$. Set $a_3 = c_{13} = 30$, and label node 3 with [30,1]. Set $i = 3$, and repeat step 2.

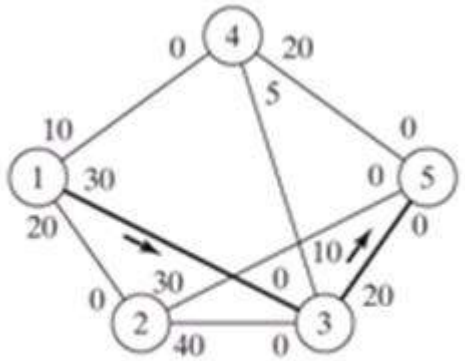
Step 2. $S_3 = (4, 5)$.

Step 3. $k = 5$ and $a_5 = c_{35} = \max\{10, 20\} = 20$. Label node 5 with [20, 3]. Breakthrough is achieved. Go to step 5.

Step 5. The breakthrough path is determined from the labels starting at node 5 and moving backward to node 1—that is, (5) → [20, 3] → (3) → [30, 1] → (1). Thus, $N_1 = \{1, 3, 5\}$ and $f_1 = \min\{a_1, a_3, a_5\} = \{\infty, 30, 20\} = 20$. The residual capacities along path N_1 are

$$(c_{13}, c_{31}) = (30 - 20, 0 + 20) = (10, 20)$$

$$(c_{35}, c_{53}) = (20 - 20, 0 + 20) = (0, 20)$$

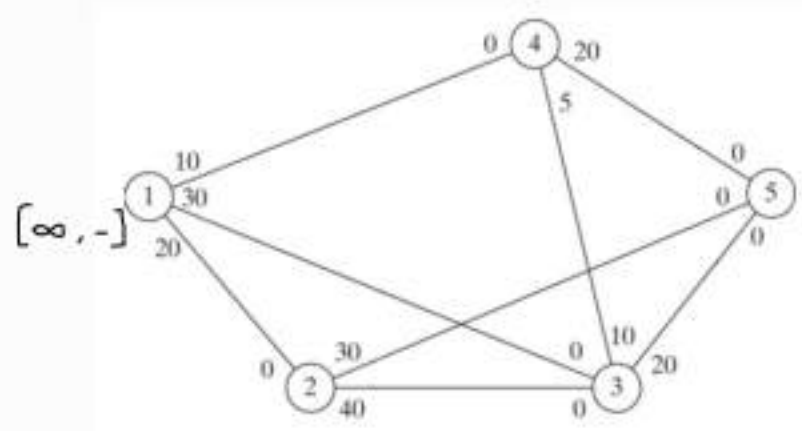


(Iteration 1) $f_1 = 20$

Source : Node 1

Sink : Node 5

Maximal flow $1 \rightarrow 5$



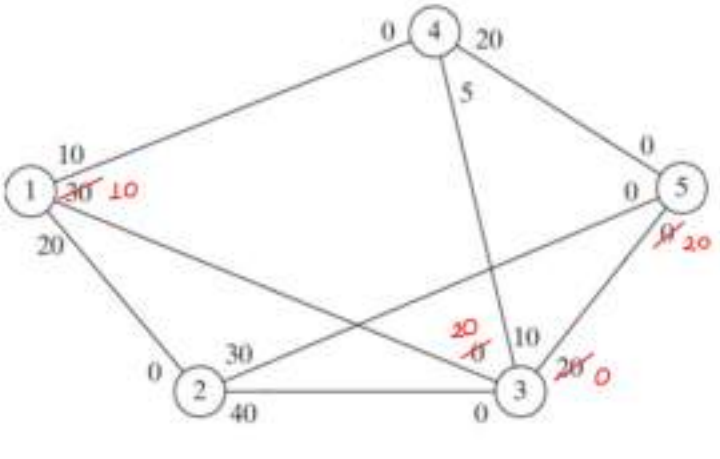
Step 1: $[\infty, -]$, $i = 1$

$S_1 = \{4, 3, 2\}$

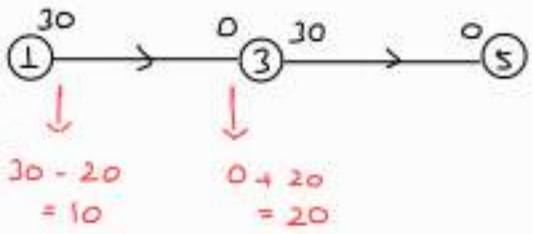
$\max\{c_{14}, c_{13}, c_{12}\} = 30$ (10, 30, 20) → $1 - 3$

$S_3 = \{4, 5\}$

$1 \rightarrow 3 \rightarrow 5$: $\min\{\infty, 30, 20\} = 20 = f_1$



1. iteration

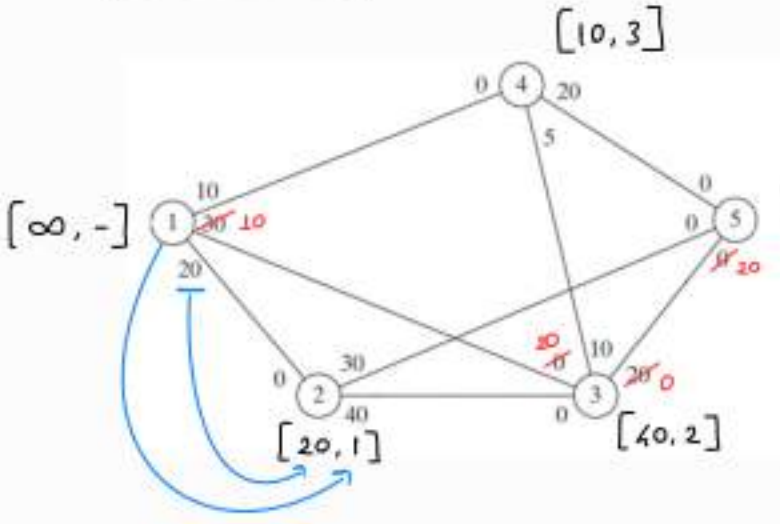


30 + 0 = 30

10 + 20 = 30

2. iteration

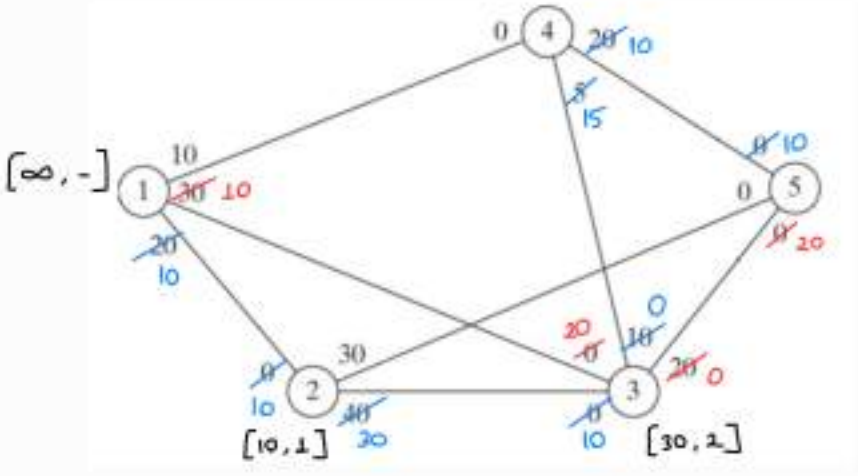
Shortest path
adding the distances up !



[30, 1] —20→ [30 + 20, 3]

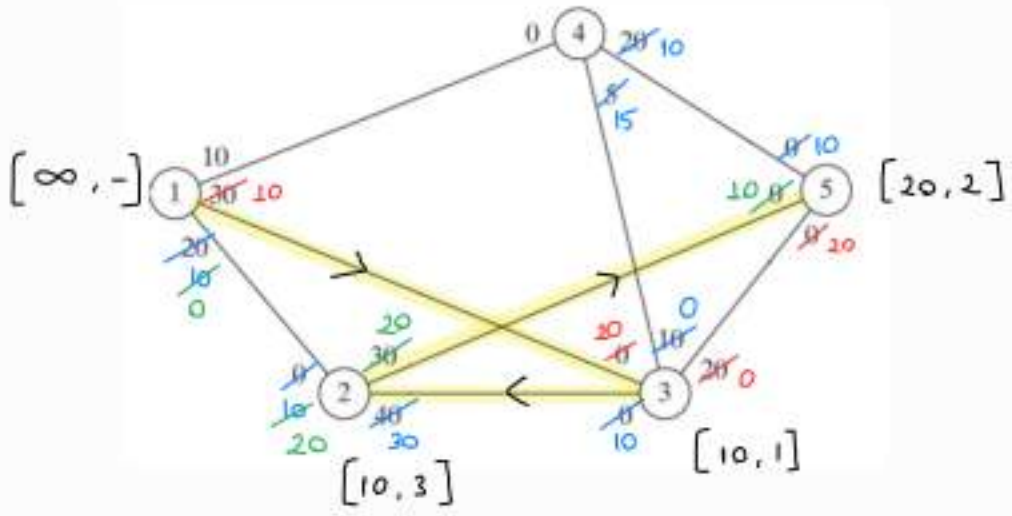
$$f_2 = \min\{\infty, 20, 40, 10, 20\} = 10$$

3. iteration



$S_3 = \{\underbrace{1}_{\text{also}}, \underbrace{4, 5}_{\text{zero capasity}}\}$

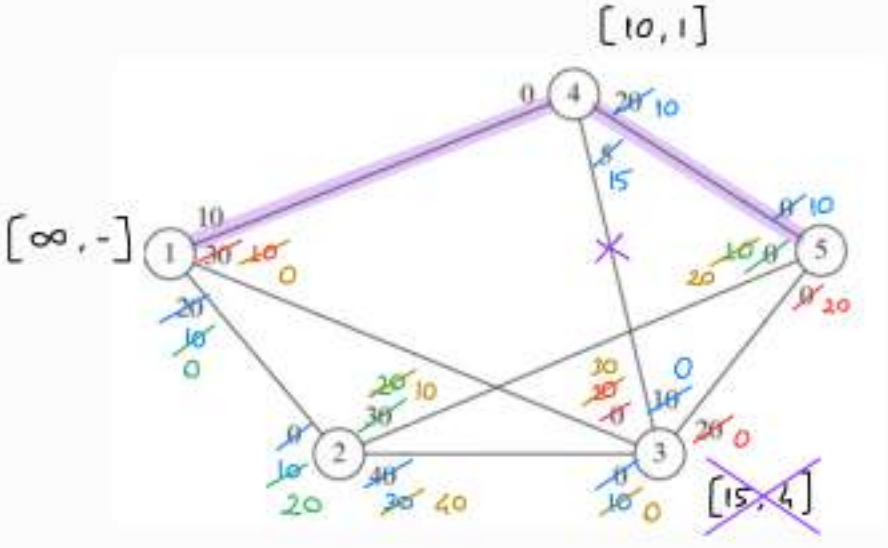
4. iteration

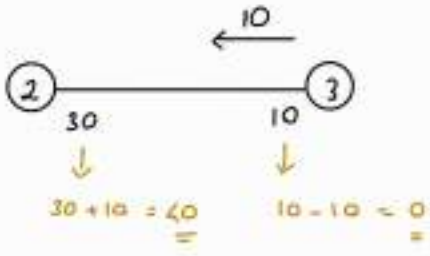


$\perp \rightarrow 3 \rightarrow 2 \rightarrow 5$

$f_4 = \min \{10, 30, 10, 20\} = 10$

5. iteration



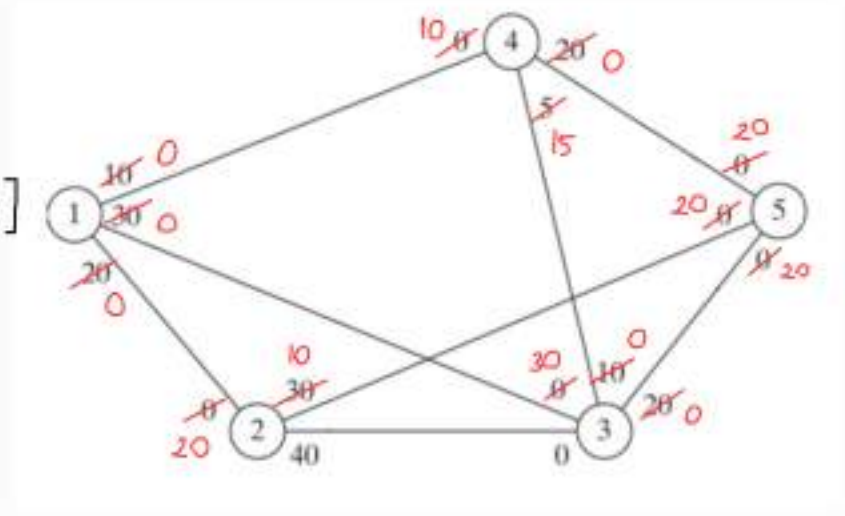


$f_5 = \min \{\infty, 10, 10\}$

$= 10$

6. iteration)

[∞, -]



$S_1 = ?$

$$F = f_1 + f_2 + f_3 + f_4 + f_5$$

$$= 20 + 10 + 10 + 10 + 10$$

$$= 60 \quad \text{(maximal flow amount)}$$

Iteration 2

Step 1. Set $a_1 = \infty$, and label node 1 with $[\infty, -]$. Set $i = 1$.

Step 2. $S_1 = \{2, 3, 4\}$.

Step 3. $k = 2$ and $a_2 = c_{12} = \max\{20, 10, 10\} = 20$. Set $i = 2$, and repeat step 2.

Step 2. $S_2 = \{3, 5\}$.

Step 3. $k = 3$ and $a_3 = c_{23} = 40$. Label node 3 with $[40, 2]$. Set $i = 3$, and repeat step 2.

Step 2. $S_3 = \{4\}$ (note that $c_{35} = 0$—hence, node 5 cannot be included in S_3).

Step 3. $k = 4$ and $a_4 = c_{34} = 10$. Label node 4 with $[10, 3]$. Set $i = 4$, and repeat step 2.

Step 2. $S_4 = \{5\}$ (note that nodes 1 and 3 are already labeled—hence, they cannot be included in S_4).

Step 3. $k = 5$ and $a_5 = c_{45} = 20$. Label node 5 with $[20, 4]$. Breakthrough has been achieved. Go to step 5.

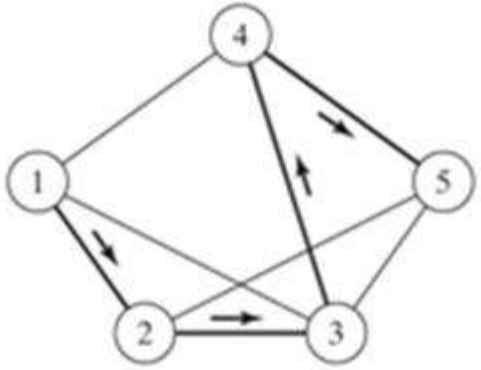
Step 5. $N_2 = \{1, 2, 3, 4, 5\}$ and $f_2 = \min\{\infty, 20, 40, 10, 20\} = 10$. The residuals along the path of N_2 are

$$(c_{12}, c_{21}) = (20 - 10, 0 + 10) = (10, 10)$$

$$(c_{23}, c_{32}) = (40 - 10, 0 + 10) = (30, 10)$$

$$(c_{34}, c_{43}) = (10 - 10, 5 + 10) = (0, 15)$$

$$(c_{45}, c_{54}) = (20 - 10, 0 + 10) = (10, 10)$$



(Iteration 2) $f_2 = 10$

Iteration 3

Step 1. Set $a_1 = \infty$ and label node 1 with $[\infty, -]$. Set $i = 1$.

Step 2. $S_1 = \{2, 3, 4\}$.

Step 3. $k = 2$ and $a_2 = c_{12} = \max\{10, 10, 10\} = 10$. (Though ties are broken arbitrarily, TORA always selects the tied node with the smallest index. We will use this convention throughout the example.) Label node 2 with [10,1]. Set $i = 2$, and repeat step 2.

Step 2. $S_2 = \{3, 5\}$.

Step 3. $k = 3$ and $a_3 = c_{23} = 30$. Label node 3 with [30, 2]. Set $i = 3$, and repeat step 2.

Step 2. $S_3 = \emptyset$ (because $c_{34} = c_{35} = 0$). Go to step 4 to backtrack.

Step 3. *Backtracking.* The label [30, 2] at node 3 gives the immediately preceding node $r = 2$. Remove node 3 from further consideration *in this iteration* by crossing it out. Set $i = r = 2$, and repeat step 2.

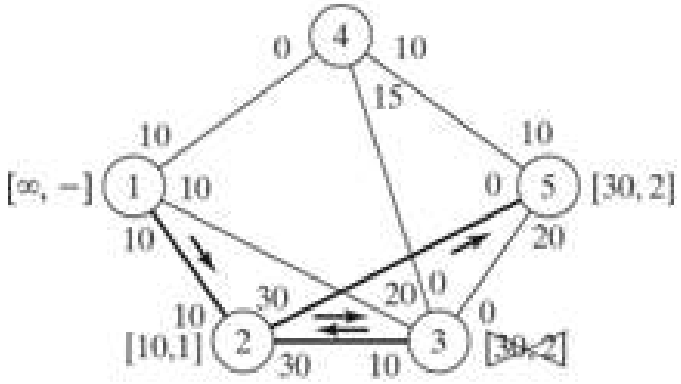
Step 2. $S_2 = \{5\}$ (note that node 3 has been removed in the backtracking step).

Step 3. $k = 5$ and $a_5 = c_{25} = 30$. Label node 5 with [30, 2]. Breakthrough has been achieved; go to step 5.

Step 5. $N_3 = \{1, 2, 5\}$ and $c_5 = \min\{\infty, 10, 30\} = 10$. The residuals along the path of N_3 are

$$(c_{12}, c_{21}) = (10 - 10, 10 + 10) = (0, 20)$$

$$(c_{25}, c_{52}) = (30 - 10, 0 + 10) = (20, 10)$$

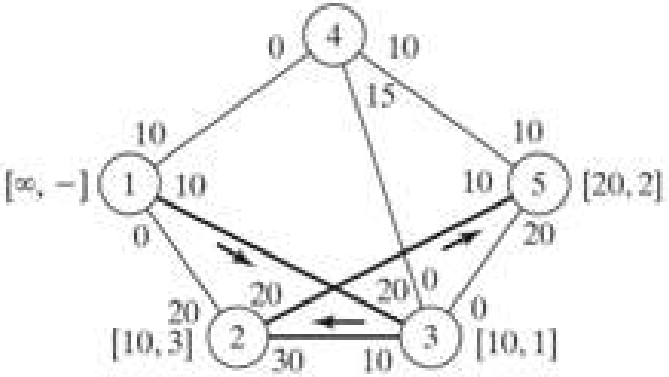


(Iteration 3) $f_3 = 10$

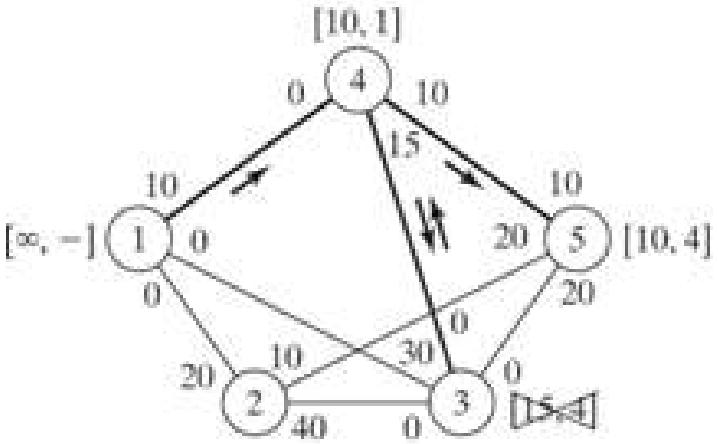
Iteration 4. This iteration yields $N_4 = \{1, 3, 2, 5\}$ with $f_4 = 10$ (verify!). **(Homework (MF-2))**

Iteration 5. This iteration yields $N_5 = \{1, 4, 5\}$ with $f_5 = 10$ (verify!). **(Homework (MF-3))**

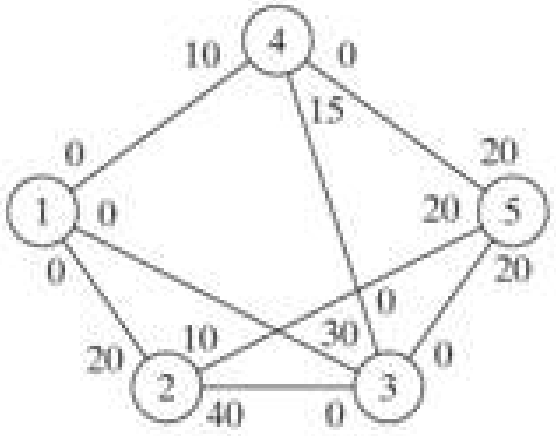
Iteration 6. All the arcs out of node 1 have zero residuals. Hence, no further breakthroughs are possible. We turn to step 6 to determine the solution.



(Iteration 4) $f_4 = 10$

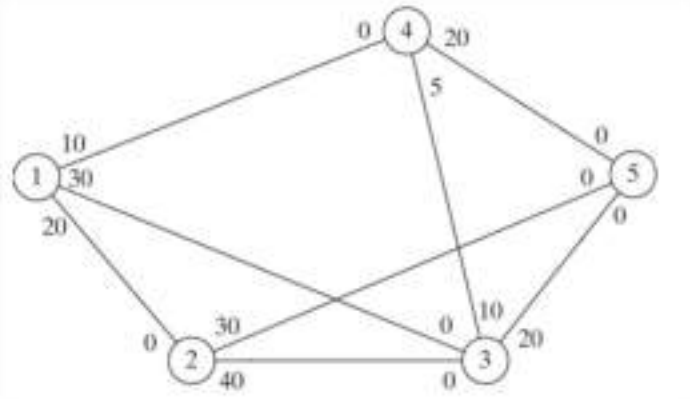


(Iteration 5) $f_5 = 10$

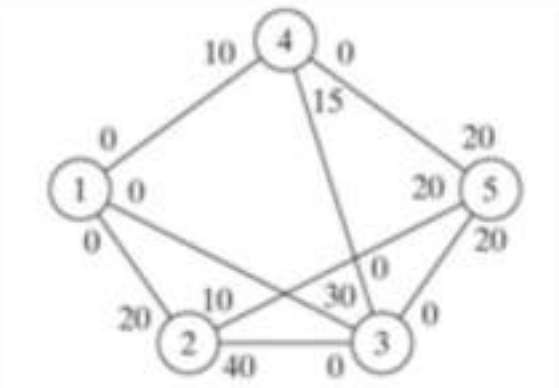


(Iteration 6) No breakthrough

First Network



final Network



$(i \to j)$ $(j \to i)$

initial - final

$(\alpha, \beta) \longrightarrow \alpha > 0 \Rightarrow i \to j$

Arc	$(\bar{C}_{ij}, \bar{C}_{ji}) - (c_{ij}, c_{ji})_0$		Flow amount	Direction
	$1 \to 2$ ↖ ↗ $2 \to 1$			
(1,2)	(20,0) - (0,20)	= (20,-20)	20	$1 \to 2$
(1,3)	(30,0) - (0,30)	= (30,-30)	30	$1 \to 3$
(1,4)	(10,0) - (0,10)	= (10,-10)	10	$1 \to 4$
(2,3)	(40,0) - (40,0)	= (0,0)	———	———
(2,5)	(30,0) - (10,20)	= (20,-20)	20	$2 \to 5$
(3,4)	(10,5) - (0,15)	= (10,-10)	10	$3 \to 4$
(3,5)	(20,0) - (0,20)	= (20,-20)	20	$3 \to 5$
(4,5)	(20,0) - (0,20)	= (20,-20)	20	$4 \to 5$

(3,7) EX: (-10,10)

Flow amount : 10

Direction : $7 \to 3$

Seni Gok Seviyorum ❤ -Yas (Yasemin)

Step 6. Maximal flow in the network is $F = f_1 + f_2 + \cdots + f_5 = 20 + 10 + 10 + 10 + 10 = 60$ units. The flow in the different arcs is computed by subtracting the last residuals (c_{ij}, c_{ji}) in iterations 6 from the initial capacities $(\overline{C}_{ij}, \overline{C}_{ji})$, as the following table shows.

Arc	$(\overline{C}_{ij}, \overline{C}_{ji}) - (c_{ij}, c_{ji})_6$	Flow amount	Direction	Surplus Capacity
(1,2)	(20,0) - (0,20) = (20,-20)	20	1 → 2	——
(1,3)	(30,0) - (0,30) = (30,-30)	30	1 → 3	——
(1,4)	(10,0) - (0,10) = (10,-10)	10	1 → 4	——
* (2,3)	(40, 0) - (40, 0) = (0, 0)	0	—	2 → 3 : 40
(2,5)	(30, 0) - (10, 20) = (20, −20)	20	2 → 5	2 → 5 : 10
(3,4)	(10, 5) - (0, 15) = (10, −10)	10	3 → 4	3 → 4 : — 4 → 3 : 5
(3,5)	(20, 0) - (0, 20) = (20, −20)	20	3 → 5	——
(4,5)	(20, 0) - (0, 20) = (20, −20)	20	4 → 5	——

Homework (MF-4)

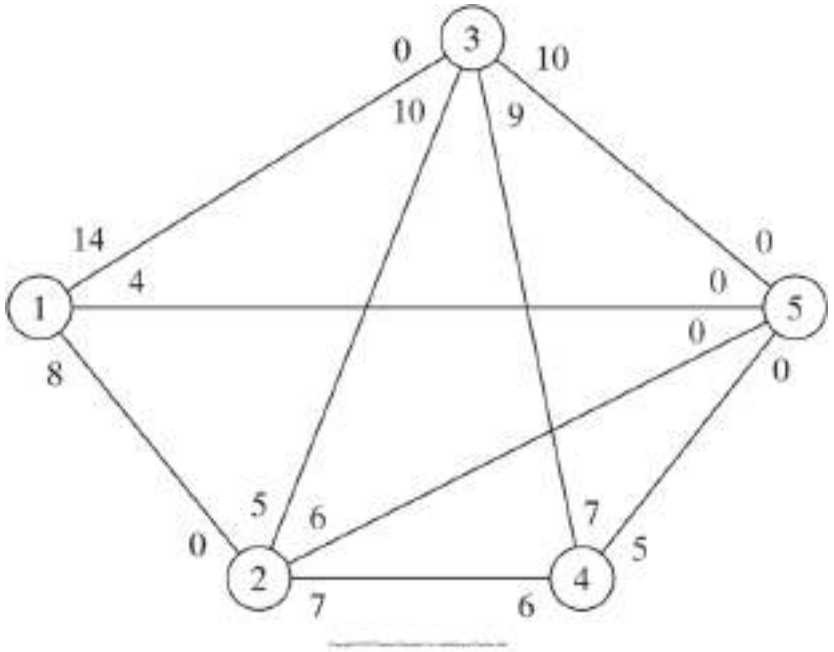
1) In Example 6.4-2,

(a) Determine the surplus capacities for all the arcs.

(b) Determine the amount of flow through nodes 2, 3, and 4.

(c) Can the network flow be increased by increasing the capacities in the directions $3 \rightarrow 5$ and $4 \rightarrow 5$?

2) Determine the maximal flow and the optimum flow in each arc for the network in Figure 6.32.



the amount of flow through

node 2 is 20

3 is 30

4 is 20

Figure 6.32 Network for Problem 2.

Linear Programming Formulation of the MF Problem

Define $x_{ij} = \text{amount of flow in arc}(\text{i, j})$

$c_{ij} = \text{capacity of arc}(\text{i, j})$

$s = \text{index of source node}$

$t = \text{index of sink node}$

Thus, the objective of linear program becomes

$$\text{maximize } V = \sum_j x_{sj}$$

$$\sum_j x_{ij} - \sum_k x_{jk} = \begin{cases} -V & , j = s \\ 0 & , j \neq s,t \\ +V & , j = t \end{cases}$$

$$0 \leq x_{ij} \leq c_{ij} \ , \quad \forall i, j$$

In the maximal flow model of Figure 6.29 (Example 6.4-2), $s = 1$ and $t = 5$. The following table summarizes the associated LP with two different, but equivalent, objective functions depending on whether we maximize the output from start node 1 ($= z_1$) or the input to terminal node 5 ($= z_2$).

	x_{12}	x_{13}	x_{14}	x_{23}	x_{25}	x_{34}	x_{35}	x_{43}	x_{45}	
Maximize $z_1 =$	1	1	1							
Maximize $z_2 =$					1		1		1	
Node 2	1			−1	−1					= 0
Node 3		1		1		−1	−1	1		= 0
Node 4			1			1		−1	−1	= 0
Capacity	20	30	10	40	30	10	20	5	20	

The optimal solution using either objective function is

$$x_{12} = 20, x_{13} = 30, x_{14} = 10, x_{25} = 20, x_{34} = 10, x_{35} = 20, x_{45} = 20$$

The associated maximum flow is $z_1 = z_2 = 60$.

References

[1] *Ahlatçıoğlu, M. , Tiryaki, F., Kantitatif Karar Verme Teknikleri, YTÜ Yayın No: YTÜ.FE.DK-98.0349, İstanbul-1998.*

[2] *H.A.Taha, Operations Research: An Introduction,* Prentice Hall; 9th edition, *Singapore, 2010.*

Lecture 7

CPM and PERT

(Critical Path Method and Program Evaluation and Review Technique)

6.5 CPM AND PERT

CPM (Critical Path Method) and PERT (Program Evaluation and Review Technique) are network-based methods designed to assist in the planning, scheduling, and control of projects.

A project is defined as a collection of interrelated activities with each activity consuming time and resources. The objective of CPM and PERT is to provide analytic means for scheduling the activities. Figure 6.38 summarizes the steps of the techniques. First, we define the activities of the project, their precedence relationships, and their time requirements. Next, the precedence relationships among the activities are represented by a network. The third step involves specific computations to develop the time schedule for the project. During the actual execution of the project things may not proceed as planned, as some of the activities may be expedited or delayed. When this happens, the schedule must be revised to reflect the realities on the ground. This is the reason for including a feedback loop between the time schedule phase and the network phase, as shown in Figure 6.38.

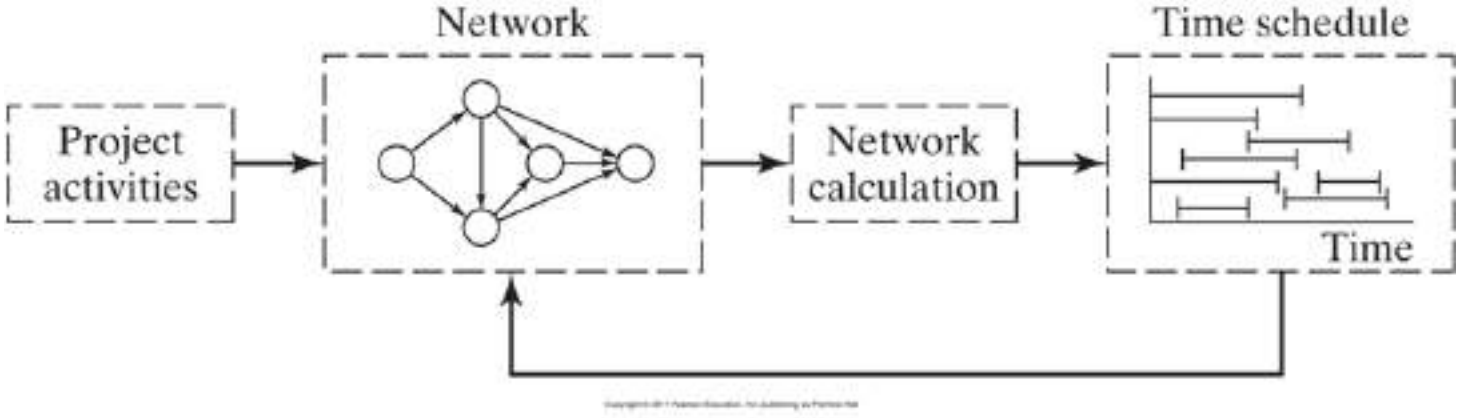


Figure 6.38 Phases for project planning with CPM-PERT

The two techniques, CPM and PERT, which were developed independently, differ in that CPM assumes deterministic activity durations and PERT assumes probabilistic durations. This presentation will start with CPM and then proceed with the details of PERT.

CPM and PERT have been successfully used in many applications, including:

1 Scheduling construction projects such as office buildings, highways, and swimming pools

2 Scheduling the movement of a 400-bed hospital from Portland, Oregon, to a suburban location

3 Developing a countdown and "hold" procedure for the launching of space flights

4 Installing a new computer system

5 Designing and marketing a new product

6 Completing a corporate merger

7 Building a ship

6.5.1 Network Representation

Each activity of the project is represented by an arc pointing in the direction of progress in the project. The nodes of the network establish the precedence relationships among the different activities.

Three rules are available for constructing the network.

Rule 1. *Each activity is represented by one, and only one, arc.*

Rule 2. *Each activity must be identified by two distinct end nodes.*

Figure 6.39 shows how a dummy activity can be used to represent two concurrent activities, *A* and *B*. By definition, a dummy activity, which normally is depicted by a dashed arc, consumes no time or resources. Inserting a dummy activity in one of the four ways shown in Figure 6.39, we maintain the concurrence of *A* and *B*, and provide unique end nodes for the two activities (to satisfy rule 2).

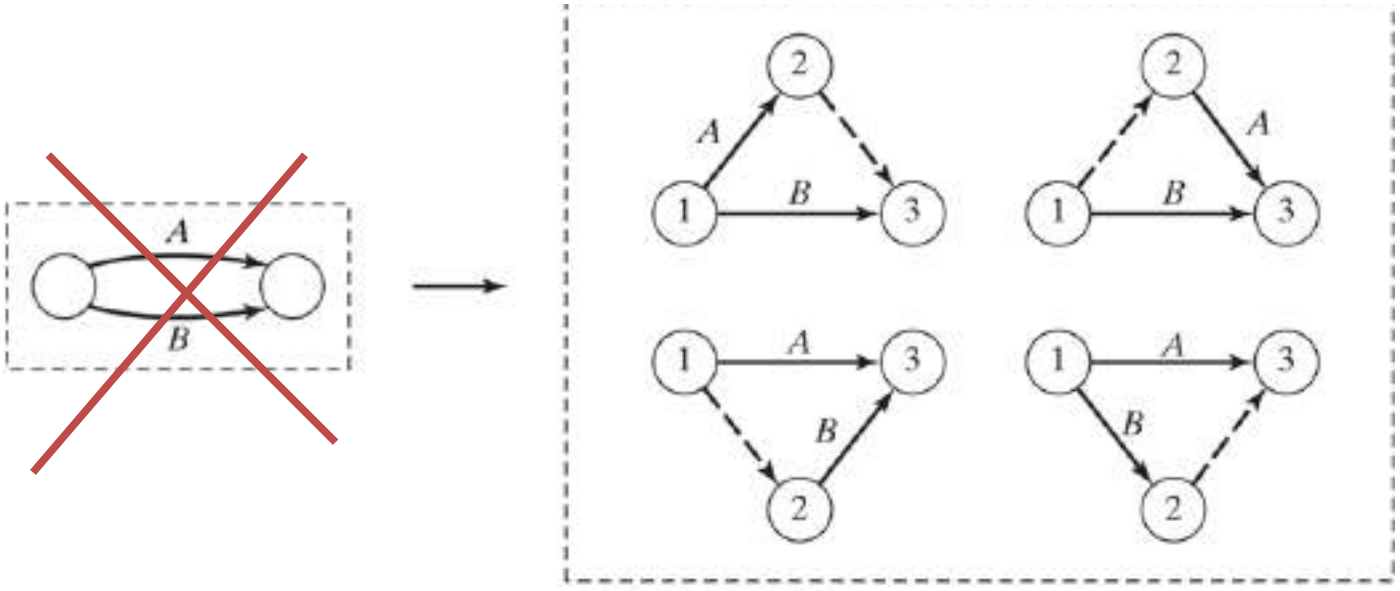


Figure 6.39 Use of dummy activity to produce unique representation of concurrent activities

Rule 3. *To maintain the correct precedence relationships, the following questions must be answered as each activity is added to the network:*

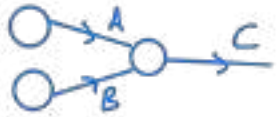
(a) *What activities must immediately precede the current activity?*

(b) *What activities must follow the current activity?*

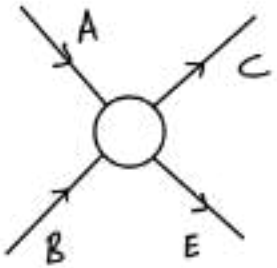
(c) *What activities must occur concurrently with the current activity?*

The answers to these questions may require the use of dummy activities to ensure correct precedences among the activities. For example, consider the following segment of a project:

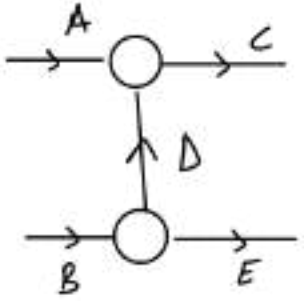
1. Activity C starts immediately after A and B have been completed.
2. Activity E starts only after B has been completed.



Part (a) of Figure 6.40 shows the incorrect representation of the precedence relationship because it requires both A and B to be completed before E can start. In part (b), the use of a dummy activity rectifies the situation.



(a)



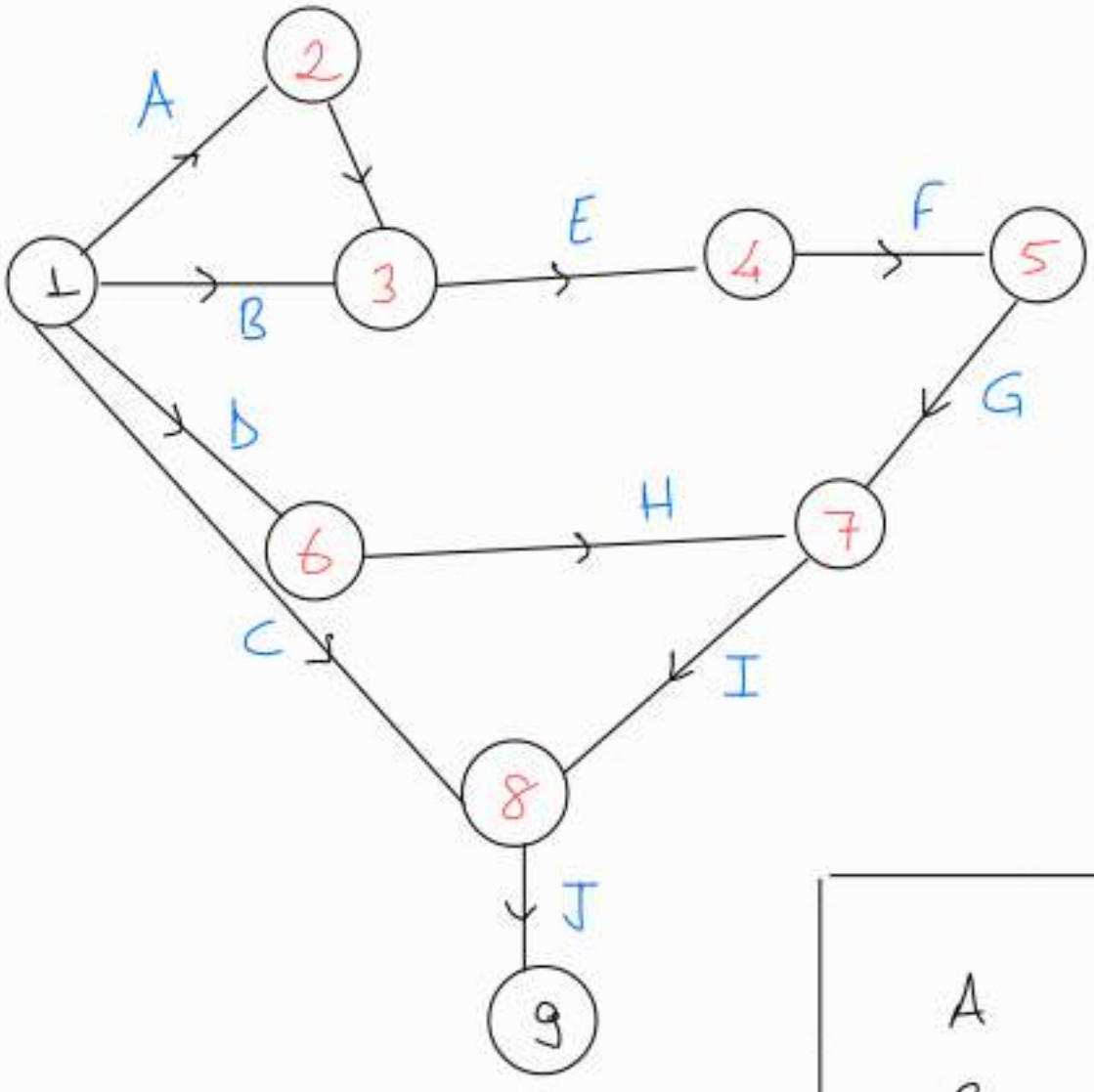
(b)

Figure 6.40
Use of dummy activity to ensure correct precedence relationship

Example 6.5-1

A publisher has a contract with an author to publish a textbook. The (simplified) activities associated with the production of the textbook are given below. The author is required to submit to the publisher a hard copy and a computer file of the manuscript. Develop the associated network for the project.

Activity	Predecessor(s)	Duration (weeks)
A: Manuscript proofreading by editor	—	3
B: Sample pages preparation	—	2
C: Book cover design	—	4
D: Artwork preparation	—	3
E: Author's approval of edited manuscript and sample pages	*A*, *B*	2
F: Book formatting	*E*	4
G: Author's review of formatted pages	*F*	2
H: Author's review of artwork	*D*	1
I: Production of printing plates	*G*, *H*	2
J: Book production and binding	*C*, *I*	4



A	—
B	—
C	—
D	—
E	A, B
F	E
G	F
H	D
I	H, G
J	C, I

1 ve 9 her zaman sabit.
(Source - Sink)

Diğer numaraların yerleri rastgele değişebilir.

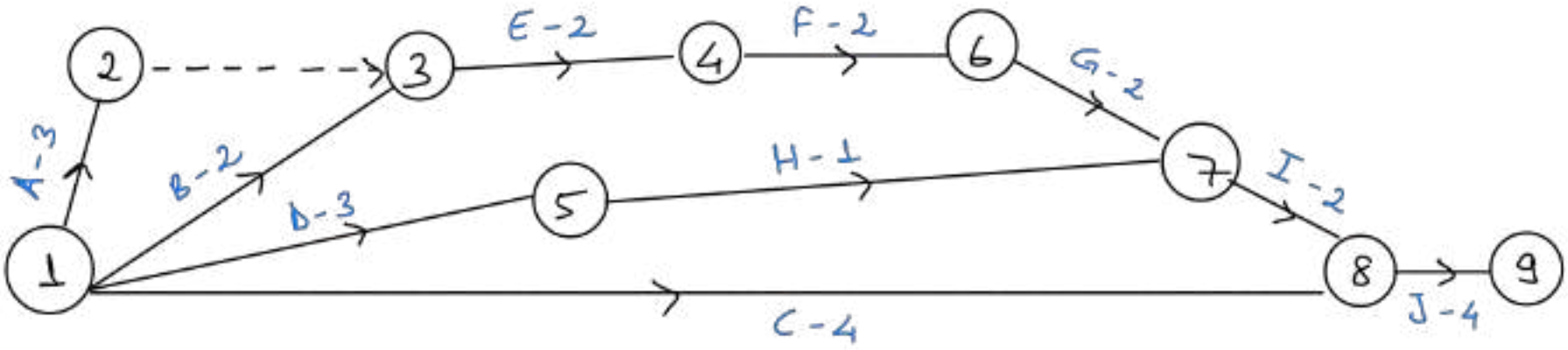


Figure 6.41 Project network for Example 6.5-1

Figure 6.41 provides the network describing the precedence relationships among the different activities. Dummy activity (2, 3) produces unique end nodes for concurrent activities A and B. It is convenient to number the nodes in ascending order in the direction of progress in the project.

PROBLEM SET 6.5A

1. Construct the project network comprised of activities *A* to *L* with the following precedence relationships:
   - **(a)** A, B, and *C*, the first activities of the project, can be executed concurrently.
   - **(b)** A and *B* precede *D*.
   - **(c)** B precedes *E*, *F*, and *H*.
   - **(d)** F and *C* precede *G*.
   - **(e)** E and *H* precede *I* and *J*.
   - **(f)** C, D, F, and *J* precede *K*.
   - **(g)** K precedes *L*.
   - **(h)** I, G, and *L* are the terminal activities of the project.

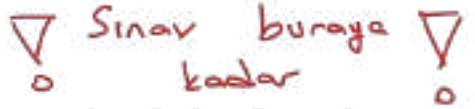
*3. The footings of a building can be completed in four consecutive sections. The activities for each section include (1) digging, (2) placing steel, and (3) pouring concrete. The digging of one section cannot start until that of the preceding section has been completed. The same restriction applies to pouring concrete. Develop the project network.

5. An opinion survey involves designing and printing questionnaires, hiring and training personnel, selecting participants, mailing questionnaires, and analyzing the data. Construct the project network, stating all assumptions.

6. The activities in the following table describe the construction of a new house. Construct the associated project network.

	Activity	Predecessor(s)	Duration (days)
A:	Clear site	—	1
B:	Bring utilities to site	—	2
C:	Excavate	*A*	1
D:	Pour foundation	*C*	2
E:	Outside plumbing	*B, C*	6
F:	Frame house	*D*	10
G:	Do electric wiring	*F*	3
H:	Lay floor	*G*	1
I:	Lay roof	*F*	1
J:	Inside plumbing	*E, H*	5
K:	Shingling	*I*	2
L:	Outside sheathing insulation	*F, J*	1
M:	Install windows and outside doors	*F*	2
N:	Do brick work	*L, M*	4
O:	Insulate walls and ceiling	*G, J*	2
P:	Cover walls and ceiling	*O*	2
Q:	Insulate roof	*I, P*	1
R:	Finish interior	*P*	7
S:	Finish exterior	*I, N*	7
T:	Landscape	*S*	3

6.5.2 Critical Path (CPM) Computations



The end result in CPM is the construction of the time schedule for the project (see Figure 6.38). To achieve this objective conveniently, we carry out special computations that produce the following information:

1. Total duration needed to complete the project.
2. Classification of the activities of the project as *critical* and *noncritical*.

An activity is said to be **critical** if there is no "leeway" in determining its start and finish times. A **noncritical** activity allows some scheduling slack, so that the start time of the activity can be advanced or delayed within limits without affecting the completion date of the entire project.

To carry out the necessary computations, we define an **event** as a point in time at which activities are terminated and others are started. In terms of the network, an event corresponds to a node. Define

$\square_j$ = Earliest occurrence time of event j (E_i)
Δ_j = Latest occurrence time of event j (L_j)
D_{ij} = Duration of activity (i, j)

The definitions of the *earliest* and *latest* occurrences of event j are specified relative to the start and completion dates of the entire project.

The critical path calculations involve two passes: The **forward pass** determines the *earliest* occurrence times of the events, and the **backward pass** calculates their *latest* occurrence times.

Forward Pass (Earliest Occurrence Times, □). The computations start at node 1 and advance recursively to end node n.

Initial Step. Set $\square_1 = 0$ to indicate that the project starts at time 0.

General Step j. Given that nodes $p, q, \ldots$, and v are linked *directly* to node j by incoming activities $(p, j), (q, j), \ldots$, and (v, j) and that the earliest occurrence times of events (nodes) $p, q, \ldots$, and v have already been computed, then the earliest occurrence time of event j is computed as

The forward pass is complete when $\square_n$ at node n has been computed. By definition $\square_j$ represents the longest path (duration) to node j.

Backward Pass (Latest Occurrence Times, Δ). Following the completion of the forward pass, the backward pass computations start at node n and end at node 1.

Initial Step. Set $\Delta_n = \square_n$ to indicate that the earliest and latest occurrences of the last node of the project are the same.

General Step j. Given that nodes $p, q, \ldots,$ and v are linked *directly* to node j by *outgoing* activities $(j, p), (j, q), \ldots,$ and (j, v) and that the latest occurrence times of nodes $p, q, \ldots,$ and v have already been computed, the latest occurrence time of node j is computed as

The backward pass is complete when Δ_1 at node 1 is computed. At this point, $\Delta_1 = \square_1 \ (= 0)$.

Based on the preceding calculations, an activity (i, j) will be *critical* if it satisfies three conditions.

The three conditions state that the earliest and latest occurrence times of end nodes i and j are equal and the duration D_{ij} fits "tightly" in the specified time span. An activity that does not satisfy all three conditions is thus *noncritical*.

By definition, the critical activities of a network must constitute an uninterrupted path that spans the entire network from start to finish.

Example 6.5-2

Determine the critical path for the project network in Figure 6.42 . All the durations are in days.

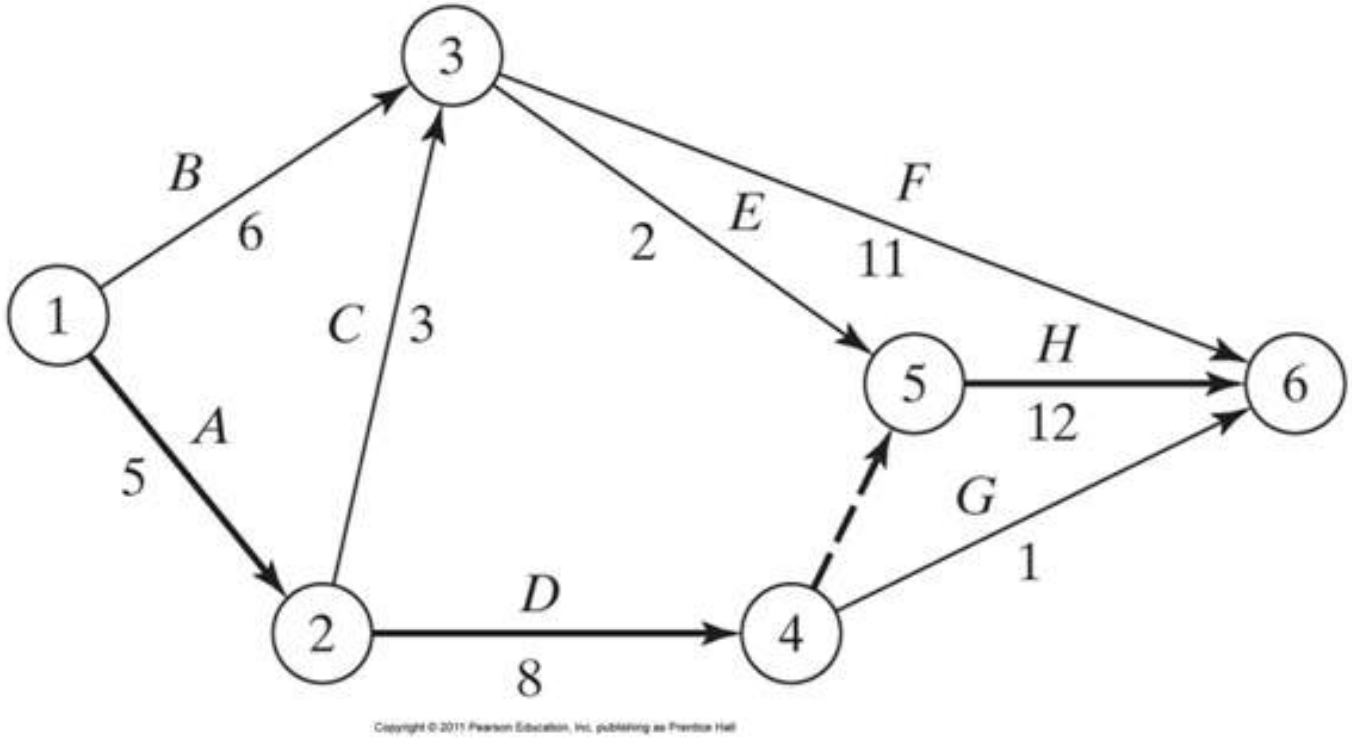


Figure 6.42 Project network for Example 6.5-2

Forward Pass

Node 1. Set $\square_1 = 0$

Node 2. $\square_2 = \square_1 + D_{12} = 0 + 5 = 5$

Node 3. $\square_3 = \max\{\square_1 + D_{13}, \square_2 + D_{23}\} = \max\{0 + 6, 5 + 3\} = 8$

Node 4. $\square_4 = \square_2 + D_{24} = 5 + 8 = 13$

Node 5. $\square_5 = \max\{\square_3 + D_{35}, \square_4 + D_{45}\} = \max\{8 + 2, 13 + 0\} = 13$

Node 6. $\square_6 = \max\{\square_3 + D_{36}, \square_4 + D_{46}, \square_5 + D_{56}\}$

$= \max\{8 + 11, 13 + 1, 13 + 12\} = 25$

The computations show that the project can be completed in 25 days.

Backward Pass

Node 6. Set $\Delta_6 = \square_6 = 25$

Node 5. $\Delta_5 = \Delta_6 - D_{56} = 25 - 12 = 13$

Node 4. $\Delta_4 = \min\{\Delta_6 - D_{46}, \Delta_5 - D_{45}\} = \min\{25 - 1, 13 - 0\} = 13$

Node 3. $\Delta_3 = \min\{\Delta_6 - D_{36}, \Delta_5 - D_{35}\} = \min\{25 - 11, 13 - 2\} = 11$

Node 2. $\Delta_2 = \min\{\Delta_4 - D_{24}, \Delta_3 - D_{23}\} = \min\{13 - 8, 11 - 3\} = 5$

Node 1. $\Delta_1 = \min\{\Delta_3 - D_{13}, \Delta_2 - D_2\} = \min\{11 - 6, 5 - 5\} = 0$

Correct computations will always end with $\Delta_1 = 0$.

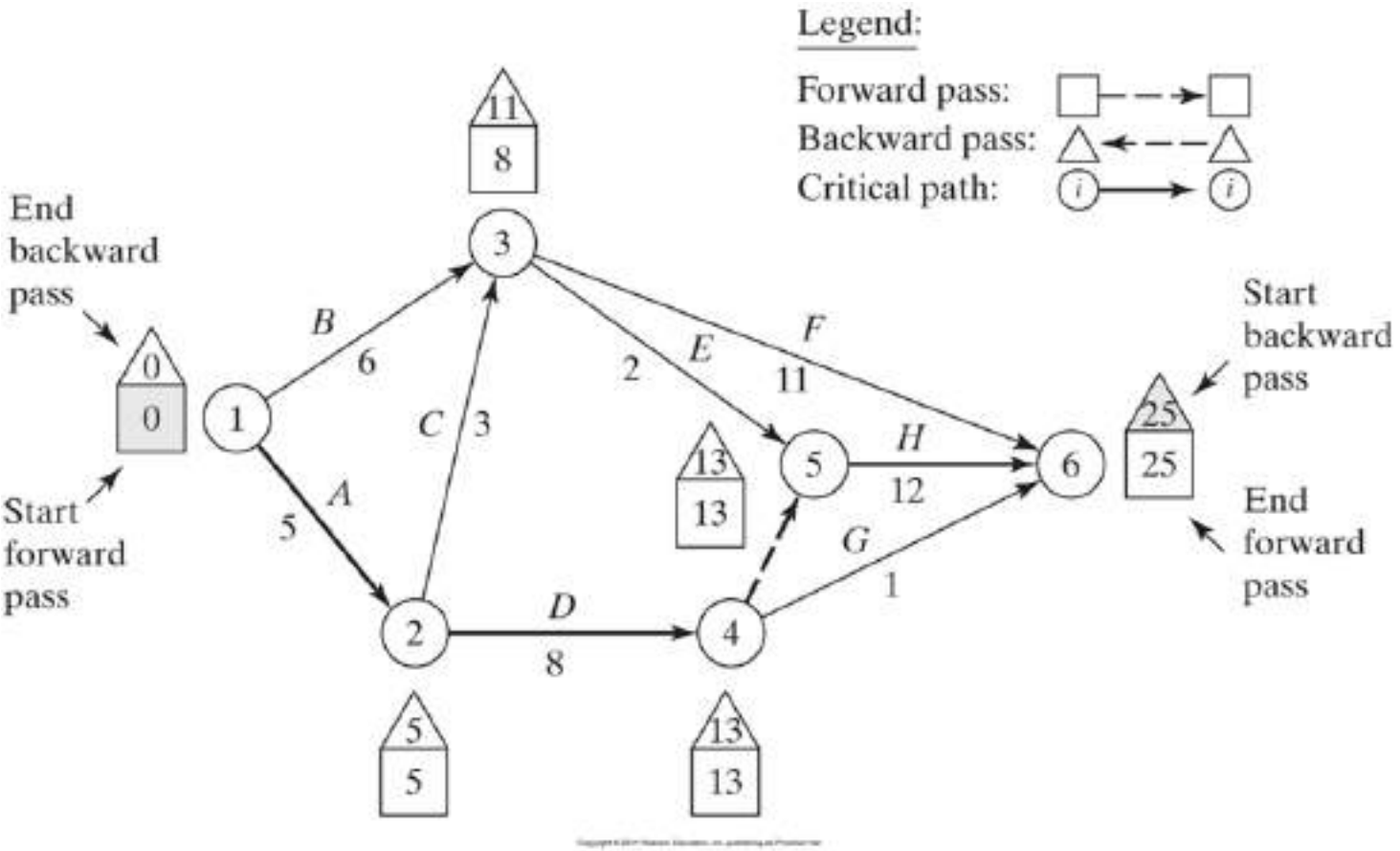


Figure 6.42 Project network for Example 6.5-2

The forward and backward pass computations can be made directly on the network as shown in Figure 6.42. Applying the rules for determining the critical activities, the critical path is $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6$, which, as should be expected, spans the network from start (node 1) to finish (node 6). The sum of the durations of the critical activities [(1, 2), (2, 4), (4, 5), and (5, 6)] equals the duration of the project (= 25 days). Observe that activity (4, 6) satisfies the first two conditions for a critical activity ($\Delta_4 = \square_4 = 13$ and $\Delta_5 = \square_5 = 25$) but not the third ($\square_6 - \square_4 \neq D_{46}$). Hence, the activity is noncritical.

PROBLEM SET 6.5B

*1. Determine the critical path for the project network in Figure 6.43.

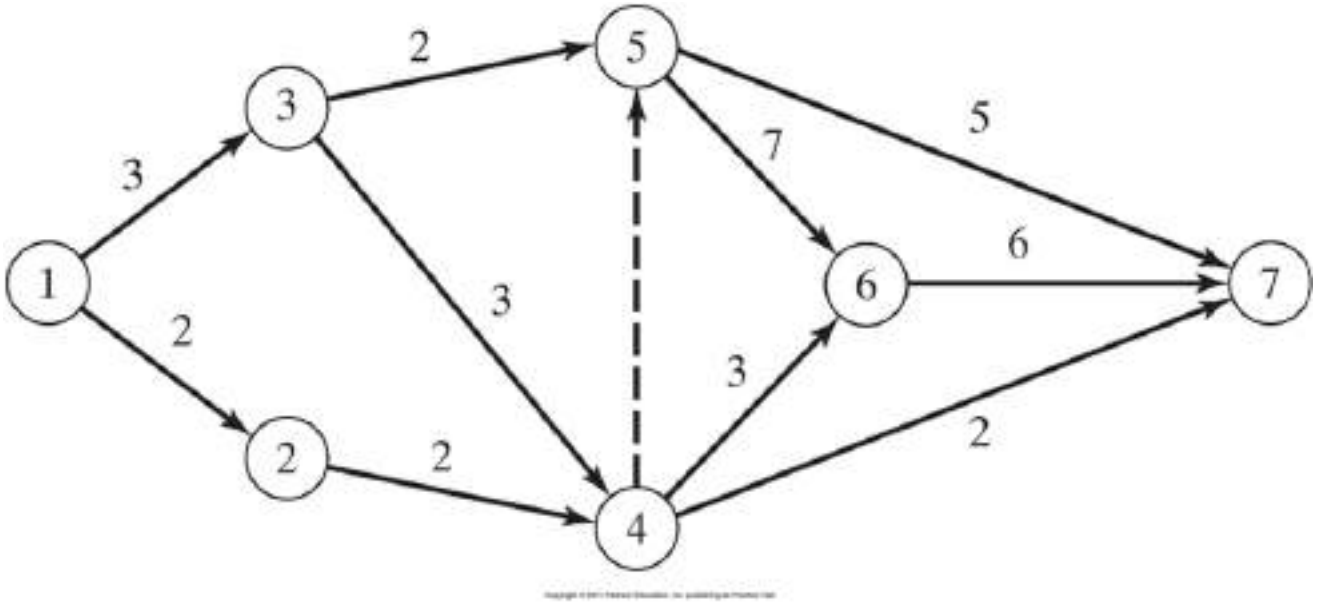
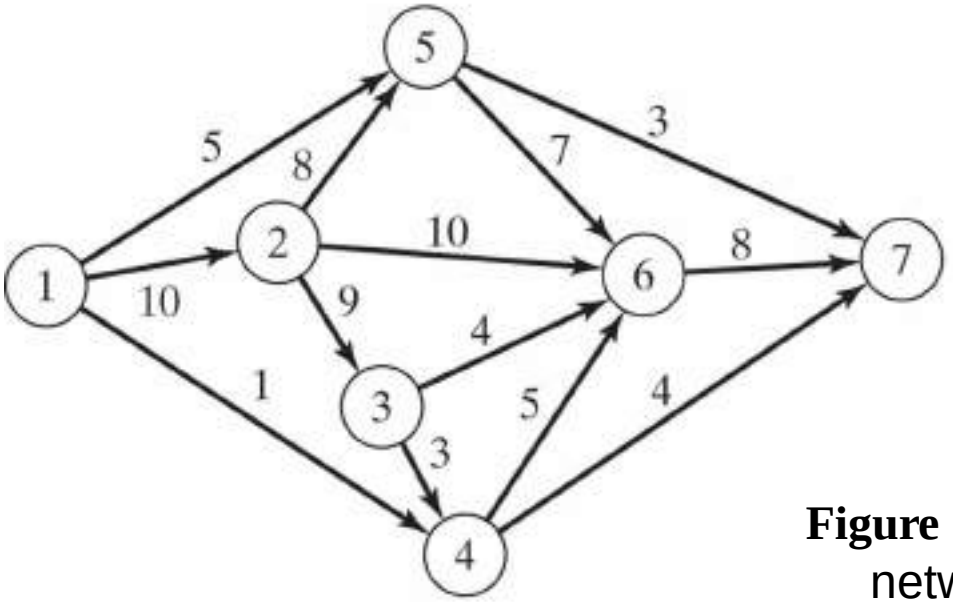
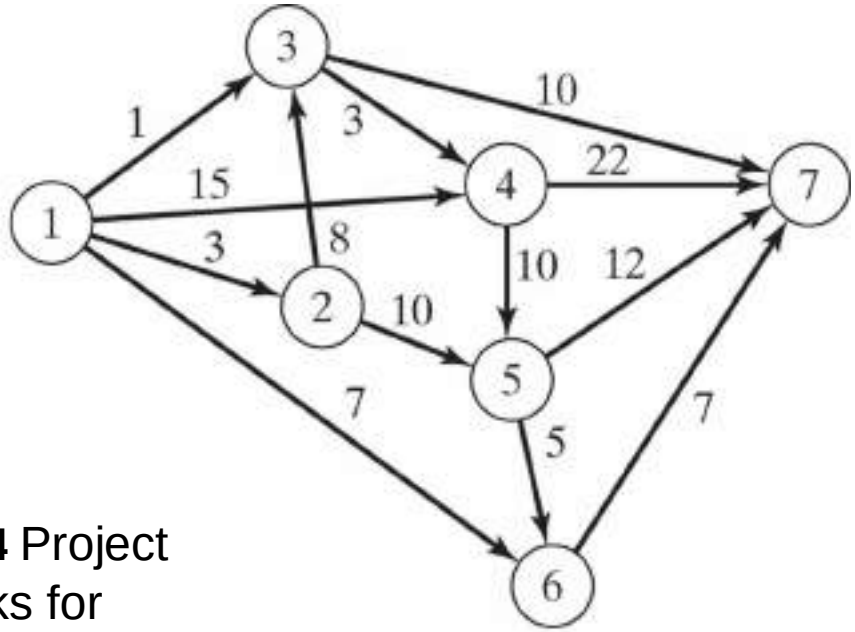


Figure 6.43 Project networks for Problem 1, Set 6.5b

2. Determine the critical path for the project networks in Figure 6.44.



Project (a)



Project (b)

Figure 6.44 Project networks for Problem 2, Set 6.5b

6.5.3 Construction of the Time Schedule

This section shows how the information obtained from the calculations in Section 6.5.2 can be used to develop the time schedule. We recognize that for an activity (i, j), $\square_i$ represents the *earliest start time*, and Δ_j represents the *latest completion time*. This means that the interval $(\square_i, \Delta_j)$ delineates the (maximum) span during which activity (i, j) may be scheduled without delaying the entire project.

Construction of Preliminary Schedule. The method for constructing a preliminary schedule is illustrated by an example.

Example 6.5-3

Determine the time schedule for the project of Example 6.5-2 (Figure 6.42).

Figure 6.45 Preliminary schedule for the project of Example 6.5-2

We can get a preliminary time schedule for the different activities of the project by delineating their respective time spans as shown in Figure 6.45. Two observations are in order.

1. The critical activities (shown by solid lines) must be stacked one right after the other to ensure that the project is completed within its specified 25-day duration.
2. The noncritical activities (shown by dashed lines) have time spans that are larger than their respective durations, thus allowing slack (or "leeway") in scheduling them within their allotted time intervals.

How should we schedule the noncritical activities within their respective spans? Normally, it is preferable to start each noncritical activity as early as possible. In this manner, slack periods will remain opportunely available at the end of the allotted span where they can be used to absorb unexpected delays in the execution of the activity. It may be necessary, however, to delay the start of a noncritical activity past its earliest start time. For example, in Figure 6.45, suppose that each of the noncritical activities E and F requires the use of a bulldozer, and that only one is available. Scheduling both E and F as early as possible requires two bulldozers between times 8 and 10. We can remove the overlap by starting E at time 8 and pushing the start time of F to somewhere between times 10 and 14.

If all the noncritical activities can be scheduled as early as possible, the resulting schedule automatically is feasible. Otherwise, some precedence relationships may be violated if noncritical activities are delayed past their earliest time. Take for example activities C and E in Figure 6.45. In the project network (Figure 6.42), though C must be completed before E, the spans of C and E in Figure 6.45 allow us to schedule C between times 6 and 9, and E between times 8 and 10, which violates the requirement that C precede E. The need for a "red flag" that automatically reveals schedule conflict is thus evident. Such information is provided by computing the *floats* for the noncritical activities.

Determination of the Floats. Floats are the slack times available within the allotted span of the noncritical activity. The most common are the **total float** and the **free float.**

Figure 6.46 gives a convenient summary for computing the total float (TF_{ij}) and the free float (FF_{ij}) for an activity (i, j). The total float is the excess of the time span defined from the *earliest* occurrence of event i to the *latest* occurrence of event j over the duration of (i, j)—that is,

The free float is the excess of the time span defined from the *earliest* occurrence of event i to the *earliest* occurrence of event j over the duration of (i, j)—that is,

By definition, $FF_{ij} \leq TF_{ij}$.

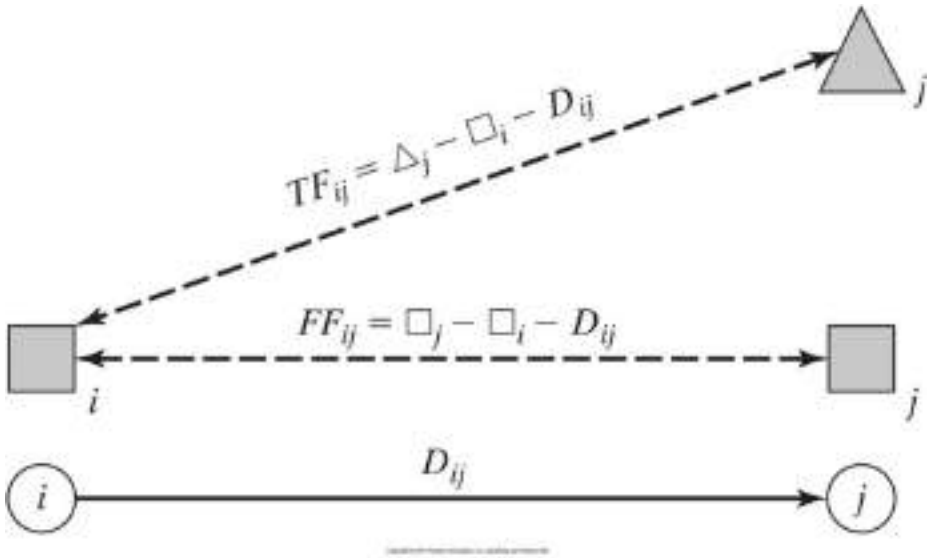


Figure 6.46
Computation of total and free floats

Red-Flagging Rule. *For a noncritical activity (i, j)*

(a) *If $FF_{ij} = TF_{ij}$, then the activity can be scheduled anywhere within its $(\square_j, \Delta_j)$ span without causing schedule conflict.*

(b) *If $FF_{ij} < TF_{ij}$, then the start of the activity can be delayed by at most FF_{ij} relative to its earliest start time $(\square_i)$ without causing schedule conflict. Any delay larger than FF_{ij} (but not more than TF_{ij}) must be coupled with an equal delay relative to $\square_j$ in the start time of all the activities leaving node j.*

The implication of the rule is that a noncritical activity (i, j) will be red-flagged if its $FF_{ij} < TF_{ij}$. This red flag is important only if we decide to delay the start of the activity past its earliest start time, $\square_i$, in which case we must pay attention to the start times of the activities leaving node j to avoid schedule conflicts.

Example 6.5-4

Compute the floats for the noncritical activities of the network in Example 6.5-2, and discuss their use in finalizing a schedule for the project.

The following table summarizes the computations of the total and free floats. It is more convenient to do the calculations directly on the network using the procedure in Figure 6.42.

Noncritical activity	Duration	Total float (TF)	Free float (FF)
$B(1,3)$	6	$11 - 0 - 6 = 5$	$8 - 0 - 6 = 2$
$C(2,3)$	3	$11 - 5 - 3 = 3$	$8 - 5 - 3 = 0$
$E(3,5)$	2	$13 - 8 - 2 = 3$	$13 - 8 - 2 = 3$
$F(3,6)$	11	$25 - 8 - 11 = 6$	$25 - 8 - 11 = 6$
$G(4,6)$	1	$25 - 13 - 1 = 11$	$25 - 13 - 1 = 11$

The computations red-flag activities B and C because their $FF < TF$. The remaining activities (E, F, and G) have $FF = TF$, and hence may be scheduled anywhere between their earliest start and latest completion times.

To investigate the significance of the red-flagged activities, consider activity B. Because its $TF = 5$ days, this activity can start as early as time 0 or as late as time 5 (see Figure 6.45). However, because its $FF = 2$ days, starting B anywhere between time 0 and time 2 will have no effect on the succeeding activities E and F. If, however, activity B must start at time $2 + d$ (≤ 5), then the start times of the immediately succeeding activities E and F must be pushed forward past their earliest start time ($= 8$) by at least d. In this manner, the precedence relationship between B and its successors E and F is preserved.

Turning to red-flagged activity C, we note that its $FF = 0$. This means that *any* delay in starting C past its earliest start time ($= 5$) must be coupled with at least an equal delay in the start of its successor activities E and F.

PROBLEM SET 6.5C

*3. For each of the following activities, determine the maximum delay in the starting time relative to its earliest start time that will allow all the immediately succeeding activities to be scheduled anywhere between their earliest and latest completion times.

(a) $TF = 10, FF = 10, D = 4$

(b) $TF = 10, FF = 5, D = 4$

(c) $TF = 10, FF = 0, D = 4$

*5. In the project of Example 6.5-2 (Figure 6.42), assume that the durations of activities B and F are changed from 6 and 11 days to 20 and 25 days, respectively.

(a) Determine the critical path.

(b) Determine the total and free floats for the network, and identify the red-flagged activities.

(c) If activity A is started at time 5, determine the earliest possible start times for activities C, D, E, and G.

(d) If activities F, G, and H require the same equipment, determine the minimum number of units needed of this equipment.

6.5.4 Linear Programming Formulation of CPM

A CPM problem can thought of as the opposite of the shortest-route problem (Section 6.3), in the sense that we are interested in finding the *longest* route of a unit flow entering at the start node and terminating at the finish node. We can thus apply the shortest route LP formulation in Section 6.3.3 to CPM in the following manner. Define

x_{ij} = Amount of flow in activity (i, j), for all defined i and j
D_{ij} = Duration of activity (i, j), for all defined i and j

Thus, the objective function of the linear program becomes

$$\text{Maximize } z = \sum_{\substack{\text{all defined} \\ \text{activities } (i,j)}} D_{ij}x_{ij}$$

(Compare with the shortest route LP formulation in Section 6.3.3 where the objective function is minimized.) For each node, there is one constraint that represents the conservation of flow:

$$\text{Total input flow} = \text{Total output flow}$$

All the variables, x_{ij}, are nonnegative.

Example 6.5-5

The LP formulation of the project of Example 6.5-2 (Figure 6.42) is given below. Note that nodes 1 and 6 are the start and finish nodes, respectively.

	A	B	C	D	E	F	Dummy	G	H	
	x_{12}	x_{13}	x_{23}	x_{24}	x_{35}	x_{36}	x_{45}	x_{46}	x_{56}	
Maximize z =	6	6	3	8	2	11	0	1	12	
Node 1	−1	−1								= −1
Node 2	1		−1	−1						= 0
Node 3		1	1		−1	−1				= 0
Node 4				1			−1	−1		= 0
Node 5					1		1		−1	= 0
Node 6						1		1	1	= 1

The optimum solution is

$$z = 25, x_{12}(A) = 1, x_{24}(D) = 1, x_{45}(\text{Dummy}) = 1, x_{56}(H) = 1, \text{all others} = 0$$

The solution defines the critical path as $A \rightarrow D \rightarrow$ Dummy $\rightarrow H$, and the duration of the project is 25 days. The LP solution is not complete, because it determines the critical path, but does not provide the data needed to construct the CPM chart.

6.5.5 PERT Networks

PERT differs from CPM in that it bases the duration of an activity on three estimates:

1. **Optimistic time,** a, which occurs when execution goes extremely well.
2. **Most likely time,** m, which occurs when execution is done under normal conditions.
3. **Pessimistic time,** b, which occurs when execution goes extremely poorly.

The range (a, b) encloses all possible estimates of the duration of an activity. The estimate m lies somewhere in the range (a, b). Based on the estimates, the average duration time, $\overline{D}$, and variance, v, are approximated as:

$$\overline{D} = \frac{a + 4m + b}{6} \qquad v = \left(\frac{b - a}{6}\right)^2$$

CPM calculations given in Sections 6.5.2 and 6.5.3 may be applied directly, with $\overline{D}$ replacing the single estimate D.

It is possible now to estimate the probability that a node j in the network will occur by a prespecified scheduled time, S_j. Let e_j be the earliest occurrence time of node j. Because the durations of the activities leading from the start node to node j are random variables, e_j also must be a random variable. Assuming that all the activities in the network are statistically independent, we can determine the mean, $E\{e_j\}$, and variance, $\text{var}\{e_j\}$, in the following manner. If there is only one path from the start node to node j, then the mean is the sum of expected durations, $\overline{D}$, for all the activities along this path and the variance is the sum of the variances, v, of the same activities. On the other hand, if more than one path leads to node j, then it is necessary first to determine the statistical distribution of the duration of the longest path. This problem is rather difficult because it is equivalent to determining the distribution of the maximum of two or more random variables. A simplifying assumption thus calls for computing the mean and variance, $E\{e_j\}$ and $\text{var}\{e_j\}$, as those of the path to node j that has the largest sum of *expected* activity durations. If two or more paths have the same mean, the one with the largest variance is selected because it reflects the most uncertainty and, hence, leads to a more conservative estimate of probabilities.

Once the mean and variance of the path to node j, $E\{e_j\}$ and $\text{var}\{e_j\}$, have been computed, the probability that node j will be realized by a preset time S_j is calculated using the following formula:

$$P\{e_j \leq S_j\} = P\left\{\frac{e_j - E\{e_j\}}{\sqrt{\text{var}\{e_j\}}} \leq \frac{S_j - E\{e_j\}}{\sqrt{\text{var}\{e_j\}}}\right\} = P\{z \leq K_j\}$$

where

z = Standard normal random variable

$$K_j = \frac{S_j - E\{e_j\}}{\sqrt{\text{var}\{e_j\}}}$$

The standard normal variable z has mean 0 and standard deviation 1 (see Section 12.4.4). Justification for the use of the normal distribution is that e_j is the sum of independent random variables. According to the *central limit theorem* (see Section 12.4.4), e_j is approximately normally distributed.

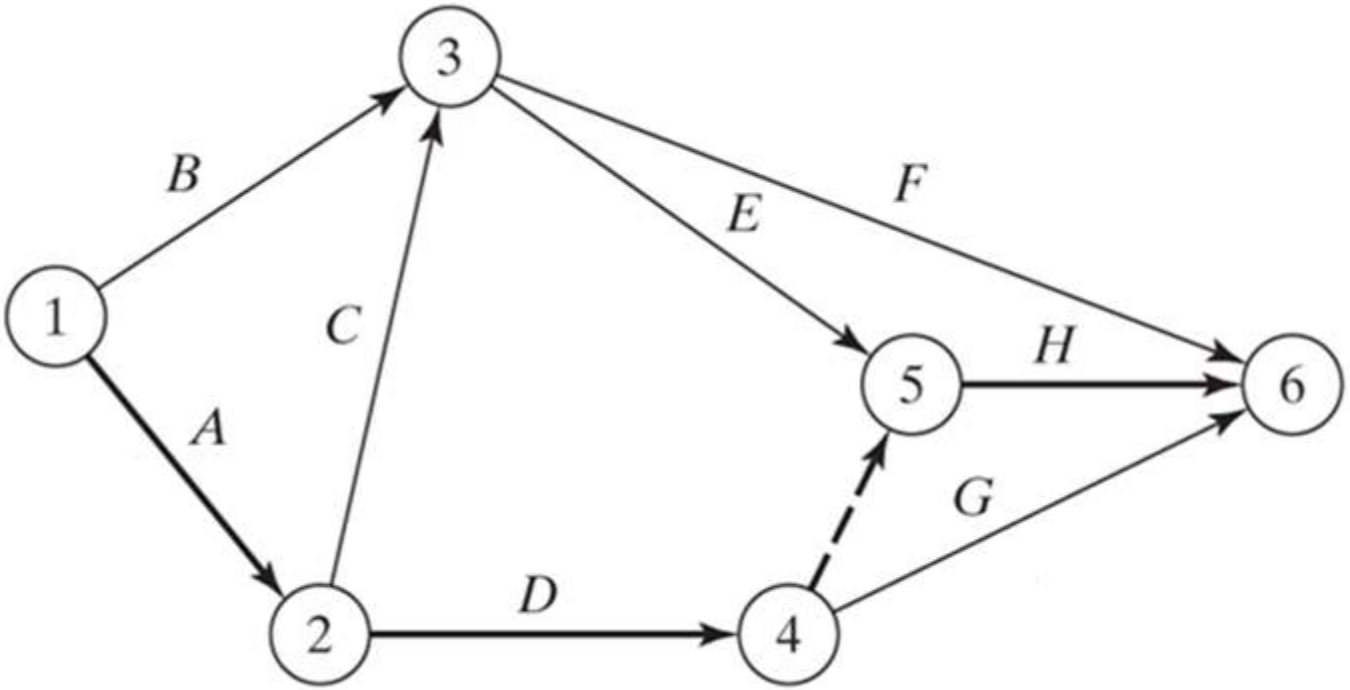
Example 6.5-6

Consider the project of Example 6.5-2. To avoid repeating critical path calculations, the values of a, m, and b in the table below are selected such that $\overline{D_{ij}} = D_{ij}$ for all i and j in Example 6.5-2.

Activity	i–j	(a, m, b)	Activity	i–j	(a, m, b)
A	1–2	(3, 5, 7)	E	3–5	(1, 2, 3)
B	1–3	(4, 6, 8)	F	3–6	(9, 11, 13)
C	2–3	(1, 3, 5)	G	4–6	(1, 1, 1)
D	2–4	(5, 8, 11)	H	5–6	(10, 12, 14)

The mean $\overline{D_{ij}}$ and variance v_{ij} for the different activities are given in the following table. Note that for a dummy activity $(a, m, b) = (0, 0, 0)$, hence its mean and variance also equal zero.

Activity	i–j	$\overline{D}_{ij}$	V_{ij}	Activity	i–j	$\overline{D}_{ij}$	V_{ij}
A	1–2	5	.444	E	3–5	2	.111
B	1–3	6	.444	F	3–6	11	.444
C	2–3	3	.444	G	4–6	1	.000
D	2–4	8	1.000	H	5–6	12	.444



The next table gives the longest path from node 1 to the different nodes, together with their associated mean and standard deviation.

Node	Longest path based on mean durations	Path mean	Path standard deviation
2	1–2	5.00	0.67
3	1–2–3	8.00	0.94
4	1–2–4	13.00	1.20
5	1–2–4–5	13.00	1.20
6	1–2–4–5–6	25.00	1.37

Finally, the following table computes the probability that each node is realized by time S_j specified by the analyst.

Node j	Longest path	Path mean	Path standard deviation	S_j	K_j	$P\{z \leq K_j\}$
2	1–2	5.00	0.67	5.00	0	.5000
3	1–2–3	8.00	0.94	11.00	3.19	.9993
4	1–2–4	13.00	1.20	12.00	−.83	.2033
5	1–2–4–5	13.00	1.20	14.00	.83	.7967
6	1–2–4–5–6	25.00	1.37	26.00	.73	.7673

PROBLEM SET 6.5E

1. Consider Problem 2, Set 6.5b. The estimates (a, m, b) are listed below. Determine the probabilities that the different nodes of the project will be realized without delay.

Project (a)				Project (b)			
Activity	(a, m, b)	Activity	(a, m, b)	Activity	(a, m, b)	Activity	(a, m, b)
1-2	(5, 6, 8)	3-6	(3, 4, 5)	1-2	(1, 3, 4)	3-7	(12, 13, 14)
1-4	(1, 3, 4)	4-6	(4, 8, 10)	1-3	(5, 7, 8)	4-5	(10, 12, 15)
1-5	(2, 4, 5)	4-7	(5, 6, 8)	1-4	(6, 7, 9)	4-7	(8, 10, 12)
2-3	(4, 5, 6)	5-6	(9, 10, 15)	1-6	(1, 2, 3)	5-6	(7, 8, 11)
2-5	(7, 8, 10)	5-7	(4, 6, 8)	2-3	(3, 4, 5)	5-7	(2, 4, 8)
2-6	(8, 9, 13)	6-7	(3, 4, 5)	2-5	(7, 8, 9)	6-7	(5, 6, 7)
3-4	(5, 9, 19)			3-4	(10, 15, 20)		

Cost analysis in project management
Crashing the project

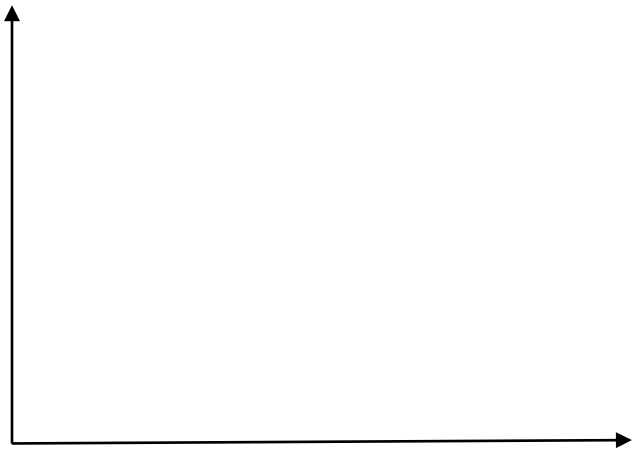
- In many situations, the project manager must complete the project in a time that is less than the length of the critical path.
- So, it is necesseary to determine the allocation of resources that minimizes the cost of meeting the project deadline.
- In practice, Linear programming is commonly used while crashing the Project.
- <u>The parameters:</u>

D_n : Normal completion time

C_n : The cost of completion of activity in normal completion time (D_n)

D_h : Fastest completion time

C_h : The cost of completion of activity in fastest completion time (D_h)



- Suppose that the critical path is determined.
- If we aim to shorten the total duration of project, it is clear that we should select the critical activity or a group of critical activity which has total minimal slope (unit cost).
- The crashing amount is determined by two factors: (1) maximal crashing amount of selected critical activities (2) Free floats of *alternative noncritical paths.*

Alternative noncritical paths

- Starts from the same or backward event with the selected critical activities
- Leads to the critical path(s).

After crashing the project, it is possible to have more than one critical path.

When there doesn’t exist any candidate activity to shorten or a critical path is reached its crashing limit, it is not possible to shorten the total project duration, that is crashing process is over.

It should also be noted that a project may have initially more than one critical path.

Example1: Table on below gives the activities of a project, their predecessor(s), durations and costs.

a) Draw the network for the project.
b) Determine the critical path.
c) Construct the Time Schedule.
d) Obtain programs with minimum costs.

Activities	Predecessor(s)	Durations		Costs	
		Normal	Crashed	Normal	Crashed
A	---	4	2	100	160
B	---	5	5	200	200
C	---	7	4	250	310
D	A	7	5	180	210
E	B	6	6	120	120
F	C	8	4	150	250
G	C	9	6	200	320
H	E, F	4	3	160	220
I	E, F	6	5	240	300
J	E, F, D	10	5	150	350
K	H, G	5	5	250	250

Example2: Table on below gives the activities of a project, their predecessor(s), durations and costs.

a) Draw the network for the project.

b) Determine the critical path.

c) Obtain programs with minimum costs.

Activities	Predecessor(s)	Durations		Costs	
		Normal	Crashed	Normal	Crashed
A	---	5	3	100	150
B	---	4	2	180	200
C	---	6	3	170	200
D	A	7	5	200	260
E	B, C	4	4	120	120
F	B, C	7	7	150	150
G	C	5	3	180	250
H	D, F	8	6	200	300
I	E, G	9	6	100	160

References
[1] *Ahlatçıoğlu, M. , Tiryaki, F., Kantitatif Karar Verme Teknikleri, YTÜ Yayın No: YTÜ.FE.DK-98.0349, İstanbul-1998.*
[2] *H.A.Taha, Operations Research: An Introduction,* Prentice Hall; 9th edition, *Singapore, 2010.*
[3] *Winston, W., Operations Research and Applications and Algorithms. 1990.*

Inventory Models

From this standpoint, an inventory policy answers two questions:

- The inventory problem involves placing and receiving orders of given sizes periodically.
- The basis for the decision is a model that balances the cost of capital resulting from holding too much inventory against the penalty cost resulting from inventory shortage.
- The principal factor affecting the solution is the nature of the demand: deterministic or probabilistic.
- In real life, demand is usually probabilistic, but in some cases the simpler deterministic approximation may be acceptable.

The study of **inventory models** is concerned with two basic questions:

1. How much to order?
2. When to order?

If there exists a **production process**, then this basic questions:

1. How much to produce?
2. When to produce?

The **objective** is to minimize total variable cost over a specified time period.

Inventory Costs

Ordering or/and Setup cost: salaries and expenses of processing an order, regardless of the order quantity.
For example, ordering cost would include the cost of paperwork and billing associated with an order. If the product is made internally rather than ordered from an external source, the cost of labor (and idle time) for setting up and shutting down a machine for a production run would be included in the ordering cost.

Holding or Carrying cost: This is the cost of carrying one unit of inventory for one time period or keeping an item in inventory.
If the time period is a year, the carrying cost will be expressed in dollars per unit per year. The holding cost usually includes storage cost, insurance cost, taxes on inventory, and a cost due to the possibility of spoilage, theft, or obsolescence. Usually, however, the most significant component of holding cost is the opportunity cost incurred by tying up capital in inventory. For example, suppose that one unit of a product costs $100 and the company can earn 15% annually on its investments. Then holding one unit in inventory for one year is costing the company 0.15(100) _ $15. When interest rates are high, most firms assume that their annual holding cost is 20%–40% of the unit purchase cost.

Backorder cost (Shortage or Stockout cost): costs associated with being out of stock when an item is demanded.
When a customer demands a product and the demand is not met on time, a stockout, or shortage, is said to occur. If customers will accept delivery at a later date (no matter how late that date may be), we say that demands may be back-ordered. The case in which back-ordering is allowed is often referred to as the backlogged demand case. If no customer will accept late delivery, we are in the lost sales case. Of course, reality lies between these two extremes, but by determining optimal inventory policies for both the backlogged demand and

the lost sales cases, we can get a ballpark estimate of what the optimal inventory policy should be.
Many costs are associated with stockouts. If back-ordering is allowed, placement of back orders usually results in an extra cost. Stockouts often cause customers to go elsewhere to meet current and future demands, resulting in lost sales and lost goodwill. Stockouts may also cause a company to fall behind in other aspects of its business and may force a plant to incur the higher cost of overtime production. Usually, the cost of a stockout is harder to measure than ordering, purchasing, or holding costs.

Purchase cost: the actual price of the items.
Typically, the unit purchasing cost includes the variable labor cost, variable overhead cost, and raw material cost associated with purchasing or producing a single unit. If goods are ordered from an external source, the unit purchase cost must include shipping cost.

Other costs

MODEL 1: ECONOMIC ORDER QUANTITY (EOQ)

Assumptions:

- Demand is deterministic and occurs at a constant rate.
- Purchase cost per unit is constant (No quantity discount).
- Delivery time (lead time) is constant. (The lead time for each order is zero.)
- No shortages are allowed

Parameters of the model

R : Demand rate (Consumption quantity per unit time)
c_1 : Holding cost ($/unit/unit time)
c_2 : Cost of placing order (Ordering cost) ($/order)

Variables of the model

Q : Order quantity
t : Order-cycle time

Quantity-time figure

Total Inventory Cost = Holding cost + Cost of placing order

Example [Winston, Page 857]:

Braneast Airlines uses 500 taillights per year. Each time an order for taillights is placed, an ordering cost of $5 is incurred. Each light costs 40$ and the holding cost is 8 cent/light/year. Assume that demand occurs at a constant rate and shortages are not allowed.

a) What is the optimal order quantity?
b) How many orders will be placed per year?
c) How much time will elapse between the placement of orders?
d) What is the total inventory cost?
e) What is the total inventory cost per year?

We are given that
$c_2 = \$5$
$c_1 = \$0.08$ / light / year
R = 500 lights / year

Hence, the airline should place an order for 250 taillights each time that inventory reaches zero.

MODEL 2: PRODUCTION MODEL

During the production process, demands are met concurrently. The excess of demand accumulates in stocks. When stocks are in a specified level, which is called the maximum inventory level, the production process is stopped. After this, demands are met from stocks. When the stocks are depleted, the production process starts over again.

Quantity-time figure

Parameters of the model

K : Production rate (K > R)
R : Demand rate
c_1 : Holding cost ($/unit)
c_2 : Production cost – Production preparation cost – cost per production ($/production)

Variables of the model

Q : Production quantity
t : Production -cycle time
t_1 : Production time in a cycle
t_2 : The time length which demands are met from stocks
I_m : Maximum inventory level

Total Inventory Cost = Holding cost + Cost per production

Example:
The productive capacity of a company is 125 units per month. Customer needs 1000 units of product in one year. Unit holding cost per day is 10 TL, cost per production is 67500 TL. Determine the following:
(1 year = 12 months = 360 days)

a) Optimal production amount.
b) Optimal production - cycle time.
c) Optimal production time in a cycle.
d) Maximum inventory level
e) Total inventory cost.
f) The number of production process in a year.
g) Total inventory cost per year.

We are given that
K = 125 units / month
R = 1000 units / year
c_2 = 67500 TL / cycle
c_1 = 10 TL units / day

MODEL 3: ECONOMIC ORDER QUANTITY WITH BACKORDERS (SHORTAGES) ALLOWED

Quantity-time figure

Parameters of the model

R : Demand rate
c_1 : Holding cost ($/unit/ unit time)
c_2 : Ordering cost ($/order)
c_3 : Shortage cost ($/unit/ unit time)

Variables of the model

Q : Order quantity.
t : Order-cycle length.
t_1 : The time length that demands met from stocks.
t_2 : The time length which demands constitute a queue.
I_m : Maximum inventory level (in an order cycle)
S : Maximum shortage level (in an order cycle)

Total Inventory Cost = Holding cost + Shortage cost + Ordering cost

Example [Winston]:

Each year, the Smalltown Optometry Clinic sells 10000 frames for eyeglasses. The clinic orders frames from a regional supplier, which charges $15 per frame. Each order incurs an ordering cost of $50. Smalltown Optometry believes that the demand for frames can be backlogged and that the cost of being short one frame for one year is $15 (because of loss of future business). The annual holding cost for inventory is 30 cent per dollar value of inventory.

a) What is optimal order quantity?
b) What is the maximum shortage that will occur?
c) What is the maximum inventory level that will occur?
d) What is the optimal order cycle length?

R = 10000 units / year.
c_1 = 15 $ / unit / year.
c_2 = 50 $ / order.
c_3 = 0.3 (15) = 4.5 $ / unit / year.

MODEL 4: PRODUCTION MODEL WITH SHORTAGES ALLOWED

Quantity-time figure

Parameters of the model

K : Production rate (K > R)
R : Demand rate
c_1 : Holding cost ($/unit/unit time)
c_2 : Cost per production ($/production cycle)
c_3 : Shortage cost ($/unit/ unit time)

Variables of the model

Q : Production quantity
t : Production - cycle length
I_m : Maximum inventory level
S : Maximum shortage level
t_1 : The time length for accumulating stocks
t_2 : The time length for melting stocks
t_3 : The time length for melting queue.
t_4 : The time length for accumulating queue.

Total Inventory Cost = Holding cost + Shortage cost + Cost per production cycle.

Example: The annual demand of a product is 18000 units. The production rate of the company is 3000 units per month. The cost of setting up a production run is 204000 TL. The holding cost of a unit is 17 TL per month and the shortage cost of a unit is 2000 TL per year. Determine the following:
(1 year = 12 months = 360 days)

a) Optimal production quantity.
b) Optimal production - cycle length.
c) Optimal shortage level.
d) Maximum inventory level.
e) The time length that made production in a cycle.
f) Total inventory cost per year.

R = 18000 units / year.
K = 3000 units / month = 36000 units / year.
c_1 = 17 TL / unit / month = 204 TL / unit / year.
c_2 = 204000 TL / cycle.
c_3 = 2000 TL / unit / year.

WAITING LINE MODELS (QUEUEING MODELS)

MODEL 1: QUEUEING MODEL WITH INFINITE ARRIVAL SOURCE AND A SINGLE SERVER

P_n : Probability of n customers in the system

n : Number of customers in the system (both in queue and service)

This model derives P_n as a function of arrival and service rate (λ and μ). These probabilities are then used to determine the system's measures of performance, such as the average queue length, the average waiting time, and the average utilization of the facility.

The probability P_n is determined by using the transtion-rate diagram in the Figure below.

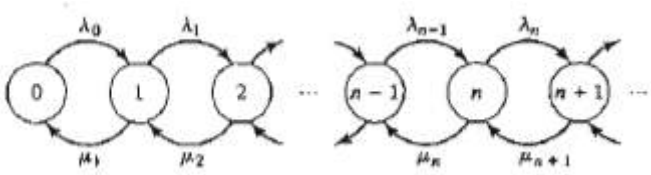


Figure: Transition Diagram

The queueing system is in state n when the number of customers in the system is n.

λ : Arrival rate μ : Service rate

When a system is in state n, the following three states may occur:

i) When a departure occurs at the rate μ, then the system is in state n-1.

ii) When an arrival occurs at the rate λ , then the system is in state n+1.

iii) When no departure or no arrival occurs, then the system is again in state n.

See these states in the last three nodes of transition-diagram.

State 0 can only change to state 1 when an arrival occurs at the rate λ. Notice that μ is undefined in state 0. Because no departure can occur if the system is empty.

The expected rate of flow into and out of state n must be equal. Based on the fact that state n can be changed to states (n-1) and (n+1) only, we get

Expected rate of flow **into** state n $= \lambda . P_{n-1} + \mu . P_{n+1}$

Similarly,

Expected rate of flow **out** of state n $= \lambda . P_n + \mu . P_n$

Equating these two rates, we get the following **balance equation**:

$\lambda . P_{n-1} + \mu . P_{n+1} = \lambda . P_n + \mu . P_n$, n = 1,2,…

The balance equation associated with n = 0, is

$\lambda . P_0 = \mu . P_1$ [$P_{n-1} = 0$ for n = 0]

The balance equations are arranged recursively in terms of P_0 as follows:

Example: In a factory, the average time of malfunction (broken) of a machine is 12 minutes and the average time of repairing of it is 8 minutes.

i) a) At any time, how many machines are out of production?

b) How long does it take to go reproduction of a broken machine?

c) What is the probability of being out of work of repairman.

ii) Find the answers of a, b and c in case of increasing of malfunctions at a rate %20.

MODEL 2: QUEUEING MODEL WITH INFINITE ARRIVAL SOURCE, A SINGLE SERVER WHICH HAS A QUEUE WITH FINITE LENGTH

The system can contain a maximum number of m customers at any time. While single server provides service to one cutomer, the length of queue can not exceed m-1.

λ : Arrival rate μ : Service rate

Balance equations

For $n = 0 \rightarrow \lambda \, . \, P_0 = \mu \, . \, P_1$